ADL Lite: An Event-First Capability-Lifecycle Registry for LLM Agent Ecosystems

Anonymous Authors
Affiliation withheld for peer review

Abstract

ADL Lite is an event-first, Markdown-native capability-lifecycle audit system for LLM agent ecosystems under a collaborative non-Byzantine trust model. Drawing on Wittgenstein's dictum that "the world is the totality of facts, not of things," the system treats capabilities as append-only, hash-linked EventChains—serialized information content entities—whose lifecycle state (status, confidence, and validators) is derived exclusively from event history via deterministic functions. A two-level ontological account distinguishes the EventChain as an occurrent process (what happens) from the EventChain-record as an information content entity (what is recorded), thereby resolving the process–record ambiguity inherent to event-sourced systems. We map categories to BFO, DOLCE, and UFO, and evaluate identity conditions through the OntoClean lens. This paper presents: a formal specification with nine machine-verified algebraic properties in Coq 8.18.0 (Theorems 1–7, 9, and Lemma E2) and one Python-level computational property (Theorem 8); a decidable precondition language with $O(1)$ per-rule evaluation; architectural enhancements for 10^4-agent contention, incremental verification, and CRDT $N{\geq}3$ multi-way merge; and empirical validation on 201 EventChains (9,300 events) demonstrating linear-time verification at 5×10^5 events (10^6 projected), zero integrity failures under 10K concurrent agents, and >0.80 vector index recall. These results demonstrate that event-first operational ontology—combining cryptographic content integrity with deterministic lifecycle derivation—is computationally feasible and deployable without external reasoners or triple stores.

Keywords: agent audit; capability registry; lifecycle audit; event sourcing; multi-agent systems; provenance; operational ontology

1 Introduction

1.1 Background: The Agent Capability Governance Gap

LLM agents expose *capabilities*: tools, APIs, knowledge domains, and interaction protocols. A capability is a claim—"I can retrieve weather data," "I can execute SQL," "I can reason about financial risk"—that other agents and operators must evaluate before delegation. Yet no existing system records what a capability does, how it evolves, or whether it is trustworthy under a lightweight, auditable framework. Capability claims are scattered across README files, API documentation, and prompt templates; they are revised without audit trails, deprecated without formal process, and forked without attribution.

Existing governance layers. Recent work addresses adjacent aspects of agent governance. KYA provides trust-layer permissions with HMAC-chained integrity but omits capability lifecycles Quadri [2026]; AgentSafe requires heavyweight infrastructure for architecture-level governance Khan et al. [2025]; Talukdar et al. produce static ontologies without lifecycle governance; Agent Traces notes

that a unified trace schema remains unrealized; and SafeAgent governs safety assessment, not capability registration Talukder et al. [2026], Wang et al. [2026], Liu et al. [2026]. Runtime safety systems (SafeHarness Lin et al. [2026], OpenClaw PRISM Li [2026]) protect execution-time safety but do not register capabilities. A detailed comparison is in Section 2.1.

Defining the gap. No existing system combines (a) Markdown-native authoring, (b) cryptographic hash chaining, (c) lifecycle state machines, and (d) pip-installable deployment.

Defining "operational ontology." We use *operational ontology* to mean an ontology in which lifecycle-transition semantics—preconditions, state derivation, and deterministic merge rules—are co-located with the EventChain data structure in a single lightweight package. The ActionExecutor enforces these semantics without external workflow engines, OWL reasoners, or triple stores. This distinguishes our usage from OBO Foundry practice (centralized expert curation, OWL 2 DL reasoners, external release pipelines) Smith et al. [2007] and from executable ontologies (operational constraints enforced by external workflow engines, separate from the data structure) Guizzardi et al. [2024]. ADL Lite inverts this separation: the ontology *is* the executable structure, and the ActionExecutor enforces constraints directly over the event sequence. The distinction between the *causal origin* of a concept (traceable to its genesis REGISTER event, an occurrent) and its *ontological bearer* (the EventChain-record as an information content entity) is central to our two-level account. We develop this distinction in §3.2.6.

Ontological novelty of the event-first stance. ADL Lite's contribution is not the philosophical thesis that events are fundamental—already established by Wittgenstein, BFO, DOLCE, and UFO Wittgenstein [1922], Arp et al. [2015], Gangemi et al. [2002], Guizzardi [2005]—but its *operationalization*: translating an event-first commitment into a deployable, Markdown-native, cryptographically enhanced capability-lifecycle mechanism. This distinguishes our pragmatic pluralism from occurrence-only ontologies Al-Fedaghi [2024], yet we retain `Concept` as a generically dependent continuant.

1.2 Research Gap: Capability Lifecycle Governance as Ontological Analysis

The research gap is: *How can an event-first capability system achieve lifecycle traceability and multi-agent deterministic state derivation without object-first mutable state, expert curation, or heavyweight tooling?* The engineering dimension concerns deployment friction: existing platforms require specialized expertise and infrastructure that do not scale with agent-generated artifacts, and LLM agents lack programmatic access to proprietary ontology interfaces.

Ontological grounding of the event-first stance. The ontological dimension is not merely a design preference but reflects a *categorical distinction* between continuants and occurrents. In BFO, a generically dependent continuant (GDC) such as a capability cannot have mutable properties stored as fields of the entity itself, because a GDC has no physical bearer in which such properties could inhere Arp et al. [2015]. DOLCE likewise treats social entities as cognitive constructs whose properties are projected from historical events rather than discovered as intrinsic qualities Gangemi et al. [2002]. Consequently, a capability's lifecycle status (e.g., `validated`) cannot be an intrinsic property of the capability record; it must be a *derivative* computed from the history of events that have occurred to that capability. An object-first approach—storing status as a mutable field—commits the category error of treating a GDC as if it were an independent continuant with

inherent qualities. ADL Lite's event-first design is therefore *ontologically grounded*: it respects the categorical distinction between continuants and occurrents by deriving all mutable state from the event sequence. We do not claim that event-first modeling is the only ontologically valid approach—alternative BFO/IAO-compatible representations (e.g., mutable status fields updated by events) are theoretically possible—but we argue that the event-first stance provides superior audit properties and operational clarity for capability-lifecycle governance.

Scope of the trust model. The present paper is scoped to a **collaborative non-Byzantine trust model**: agents are assumed to be cooperative but not authenticated, and the cryptographic hash chain detects tampering post-hoc rather than preventing it. This scope is deliberate—it separates the ontological and formal foundations (this paper) from adversarial-resistance engineering (future work). No prior work has systematically operationalized event-first semantics—combining cryptographic content integrity, deterministic lifecycle derivation, and declarative preconditions—in a single lightweight system under this trust model. This paper presents ADL Lite, an event-first operational ontology in which capabilities are modeled as append-only, hash-linked EventChains and all lifecycle state is derived from event history. The primary contribution is an **ontological analysis** situating ADL Lite within foundational ontologies (BFO, DOLCE, and UFO), formalizing its categories and identity conditions, and proving key algebraic properties of its derivation semantics.

1.3 Contributions

The contributions are organized around three axes: ontological analysis, formal semantics, and architectural verification.

Contribution 1: Ontological analysis of event-first capability recording and deterministic derivation. We analyze ADL Lite's core categories through the lens of BFO, DOLCE, and UFO:

- `Event` corresponds to the BFO *occurrent* and DOLCE *perdurant*.
- `Concept` (a capability definition) is a BFO *generically dependent continuant* or DOLCE *non-physical object*.
- L3 relations instantiate UFO *relators* with event-based lifecycles.
- Identity conditions for events, capabilities, and chains, and three forms of ontological dependence (historical, generic, and mutual).

Contribution 2: Formal derivation semantics with proven properties. We provide:

- An event alphabet $\Sigma = \Sigma_{\text{life}} \cup \Sigma_{\text{comm}}$ partitioned into lifecycle and communication events.
- A well-formedness predicate $\text{WF}(C)$ over EventChains.
- Status derivation $\delta(C)$ and confidence aggregation $\gamma(C)$ as deterministic functions (δ in $O(n)$, γ in $O(1)$).
- Fork and fork-determinism semantics.
- Nine proven properties: determinism (Theorem 1), fork determinism (Theorem 2), transition monotonicity (Theorem 3), confidence boundedness (Theorem 4), confidence monotonicity under non-decreasing validation (Theorem 5), status–confidence consistency (Theorem 6), well-formedness preservation (Theorem 7), precondition evaluation complexity (Theorem 8), and CRDT merge convergence (Theorem 9).

- An analysis of expressive power relative to Event Calculus and Description Logic.

Contribution 3: Architectural verification of the capability audit system implementation. We evaluated ADL Lite on 201 EventChains derived from the IBM Anti-Money Laundering (AML) HI-Small dataset (9,300 events), verifying the implementation against the eight machine-verified algebraic properties in Coq 8.18.0 (Theorems 1–7, 9) and the one Python-level computational property (Theorem 8). The evaluation confirmed:

- Zero integrity failures across all chains, including the longest chain (300 events).
- 100% accuracy on 3,382 exhaustive status derivation test cases (all 13^3 event-type combinations of length ≤ 3 plus communication-event interleavings).
- Perfect precision and recall (P = R = F1 = 1.0) on 141 precondition enforcement test cases (19 base + 72 boundary + 50 concurrency).
- Linear scalability: 20,955 events per second on modest hardware.

These results validate the event-first ontological architecture—deterministic derivation semantics, hash-linked content integrity, and nine proven algebraic properties—as computationally feasible under a collaborative non-Byzantine trust model. Four additional experiments (E27–E30) validate scalability to 5×10^5 events with split-lock concurrency and incremental verification (with 10^6 projected), integrity under 10K-agent contention, FAISS vector index recall >0.80, and safe LLM-driven canonicalization. Multi-agent collaboration simulation (E17) is already operational; domain-level evaluation with human experts (E5) is planned (expert recruitment and scoring protocol completed). The present paper establishes the **ontological and formal foundations** of capability-lifecycle recording and deterministic state derivation.

1.4 Implementation Status and Roadmap

ADL Lite is implemented as a pip-installable Python package (v0.6.0-alpha in Git, v0.2.0 on PyPI) operating on Markdown files in standard Git repositories.

Implemented and validated. The core EventChain data structure (append-only, SHA-256 hash-linked events), deterministic derivation semantics (δ and γ), and the decidable precondition language (8 comparators, closed action registry) are fully implemented and validated by the experiments reported in this paper (E1–E4, E6, E13–E16, E20–E23, E27–E30). The L1–L4 document model, transition engine, and integrity verification (VerifyIntegrity) are functionally complete. As of v0.6.0-alpha, the EventChain employs a split-lock concurrency model (Appendix A.6) that enables 10^4-agent contention without data races in the E28 benchmark, an incremental verification cache for $O(1)$ post-append checks (E27), and zstd+msgpack cold storage compression. The REST API, MCP server, SHACL validation, and Neo4j adapter (Appendix A.6) provide programmatic access, agent integration, document conformance checking, and scalable graph queries. The trust model is collaborative audit: non-Byzantine agents, self-declared string identifiers, and CRDT lattice semantics (LUB for status, G-Counter for confidence) for fork resolution.

Implemented but not exercised in experiments. Additional features present in the codebase but not exercised in the reported experiments include: L2 Markdown template validation with Pydantic-governed sections ([Observation], [Reasoning], [Conclusion]); ARCHIVE events with cold storage migration; per-actor linear confidence calibration (`calibration.py`); an Ed25519 key registry with Git commit signature binding; and Merkle transparency anchors for $O(\log n)$ batch verification (Appendix A.6).

Planned for future releases. Future work covers W3C Linked Data Proofs (DIDs), an optional BFT transport layer, domain-level evaluation with AML experts (E5), unbounded machine-checked proofs (TLA$^+$/Coq), OWL 2 DL axiomatization, and adapters for emerging agent registry standards (A2A, AGNTCY, NANDA, Entra Verifiable Credentials).

We scope the audit claims in this paper to the collaborative-audit trust model (non-Byzantine agents, self-declared string identifiers). Advanced authentication and adversarial robustness features are planned for future releases.

1.5 Paper Organization

Section 2 positions ADL Lite within the emerging agent audit landscape and surveys related work in foundational ontologies, ontology engineering, and event-centric modeling. Section 3 presents the core ontological analysis: categories, identity conditions, ontological dependence, and alignment with BFO, DOLCE, and UFO. Section 4 describes the architecture, formal semantics, and comparison with Event Calculus and Description Logic. Section 5 reports architectural verification of correctness. Section 6 discusses implications, limitations, and future work. Section 7 concludes.

2 Related Work

2.1 Agent Governance Landscape

LLM agent proliferation has created an urgent need for governance beyond traditional ontology engineering. ADL Lite fills a gap that trust, safety, and provenance layers overlook: a lightweight, event-sourced, tamper-evident registry of agent capabilities and their lifecycle states.

AgentHub Pautsch et al. [2025] proposes a registry layer for agent sharing with discovery, workflow integration, and lifecycle transparency, but it remains a research agenda without an implemented lifecycle state machine or deterministic derivation semantics. ADL Lite extends beyond AgentHub's agenda by providing an implemented registry with formal lifecycle semantics (register $\rightarrow$ validate $\rightarrow$ deprecate), cryptographic hash chaining, and deterministic status derivation.

KYA Quadri [2026] verifies agent identity via HMAC chains but does not audit capability lifecycles. **Zhou** Zhou [2026] proposes cryptographic binding and reproducibility verification for agent tool use, addressing tool-call provenance at the invocation level; ADL Lite audits the capability lifecycle that such tool use presupposes. **AgentSafe** Khan et al. [2025] offers comprehensive architecture governance but requires heavyweight infrastructure and lacks Markdown-native authoring. **From Agent Traces to Trust** Wang et al. [2026] concludes that "a unified trace schema is still needed"—ADL Lite provides a unified capability-lifecycle trace schema. **Talukdar et al.** Talukder et al. [2026] demonstrate multi-agent ontology engineering but produce static artifacts without lifecycle audit. **SafeAgent** Liu et al. [2026] operates at the interaction layer rather than the artifact layer. Runtime safety systems (SafeHarness Lin et al. [2026], OpenClaw PRISM Li [2026]) and runtime infrastructure (Institutional AI Syrnikov et al. [2026], GaaS Gaurav et al. [2025], COMPASS Dessureault et al. [2026], UALM Prakash et al. [2026]) consume capability provenance for policy decisions; ADL Lite occupies the *registry layer* between permissioning and runtime enforcement.

Table 1 compares these systems.

Runtime safety systems (SafeHarness Lin et al. [2026], OpenClaw PRISM Li [2026]) govern execution-time safety rather than capability registration; they complement ADL Lite by leveraging the capability metadata that ADL Lite produces.

Table 1: Comparison of agent governance systems.

Dimension	KYA	AgentSafe	Agent Traces	Talukdar et al.	SafeAgent	ADL Lite
Governance Scope	Trust / permissions	Architecture safety	Provenance / trace	Ontology production	Protocol safety	Capability lifecycle
Lifecycle Awareness	None	Design-time only	None	None	None	Full (register →validate → deprecate)
Deployment Weight	Lightweight	Heavyweight	Survey / framework	Research pipeline	Lightweight	Very lightweight (pip + Git)
Tamper-evidence	HMAC chain	Audit logs	Trace schema (proposed)	Manual QA review	Sandbox checks	SHA-256 hash chain + preconditions
Agent-Native	Yes (15+ adapters)	Yes (design-time)	Yes (provenance)	Yes (multi-agent)	Yes (runtime)	Yes (Markdown-native, multi-agent)

Agent capability standards. Emerging agent standards—the Model Context Protocol (MCP) Anthropic [2024], Google's Agent2Agent (A2A) Google [2025], and the Linux Foundation's AGNTCY Open Agent Schema Framework Linux Foundation [2025]—provide agent capability registration and discovery, but they do not provide lifecycle audit semantics: none offers an append-only, tamper-evident chain of capability states, LUB-derived status, or precondition enforcement over lifecycle transitions. General-purpose orchestration frameworks (LangChain LangChain, Inc. [2023], LlamaIndex LlamaIndex, Inc. [2023], the OpenAI Agents SDK OpenAI [2024]) supply tool-calling and workflow primitives but likewise treat capability metadata as ephemeral runtime state rather than an auditable lifecycle. This is precisely the layer that ADL Lite adds on top of such standards and frameworks (Section 6.8).

2.2 Foundational Ontologies and Event-First Semantics

ADL Lite's `Event` corresponds to BFO's *process* Arp et al. [2015], DOLCE's *perdurant* Gangemi et al. [2002], and the event/action distinction in UFO Guizzardi [2005]. ADL Lite adopts *pragmatic pluralism*: BFO as conceptual guide, DOLCE as design principle, and UFO as modelling framework. A detailed cross-mapping appears in Section 3 (Table 3).

We next review knowledge graph construction and ontology engineering systems that provide the broader tooling context for ADL Lite's document-native approach.

2.3 Knowledge Graph Construction and Ontology Engineering

The Semantic Web vision Berners-Lee et al. [2001] was operationalised through OWL 2 W3C OWL Working Group [2012]; toolchains enable ontology authoring at scale, but reasoners impose computational costs that render them ill-suited to rapidly evolving knowledge bases Glimm et al. [2014], Sirin et al. [2007], Hogan et al. [2021]. The OBO Foundry provides centralised versioning Smith et al. [2007]; lightweight alternatives (Schema.org Guha et al. [2016], JSON-LD Sporny et al. [2020], SHACL W3C [2017], ShEx W3C ShEx Community Group [2019]) and Markdown tools Foam Contributors [2020], Lin [2020], Obsidian Team [2024] lower the barrier to entry but either remain bound to RDF or lack machine-checkable coordination. Industry-scale knowledge graphs Noy et al. [2019] and collaborative knowledge bases such as Wikidata Vrandečić and Krötzsch [2014]

show the curation challenge at scale. A systematic literature review of LLM-based ontology engineering Li et al. [2025] catalogues this landscape; LLM4ACOE Soularidis et al. [2025], Sim-HCOME Doumanas et al. [2025], and Ontology-Constrained Neural Reasoning Tuan and Sanyal [2026] generate OWL automatically but without lifecycle audit. ADL Lite differs from these systems in three respects: (i) *authoring paradigm:* ADL Lite uses Markdown-native documents rather than OWL/RDF output; (ii) *audit scope:* ADL Lite provides built-in lifecycle state machines (register $\rightarrow$ validate $\rightarrow$ deprecate) rather than static ontology production; (iii) *deployment weight:* ADL Lite is pip-installable and operates on Git repositories, whereas LLM4ACOE and Sim-HCOME require multi-agent orchestration pipelines. These are complementary trade-offs, not competitive disadvantages: LLM4ACOE/Sim-HCOME excel at automated ontology generation with ReAct reasoning traces, while ADL Lite excels at lightweight, auditable management of existing capabilities. Table 2 presents the dimension-level comparison.

Table 2: Dimension-level comparison: ADL Lite vs. proximate systems.

Dimension	**LLM4ACOE / Sim-HCOME**	**Blocklace (2024)**	**ADL Lite**
Primary output	OWL 2 DL ontology	Generic BFT-CRDT DAG	Markdown-native capability registry
LLM agent role	Multi-agent ReAct ontology generation	None (transport layer)	Actor in lifecycle events
Lifecycle audit	None (static ontology)	None (application-agnostic)	Register $\rightarrow$validate $\rightarrow$ deprecate
Cryptographic integrity	None (manual QA review)	SHA-256 + BFT consensus	SHA-256 hash chain
Deterministic state	External reasoner required	None (raw data structure)	Built-in $\delta(C)$, $\gamma(C)$
Deployment	Research pipeline (multi-agent)	Library (distributed systems)	`pip install` + Git

Comparison with FAOS. The Foundation AgenticOS (FAOS) platform Tuan and Sanyal [2026] proposes a three-layer enterprise ontology (Role, Domain, Interaction) for grounding LLM agents in organisational contexts. Whereas FAOS's layers govern *agent roles* and *domain interactions* within an enterprise, ADL Lite's L1–L4 layers audit *capability lifecycle events* across decentralized multi-agent ecosystems; FAOS does not provide cryptographic hash chaining, deterministic status derivation, or append-only event sourcing. The two are complementary: ADL Lite could consume FAOS-generated ontologies and audit their lifecycle through EventChains.

Blocklace Almeida and Shapiro [2024] is a general-purpose BFT-CRDT with a cryptographic hash DAG, providing Byzantine fault tolerance and equivocation detection at the transport layer. ADL Lite is complementary: Blocklace provides the *transport* (distributed consensus over a DAG of events), while ADL Lite provides the *application semantics* (deterministic status derivation $\delta(C)$, confidence aggregation $\gamma(C)$, and declarative preconditions); the proposed Phase 3 architecture uses Blocklace's DAG beneath ADL Lite's lifecycle semantics. SEO 2025 Boldachev [2025] uses causal DAGs with reactive guards for distributed multi-agent event contribution; ADL Lite's linear hash chain is less expressive but offers $O(1)$ per-event append latency and deterministic $O(n)$ verification—a deliberate expressivity–simplicity trade-off.

2.4 Provenance, Trust, and Verifiable Publishing

W3C PROV-O W3C [2013] is the W3C standard provenance interchange model, surveyed in detail by Moreau and Groth Moreau and Groth [2013]; the earlier named-graph account Carroll

et al. [2005] already identified graph provenance as a basis for trust on the Web. Linked Data Proofs W3C Verifiable Credentials Working Group [2024] and RDF-star W3C RDF-star Working Group [2024] provide statement-level annotation but lack both cryptographic integrity and lifecycle audit. Secure-provenance research Braun et al. [2010], Hasan et al. [2009] formalises adversarial tamper-evidence for provenance chains but targets low-level data lifecycle management rather than capability lifecycle audit. Decentralised identifiers and verifiable credentials (DID/VC) Mazzocca et al. [2024] provide strong actor authentication but weak content binding: a DID identifies *who* made a claim, not *what* the claim is about. ADL Lite complements DID-based identity by binding the genesis hash to the specific event (Section 6.6). Git-native systems (RO-Crate Soiland-Reyes et al. [2022], DataLad Halchenko et al. [2021], TerminusDB TerminusDB/DFRNT [2024]) share immutable history but lack document-native authoring. Full mappings to standards and loss analysis are in Appendix F.

2.5 Event-Centric Ontologies

CIDOC CRM ICOM-CIDOC [2014], SEM/LODE van Hage et al. [2011], Shaw et al. [2011], and RDF Stream Processing W3C RDF Stream Processing Community Group [2013], Barbieri et al. [2009], Le-Phuoc et al. [2011] provide event vocabularies but lack both constraint mechanisms and retrospective verification. Boldachev's SEO Boldachev [2025], Al-Fedaghi's occurrence-only paradigm Al-Fedaghi [2024], and Guizzardi's UFO-B Guizzardi et al. [2024] offer alternative event models. CRDTs Shapiro et al. [2011], Martin et al. [2020], Nicolai and Dadam [2021], Gruss et al. [2023], Nieto et al. [2024] and Blocklace Almeida and Shapiro [2024] reconcile concurrent updates; ADL Lite complements these with append-only event chains and deterministic lifecycle audit. Real-time collaborative ontology editing using distributed version control (OntoEditor, which uses Operational Transformation rather than CRDTs) provides a related but distinct approach to multi-author ontology development Hemid et al. [2024]. ADL Lite's EventChain extends the event-sourced aggregate root pattern Young [2010], Fowler [2005] with ontological analysis, SHA-256 chaining, and Markdown-native authoring, while its separation of append-only event streams from derived query state follows the command-query responsibility segregation (CQRS) pattern Young [2012].

2.6 Transparency Logs and Software Supply Chain

Certificate Transparency (CT) Laurie et al. [2013], Crosby and Wallach [2009], Sigstore Rekor Denker et al. [2022], in-toto Torres-Arias et al. [2019], and SLSA Team [2023] provide tamper-evident registries for software artifacts, certificates, and supply-chain provenance. These systems use Merkle-tree or hash-DAG structures to enable $O(\log n)$ inclusion proofs and $O(1)$ consistency proofs, making them highly scalable for globally distributed verification with millions of entries. ADL Lite's linear SHA-256 chain, by contrast, requires $O(n)$ sequential verification for full integrity checking—a deliberate design trade-off: linear chains are simpler to implement in Markdown-native environments, produce human-readable audit trails, and require no external gossiping or witness infrastructure. Merkle trees excel at proving that an entry exists in a large append-only log; ADL Lite excels at auditing the lifecycle of a single capability through deterministic status derivation and confidence aggregation. The two are complementary: future Phase 3 integration (FW4, FW9) may adopt Merkle-tree batching for cross-repository verification while retaining per-chain lifecycle semantics. Blockchain-based provenance Akbarfam and Maleki [2024] and the records-management lifecycle view of distributed ledgers Barbereau et al. [2026] reinforce this design: Barbereau et al. reconceptualize blockchain systems as record-management systems with a seven-stage lifecycle (ISO

15489-1:2016), echoing the register → validate → deprecate lifecycle that ADL Lite makes native and deterministic.

Quantitative comparison with nanopublications and Git-native provenance. Supplementary Table S19 compares ADL Lite with two related lightweight approaches on operational metrics relevant to capability-lifecycle audit.

Nanopublications with trusty URIs Groth et al. [2023] provide $O(1)$ hash verification for individual assertions but lack lifecycle semantics: each nanopublication is a standalone, immutable statement with no formal status derivation or precondition-governed transitions. Recent NanoPub-Jelly streaming formats (2024) support chained nanopublications with temporal ordering, though without the lifecycle audit semantics (register → validate → deprecate) and deterministic status derivation (δ, γ) that ADL Lite provides. Git-native systems (RO-Crate, DataLad) provide commit-level provenance but treat the entire commit as a unit of change, making it impossible to trace the lifecycle of an individual capability within a repository containing many concepts. ADL Lite trades $O(1)$ verification for $O(n)$ chain scanning in exchange for per-event lifecycle audit: each event (REGISTER, VALIDATE, DEPRECATE, FORK) is individually typed, preconditioned, and auditable. The migration path to nanopublications is bidirectional: ADL Lite events export to RDF-star quoted triples (Appendix F), and nanopublication assertions can be imported as RELATE events with SHA-256 content-addressed references. The migration path to Git-native provenance is direct: ADL Lite EventChains are stored as Markdown files in standard Git repositories, with commit signatures providing repository-level integrity and event hashes providing concept-level integrity.

2.7 Deep Comparison with Nanopublications

Nanopublications with trusty URIs Groth et al. [2023] are the closest Semantic Web analogue to ADL Lite's per-event integrity and provenance tracking; Supplementary Table S21 summarises the dimension-level comparison.

Immutability granularity: per-capability vs. per-statement. Nanopublications enforce immutability at the *statement* level: each is a single signed RDF assertion with a trusty URI containing a SHA-256 hash; retraction is a new nanopublication linked via `prov:wasDerivedFrom`. ADL Lite enforces immutability at the *capability* level: an EventChain is an append-only sequence of typed events that collectively govern a single concept's lifecycle, and the chain's state is derived from the entire history—appropriate for capability governance, where the *process* of validation, deprecation, and forking matters as much as any individual assertion.

Key structure: trusty URI vs. genesis hash. A nanopublication trusty URI encodes the publisher's RSA key, creation timestamp, and the content hash; the URI is immutable, so renaming requires publishing a new nanopublication. In ADL Lite, the `concept_id` is a human-readable alias while the genesis hash (SHA-256 of the first event) is the operational identifier—a content-derived property, not part of the URI—allowing concepts to be renamed (via a fork) while preserving the cryptographic identity of the original chain.

Lifecycle semantics: absent vs. native. Nanopublications have no built-in lifecycle semantics: an assertion's status is a consumer inference, not encoded in the format. ADL Lite natively encodes lifecycle events (`REGISTER`, `VALIDATE`, `DEPRECATE`, `FORK`, `ARCHIVE`) as first-class chain entities with deterministic status derivation $\delta(C)$ and confidence aggregation $\gamma(C)$; the status of a concept is a mathematically defined function of the chain's history, not a consumer inference.

Worked example: capability deprecation. Deprecating a validated capability in a nanopublication pipeline requires publishing a new "deprecated" nanopublication, linking it via `prov:wasDerivedFrom`, and relying on consumers to query both and infer supersession—there is no

mechanism to enforce that the link is created or interpreted correctly. In ADL Lite, deprecation is a single `DEPRECATE` event appended to the existing chain: typed, preconditioned (requires current status `validated`), hash-linked, and immediately making $\delta(C) =$ `deprecated` for all consumers, with no ambiguity and a complete audit trail.

Export/import mapping. The migration path is bidirectional. ADL Lite exports to nanopublication-like structures via RDF-star—each event becomes a quoted triple with its event hash as provenance, e.g.:

```
<<␣:gradient-explosion␣:isomorphic-to␣:vanishing-gradient␣>>
    prov:wasGeneratedBy :evt-gradient-explosion-validate-001 ;
    adl:eventHash "sha256:a3f2..." ;
    adl:confidence 0.91 .
```

Conversely, a nanopublication assertion can be imported as an ADL Lite `RELATE` event with the trusty URI hash stored as a content-addressed reference, enabling ADL Lite to act as a lifecycle audit layer *on top of* existing nanopublication repositories.

2.8 Ontology Registry Comparison

ADL Lite occupies a distinct position relative to mature ontology and semantic-artifact registries that provide curation, versioning, and governance for ontologies and vocabularies (Supplementary Table S22).

BioPortal and OntoPortal Whetzel et al. [2011], Jonquet et al. [2018] are the de facto infrastructure for ontology hosting in the biomedical domain. They offer versioning via release tags, collaborative submission, and Web-based curation workflows, but they do not record the *process* of validation: a submitter uploads an OWL file, a curator reviews it, and the ontology is published—the curation *decision* is not itself an event in the ontology's lifecycle. FAIRsharing Sansone et al. [2022] curates standards and databases but focuses on metadata discovery rather than lifecycle audit. ADL Lite complements these registries by providing (i) event-sourced lifecycle audit, in which each validation, deprecation, and fork is a hash-linked event; (ii) multi-agent native authoring with deterministic derivation semantics (δ, γ); and (iii) content-addressability alongside provenance-based identity, enabling automated duplicate detection. The migration path is bidirectional: ADL Lite concepts export to OWL 2 DL (Appendix A.1), and existing BioPortal ontologies can be imported as initial REGISTER events with their URI preserved as a `RELATE` target.

Release-based versus event-sourced versioning. Ontology versioning on the Semantic Web has long been recognised as a hard problem Klein and Fensel [2001]; BioPortal and OntoPortal use *release-based* versioning: an ontology is uploaded as a monolithic file, and each release is a snapshot with a version tag (e.g., "v2024-01"). This model captures the *state* of the ontology at discrete points but does not capture the *process* by which that state was reached: who proposed a change, who validated it, and what evidence was provided. ADL Lite's *event-sourced* model treats every lifecycle action (registration, validation, deprecation, fork) as an atomic event, enabling fine-grained provenance and deterministic state reconstruction from the event history. Release-based versioning is appropriate for mature, slowly evolving ontologies; event-sourced versioning is necessary for rapidly evolving, multi-agent capability ecosystems where decisions must be auditable and reversible.

2.9 Positioning

Palantir Foundry unifies "semantic elements" (nouns) with "kinetic elements" (verbs) Palantir Technologies [2024], but requires enterprise-scale deployment and proprietary tooling, and lacks

LLM-native authorship. ADL Lite offers pip-installable deployment, Markdown-native authoring, and multi-agent deterministic derivation. Full comparison tables are in Appendix F.

Within the open-source, pip-installable, Markdown-native tool space, to the best of our knowledge, ADL Lite uniquely combines four properties: (a) document-native authoring in plain Markdown, (b) built-in tamper detection through SHA-256 hash chaining, (c) lifecycle state machines with declarative preconditions, and (d) pip-installable deployment requiring no enterprise infrastructure. This enables multiple LLM agents to propose, challenge, and refine concepts within a shared document-native ontology, with the EventChain providing the audit trail and deterministic derivation mechanism that renders decentralized authorship accountable.

3 Ontological Analysis of ADL Lite

3.1 Philosophical Foundation: Event-First as Ontological Commitment

ADL Lite inverts the object-first stance of conventional operational ontologies Palantir Technologies [2024], Lin [2020], following the programme of formal ontology as an analytical tool for information systems Guarino [1998]. Drawing on Wittgenstein's *Tractatus* §1.1—*"Die Welt ist die Gesamtheit der Tatsachen, nicht der Dinge"* Wittgenstein [1922], Young [2010]—the system treats the occurrent as a fundamental primitive, and treats lifecycle statuses as institutional facts whose reality is constituted by recorded events Searle [1995]. This yields three governing principles: (1) **No implicit state changes.** Every mutation is an explicit event. (2) **Events are occurrents.** They happen, but do not endure Arp et al. [2015], Gangemi et al. [2002]. (3) **All derived properties are historical.** Status, confidence, and validators are computed by deterministic functions over the event sequence and are never stored.

3.2 Categories and Taxonomy

We analyze ADL Lite's categories with reference to BFO Arp et al. [2015], DOLCE Gangemi et al. [2002], and UFO Guizzardi [2005]. Table 3 presents the cross-mapping.

3.2.1 Event as Occurrent/Perdurant

In BFO, an *occurrent* unfolds in time and is ontologically distinct from *continuants* Arp et al. [2015]; in DOLCE, the corresponding category is *perdurant* Gangemi et al. [2002]. ADL Lite's `Event` is both: a temporal entity with a definite beginning and end, no spatial location, and no endurance. The serialized record is an information content entity (ICE), not the event itself. In UFO, `REGISTER` and `VALIDATE` are *actions* (intentional), whereas `RELATE`, `EVIDENCE`, and `SEAL` are *events* (structural changes) Guizzardi [2005].

3.2.2 Concept as Generically Dependent Continuant

An ADL Lite concept is a *generically dependent continuant* (GDC) in BFO terms Arp et al. [2015]: it depends on some continuant for its existence, but not on any *specific* one. Following the standard BFO/IAO analysis of information artifacts Smith et al. [2010, 2012], the dependence chain is:

$$\text{Concept (GDC)} \xrightarrow{\text{g-depends on}} \text{EventChain-record (ICE)} \xrightarrow{\text{concretized\_by}} \text{Markdown\_file (IBE)}$$

This is the canonical BFO pattern: a GDC (the concept) generically depends on an information content entity (ICE, the serialized chain record), which is itself concretized by an information-bearing

Table 3: Cross-mapping of ADL Lite categories to upper ontologies. Entries marked "—" indicate intentional non-alignment; those marked "≈" denote approximate correspondence.

ADL Lite	**BFO 2.0**	**DOLCE**	**UFO**	**Note**
Event	`occurrent`	`perdurant`	`Event`	Temporal entity; happens; no endurance
EventChain	`process`	`accomplishment`≈	`Complex Event`	Ordered event sequence; *process* layer (see record layer below)
EventChain-record	`information content entity`	`information object`	`Information Entity`	Serialized ICE; *two-level account* in §3.2.6
Concept	`generically dependent continuant`	`non-physical object`≈	`Conceptual Entity`	Depends on EventChain-record as ICE bearer (not occurrent)
Relation (L3)	`relational quality`	`quality`	`Relator`	Mediates between concepts
Action (L4)	`planned process`	`action`	`Action`	Intentional occurrent with preconditions
Actor	`agent role`	`physical agent`≈	`Agent`	Role realized in material entity (human or software)[1]
Payload	`information content entity`	`information object`	`Information Entity`	Structured data carried by event
Hash	`generically dependent continuant`	—	—	Cryptographic identifier; no physical bearer

entity (IBE, the Markdown file or database row) Smith et al. [2012], Limbaugh et al. [2024]. The GDC does not depend on the occurrent process of events (the EventChain-process), because BFO's axiom D5 prohibits GDCs from depending on occurrents Arp et al. [2015]. The concept's causal origin is the genesis `REGISTER` event (an occurrent), but its ontological bearer is the ICE record (a continuant)—a distinction between *causal origin* and *ontological dependence* that Fine Fine [1995] and BFO both endorse. The concept's identity is determined by the *genesis event* (the first event in the chain), not by the totality of events—analogous to a poem, which depends on some physical copy for its existence but not on any specific one. In DOLCE, the closest category is *non-physical object* Gangemi et al. [2002]; in UFO, concepts are *conceptual entities* Guizzardi [2005].

3.2.3 EventChain as Process

An EventChain is a *process* in BFO: a complex occurrent with temporal parts (the individual events) that unfolds over time Arp et al. [2015]. It is not a continuant because it does not exist in full at any moment. It is individuated by the ordered sequence of events and their cryptographic linkage (see Section 3.2.6 for the full two-level account, which distinguishes the process from its record).

3.2.4 Actor as Agent

ADL Lite's `Actor` is mapped to BFO `agent role` and UFO `Agent`; the DOLCE `physical agent` mapping is approximate because DOLCE lacks a non-physical agent category (see Table 3 footnote for full rationale). Future work (FW1) may define a DOLCE extension for software agents.

3.2.5 Relation as Relational Quality / Relator

ADL Lite's L3 relation blocks assert typed edges between concepts. In BFO, these are *relational qualities*: qualities that inhere in one entity and bear a relation to another Arp et al. [2015]. ADL Lite's L3 relation block is inscribed in the source concept's Markdown file (the bearer), while the relation mediates between two concepts. This reading is consistent with BFO relational qualities, which need not be symmetric. In UFO, the same relations are *relators*: entities that mediate between relata and are brought into existence and destroyed by events Guizzardi [2005]. The dual reading—BFO relational quality for the record, UFO relator for the mediation—reflects the two-level account of Section 3.2.6.

Creation. A `RELATE` event e establishes a relator r at time $t = e.\text{timestamp}$:

$$\text{HoldsAt}(\text{Relation}(c_1, c_2, p), t) \iff \exists e \in C.\ e.\text{type} = \text{RELATE} \wedge e.\text{timestamp} \leq t \wedge \neg\text{Revoked}(c_1, c_2, p, t) \quad (1)$$

where $\text{Revoked}(c_1, c_2, p, t)$ holds if and only if there exists a revocation event e' with $e'.\text{timestamp} \leq t$.

Revocation. A relator is terminated by an explicit `REVOKE` event e_{rev}:

$$\text{Revoked}(c_1, c_2, p, t) \iff \exists e_{\text{rev}} \in C.\ e_{\text{rev}}.\text{type} = \text{REVOKE} \wedge e_{\text{rev}}.\text{timestamp} \leq t \wedge e_{\text{rev}}.\text{payload.target\_concept\_id} = c_2 \wedge e_{\text{rev}}.\text{payload.predicate} = p \quad (2)$$

Note on revocation semantics. ADL Lite supports two semantics for relation termination. *Cessation semantics* (Equation 2): a dedicated `REVOKE` event explicitly terminates the relator, whose record (L3 block + `REVOKE` event) remains as an auditable ICE. *Epistemic weakening* (alternative): a `RELATE` event with `confidence`=0 suspends the relation's mediation without formally terminating it, allowing graded weakening. Importantly, *HoldsAt does not depend on confidence*: it checks only

whether a `REVOKE` event exists (Equation 2). Confidence is epistemic strength, not an existence condition—a relation with confidence 0.3 still holds; a relation with a `REVOKE` event does not, regardless of the original `RELATE`'s confidence. We recommend *cessation semantics* as the default for audit and compliance queries because it yields a binary, unambiguous existence condition; epistemic weakening is retained for consumer-side belief filtering (e.g., ranking by a confidence threshold θ) and for systems preferring a graded thresholded variant HoldsAt_θ. The `REVOKE` event is a communication event (not in Σ_{life}), so it does not affect $\delta(C)$; Theorems 1–6 are unchanged.

Role constraints and temporal scoping. A relation holds only while both relata exist and are neither deprecated nor archived (Equation 3, where $\text{Deprecated}(c, t)$ and $\text{Archived}(c, t)$ are derived from c's chain):

$$\begin{aligned}\text{HoldsAt}(\text{Relation}(c_1, c_2, p), t) \rightarrow \text{Exists}(c_1, t) \wedge \text{Exists}(c_2, t) \wedge \neg\text{Deprecated}(c_1, t) \wedge \\ \neg\text{Deprecated}(c_2, t) \wedge \neg\text{Archived}(c_1, t) \wedge \neg\text{Archived}(c_2, t) \quad (3)\end{aligned}$$

Every relator has a temporal extent $[t_{\text{create}}, t_{\text{revoke}})$: it is a continuant that exists only during this interval; what endures beyond it is the record (the L3 block), an ICE documenting that the relation held. This exemplifies UFO's relator theory Guizzardi [2005]: existential dependence on both relata, event-based creation, event-based destruction, and mediation between entities rather than a property of either relatum alone; DOLCE's *quality* category includes both intrinsic and relational qualities Gangemi et al. [2002], and ADL Lite relations are relational qualities in this sense.

3.2.6 Two-Level Ontological Account: Occurrents versus Records

The term "EventChain" is ambiguous between a process (events unfolding in time) and an ICE (the serialized record). These are distinct entities with distinct identity conditions.

Level 1: Occurrents (What Happens). Event (e): an individual occurrent/perdurant with a determinate timestamp. EventChain-process (P): the complex occurrent comprising the ordered event sequence. Action (a): an intentional occurrent. Level 1 entities do not endure; at time t, only events up to t have occurred.

Level 2: Information Content Entities (What Is Recorded). EventChain-record (R): the serialized, hash-linked JSON/YAML representation—an ICE, a continuant existing in full when stored. Concept (c): the GDC depending on the record; it interprets the event sequence to project status, confidence, and validators. Relation (r): the record of a relational quality in L3 blocks, an ICE documenting a relationship that held during a specific interval.

Explicit identity axioms (necessary and sufficient conditions). Following Lowe's account of identity conditions as both necessary and sufficient criteria Lowe [1998], we state each axiom as a biconditional.

Axiom I1: **Event identity.** The identity of an event is *necessary and sufficient* given by its UUID:

$$e = e' \iff e.\text{event\_id} = e'.\text{event\_id}. \quad (4)$$

The UUID is a rigid designator: if $e \neq e'$ then their event IDs differ, and identical event IDs entail the same event (no two distinct events may share a UUID).

Axiom I2: **Concept identity.** The identity of a concept is *necessary and sufficient* given by its genesis hash:

$$c = c' \iff c.\text{genesis\_hash} = c'.\text{genesis\_hash}. \tag{5}$$

The `concept_id` is a human-readable alias; two concepts with the same `concept_id` but different genesis hashes are distinct registry items (see Section 3.4 for registry-item versus domain-concept identity).

Axiom I3: **EventChain-record identity.** Two records are identical *iff* they have the same concept and the same event sequence:

$$R = R' \iff R.\text{concept\_id} = R'.\text{concept\_id} \wedge \text{events}(R) = \text{events}(R'). \tag{6}$$

Event-sequence equality is extensional: every event in R appears in R' with the same hash, and vice versa. The ordering is implied because the cryptographic hash chain enforces a unique total order.

Axiom I4: **EventChain-process identity.** Two processes are identical *iff* they have the same events in the same order with the same timestamps:

$$P = P' \iff \text{events}(P) = \text{events}(P') \wedge \text{order}(P) = \text{order}(P') \wedge \text{timestamps}(P) = \text{timestamps}(P'). \tag{7}$$

The timestamp condition is necessary because two processes with the same events in the same order but at different times are distinct occurrents (they occupy different temporal regions).

FORK identity preservation. A FORK event creates a child concept c' from a parent c. The child is a *new* entity with a *new* genesis hash:

Axiom I5: **FORK identity.** Let c' be the concept created by $\text{FORK}(c)$. Then:

$$c' \neq c \ \wedge \ \text{genesis\_hash}(c') \neq \text{genesis\_hash}(c). \tag{8}$$

The child's genesis hash is a function of the parent's genesis hash and the FORK event:

$$\text{genesis\_hash}(c') = \text{SHA-256}\big(\text{FORK\_EVENT} \,\|\, \text{genesis\_hash}(c)\big). \tag{9}$$

Even if the FORK event carries identical payload to the parent's genesis event, the child remains distinct because the FORK event has a unique `event_id` and timestamp. This is *provenance identity* (identity fixed by causal history) rather than content-addressed identity Kuhn and Dumontier [2015].

CRDT merge identity. When two branches b_1, b_2 of the same concept are merged, the result is a new record R whose event set is the union of the two branch event sets:

Axiom I6: **CRDT merge identity.** Let $R = \text{merge}(b_1, b_2)$. Then:

$$\text{events}(R) = \text{sorted\_unique}\big(\text{events}(b_1) \cup \text{events}(b_2)\big), \tag{10}$$

where sorting is deterministic by `event_id` (UUID). The merge preserves the `concept_id` of both parents:

$$R.\text{concept\_id} = b_1.\text{concept\_id} = b_2.\text{concept\_id}. \tag{11}$$

Merge never produces a new `concept_id`; it produces a new *record* of the same concept. The identity of the concept c is unchanged because the genesis hash (which fixes c's identity, Axiom I2) is preserved in both branches.

L3 relation identity. An L3 relation is a relator that exists only during the interval $[t_{\text{create}}, t_{\text{revoke}})$. Its identity is fixed by the RELATE event that created it:

Axiom I7: **L3 relation identity.** Two relators are identical *iff* they were created by the same RELATE event:

$$r = r' \iff \textsc{Relate\_event}(r).\text{event\_id} = \textsc{Relate\_event}(r').\text{event\_id}. \tag{12}$$

A relator is terminated by a REVOKE event (Equation 2), after which it ceases to exist; the record of the relator (the L3 block) persists as an ICE, but the mediating entity no longer holds.

Dependence axioms (necessary and sufficient conditions).

Axiom D1: **Existential dependence on the record (D1).** A concept c ontologically depends on the EventChain-record R: c would not exist if R had not been created. Formally, c depends on R iff R is an EventChain-record about c:

$$\text{Dep}(c, R) \iff \text{EventChainRecord}(R) \wedge \text{isAbout}(R, c). \tag{13}$$

This is a generic dependence on an ICE bearer (D2), not a dependence on the EventChain-process.[2]

Axiom D2: **Generic dependence (D2).** c generically depends on the EventChain-record: it depends on an ICE bearer, but not on any specific physical copy. This is necessary but not sufficient for D1: generic dependence is the *kind* of dependence, while D1 specifies the *relatum*.

Axiom D3: **Concretization (D3).** The EventChain-record is concretized by a material bearer (file, database row, agent cache). This is specific dependence: the record depends on *some* material bearer, and if that bearer is destroyed, the record ceases to exist unless another copy exists.

Axiom D4: **Process-to-record (D4).** The EventChain-process causally produces the EventChain-record. This is a causal, not ontological, dependence: the process is the *origin* of the record, but the record's continued existence does not depend on the process continuing (the record may outlast the process).

Axiom D5: **No cross-level identity (D5).** No occurrent is identical to any ICE (a BFO constraint). Consequently, no GDC can ontologically depend on an occurrent; a GDC may only depend on continuants (including ICEs). Formally:

$$\forall x \forall y \, (\text{GDC}(x) \wedge \text{Occurrent}(y) \rightarrow \neg \text{Dep}(x, y)). \tag{14}$$

Axiom D6: **Generic dependence of concept on record (D6).** Every concept generically depends on some EventChain-record that is about it:

$$\forall c \, (\text{Concept}(c) \rightarrow \exists R \, (\text{EventChainRecord}(R) \wedge \text{isAbout}(R, c) \wedge \text{Dep}(c, R))). \tag{15}$$

D6 is the universal closure of the left-to-right direction of D1: it states that *every* concept must have *some* record bearer. D6 is necessary for the existence of concepts; without a record, a concept cannot exist. D6 together with D2 (generic dependence) entails that a concept can survive the destruction of any particular record copy, provided at least one copy remains.

[2] We distinguish causal origin from ontological dependence. The `REGISTER` event is the *causal origin* of c—it brought c into existence—but c's continued existence depends on the ICE record (D2), not on the occurrent process. This avoids the BFO constraint that GDCs cannot depend on occurrents (D5).

First-order logic encoding. To make the dependence structure amenable to formal analysis, we translate the axioms into first-order logic with explicit world parameters (following Fine's possible-worlds framework Fine [1995]), letting $E(x, w)$ mean "x exists in world w" and $\text{Dep}(x, y, w)$ mean "x ontologically depends on y in world w". D2 (generic dependence) states that in every world where c exists some record bears it, yet no *particular* record is indispensable; D5 (no cross-level identity) states $\forall x \forall y\,(\text{Occurrent}(x) \wedge \text{ICE}(y) \rightarrow x \neq y)$, from which $\text{Concept}(c) \wedge \text{EventChainProcess}(P) \rightarrow \neg\text{Dep}(c, P, w)$ follows; D6 (universal record dependence) states that every concept depends on *some record about it*; and I3–I4, I5, and I7 render record/process identity, FORK identity, and relator identity as extensional sorted-signature criteria. The complete FOL encodings (Equations D2/D5/D6/I3–I5/I7) are given in Appendix A.5. The OWL 2 DL fragment (Appendix A.1) provides a decidable approximation of the class-level structure, while these FOL formulas capture the dependence and identity relations that exceed OWL expressivity.

Resolving the apparent ambiguity. EventChain-process and EventChain-record are distinct (Axioms I3–I4); the process causally produces the record (D4), which bears the concept (D2). A GDC cannot ontologically depend on an occurrent (D5), but can depend on the ICE that documents it (D2–D3). The concept's causal origin is traceable to the genesis event (the `REGISTER` occurrence), but its ontological bearer is the record. This resolves the category mistake.

3.2.7 Action as Planned Process / Intentional Event

ADL Lite's L4 action types are *planned processes* in BFO (processes realizing a plan) Arp et al. [2015] and *actions* in UFO (intentional events performed by agents to achieve goals) Guizzardi [2005]. The precondition system captures the plan aspect: an action can only be performed if certain conditions hold (e.g., `VALIDATE` requires status `provisional`), reflecting ontological norms governing which events are permissible in which states.

3.3 OntoClean Evaluation

We evaluate ADL Lite's core classes through the OntoClean meta-property lens Guarino and Welty [2009, 2002, 2000]. OntoClean provides four meta-properties for ontological analysis: *rigidity* (R), *identity* (I), *unity* (U), and *dependence* (D) Guarino and Welty [2009]. A class is *rigid* ($+R$) if its instances cannot cease to be members without ceasing to exist; *carries identity* ($+I$) if its instances can be re-identified across worlds; *carries unity* ($+U$) if its instances are unified wholes (not mereological sums); and *existentially dependent* ($+D$) if its instances cannot exist without some other entity Guarino and Welty [2009], Fine [1995]. Supplementary Table S24 summarises the assignments for ADL Lite's ten core classes; the rigorous justification follows.

3.3.1 Rigidity Analysis

A class is rigid ($+R$) iff every instance is necessarily an instance in every possible world in which it exists Guarino and Welty [2009]. `Concept` is $+R$ because the genesis hash is an essential property (Axiom I2): if a concept's genesis hash changed, it would be a different entity. `Hash` is $+R$ because a SHA-256 value is a fixed mathematical object. All other core classes are $-R$: an event may be re-interpreted without changing its UUID; the EventChain-process changes as events are appended; the record is replaced by a new snapshot with each append; actions are temporal slices that conclude; relations are created and terminated by `RELATE`/`REVOKE` events; payloads are transient; status values change as the chain evolves; and an actor is an agent role, which is anti-rigid by definition Guarino and Welty [2009].

3.3.2 Identity Analysis

A class carries identity ($+I$) if instances can be re-identified across worlds Guarino and Welty [2009, 2000]. All ten classes carry identity ($+I$), because the event-first design requires every entity to be traceable: events by UUID (Axiom I1); EventChain-processes by ordered events and timestamps (Axiom I4); records by concept plus event set (Axiom I3); concepts by genesis hash (Axiom I2); relations by their creating `RELATE` event (Axiom I7); actions by event UUID; actors by actor identifier; payloads by content hash; hashes by string equality; and status by concept-plus-time.

3.3.3 Unity Analysis

A class carries unity ($+U$) if its instances are unified wholes rather than mereological sums Guarino and Welty [2009]. `Concept`, `Relation`, `Actor`, and `EventChain`-record are $+U$: each is a coherent entity with a definite identity criterion (genesis hash, relator mediation, agenthood, structured schema). `Event`, `EventChain`-process, `Action`, `Payload`, `Status`, and `Hash` are $-U$: events are temporal slices, processes are mereological sums of events, and the rest are data structures or derived values.

3.3.4 Dependence Analysis

A class is existentially dependent ($+D$) if its instances cannot exist without some other entity Fine [1995], Guarino and Welty [2009]. `Concept` is $+D$ (generic): as a GDC it depends on *some* ICE bearer (BFO/IAO: GDC $\rightarrow$ ICE $\rightarrow$ IBE Smith et al. [2012], Limbaugh et al. [2024]), not on any specific copy. `EventChain`-record is $+D$ (generic, on a material bearer). `Relation` (L3) is $+D$ (specific, mutual rigid dependence on both relata). `Action`, `Payload`, and `Status` are $+D$ (specific: on actor, carrying event, and concept-plus-events respectively). `Event`, `EventChain`-process, `Actor`, and `Hash` are $-D$.

3.3.5 Constraints and Consequences

The assignments satisfy OntoClean's logical constraints Guarino and Welty [2009, 2002]: $+R \Rightarrow +I$ (`Concept`, `Hash` both carry identity); $+U \Rightarrow +I$ (all $+U$ classes carry identity); and $+D \Rightarrow +I$ (all $+D$ classes carry identity). Subclass constraints also hold: `Concept` $\sqsubseteq$ `GDC` with both $+R/+U/+D$; `EventChain`-record $\sqsubseteq$ `ICE` with both $+U/+D$; `Relation` $\sqsubseteq$ `Relator` (UFO) with both $+U/+D$.

The assignments directly justify the design: (i) the $+R$ assignment for `Concept` justifies genesis-hash identity; (ii) the $+D$ assignments justify the BFO/IAO dependence chain; (iii) forking creates a child with a new genesis hash, which is a new *instance*, not a subclass, so rigidity is preserved; (iv) the $-R$ assignment for `Event` justifies event immutability; and (v) the $-R$ assignment for `Status` justifies status as a derived, non-essential property.

3.3.6 Comparison with Alternative Readings

Three alternative readings are inferior to the chosen assignments. *Alternative 1 (Concept as independent continuant):* would make `Concept` $-D$, contradicting the BFO/IAO information-artifact analysis Smith et al. [2012] and importing spatiotemporal identity criteria inappropriate for abstract concepts. *Alternative 2 (Event as continuant):* would let events endure and change, violating the event-first principle that all derived properties are historical (§3.1) and undermining the append-only design. *Alternative 3 (Status as rigid):* would make a `validated` and a `deprecated` concept different entities, contradicting explicit lifecycle transitions. These assignments reflect author judgment and would benefit from independent ontologist assessment (FW9).

3.4 Identity Conditions

We formalize identity conditions in Section 3.2.6 as *necessary and sufficient* biconditionals: events by UUID (Axiom I1, Equation 4), concepts by genesis hash (Axiom I2, Equation 5), and chains by concept plus ordered events (Axioms I3–I4, Equations 6–7). Here we examine the consequences for FORK, CRDT merge, and L3 relations. Deprecation and re-activation are status changes, not identity changes (the genesis hash is invariant). We evaluate the concept's identity through the OntoClean lens Guarino and Welty [2009].

Registry-item identity versus domain-concept identity. We distinguish two senses of "concept" in ADL Lite. *Registry-item identity* (operational) is fixed by the genesis hash; it is the identity criterion used by the EventChain data structure. Two registry entries with the same genesis hash are the same item, regardless of their content. *Domain-concept identity* (intensional) is determined by the meaning, definition, and extensional content of the capability. Two registry items (different genesis hashes) may refer to the same domain concept (e.g., "gradient-explosion" and "exploding-gradients" in Section 5.5). The genesis-hash criterion provides *registry-item* identity, not *domain-concept* identity. It is intentionally operational: the registry must distinguish items even when their content is identical (to handle independent discovery by different agents). Domain-level identity is mediated by L3 relations (e.g., `isomorphic-to`, `specialisation-of`). This is analogous to a library catalog, in which each entry has a unique ISBN (registry identity) but multiple entries may refer to the same intellectual work (domain identity).

Rigidity and sufficiency. A concept's genesis hash is *essential*—it is both necessary and sufficient for identity (Axiom I2). It cannot change without the entity ceasing to exist, while its status is non-rigid. This aligns with OntoClean's recommendation to use rigid properties as identity criteria Guarino and Welty [2009]. The sufficiency direction ($c = c' \rightarrow \text{genesis\_hash}(c) = \text{genesis\_hash}(c')$) guarantees that distinct genesis hashes entail distinct concepts; the necessity direction guarantees that identical genesis hashes entail the same concept. This two-way criterion is stronger than a mere "individuation" principle; it is a full identity condition in Lowe's sense Lowe [1998].

3.4.1 FORK Identity Preservation

A FORK event creates a child concept c' from a parent c; the child is a *new* entity with a *new* genesis hash (Axiom I5, Equation 8), computed as $\text{genesis\_hash}(c') = \text{SHA-256}(\text{FORK\_event} \,\|\, \text{genesis\_hash}(c))$, where FORK_event includes a unique `event_id`, timestamp, and `canon_version`. Parent and child are *never* identical, even for byte-identical payloads, because the FORK events have different UUIDs and timestamps. This embodies *provenance identity* rather than content-addressed identity Kuhn and Dumontier [2015]: the FORK operation is a *constructor* of new identity, and the `fork-of` L3 predicate records the lineage. If an agent modifies the payload during a fork, the child is still a new entity, with the modification recorded in the FORK payload; the parent's identity is unchanged. This satisfies the OntoClean constraint that rigid classes cannot be split into subclasses with different identity criteria.

3.4.2 CRDT Merge Identity

When two branches b_1, b_2 of the same concept are merged, the result is a new record R whose event set is the union of the two branch event sets, sorted deterministically by `event_id` (Axiom I6, Equation 10); the merge preserves the `concept_id` of both parents (Equation 11). Merge never produces a new `concept_id` or genesis hash: it produces a new *record* of the same concept (the identity of c is unchanged because the genesis hash is preserved). Deterministic ordering by UUID (independent of any physical clock) is required for record identity: sorting by timestamp could

produce different orderings under clock skew, violating $R = R' \iff \text{events}(R) = \text{events}(R')$ (Axiom I3). Merge is defined only for branches of the same concept: attempts to merge branches with different `concept_id`s are rejected by verify_integrity (Theorem T1).

3.4.3 L3 Relation Identity and Termination

An L3 relation is a relator that exists only during $[t_{\text{create}}, t_{\text{revoke}})$; its identity is fixed by the RELATE event that created it (Axiom I7, Equation 12). A relator r between c_1 and c_2 *mutually* and *rigidly* depends on both concepts: if c_1 is archived, r ceases to hold even if c_2 still exists. Formally, $\text{HoldsAt}(r,t) \rightarrow \text{Exists}(c_1,t) \wedge \text{Exists}(c_2,t) \wedge \neg\text{Archived}(c_1,t) \wedge \neg\text{Archived}(c_2,t)$ (a stronger condition than Equation 3). A relator is terminated by a REVOKE event (Equation 2): after revocation it ceases to exist as a mediator, while the L3 block and the REVOKE event persist as ICEs—an event-based destruction pattern consistent with UFO relator theory Guizzardi [2005].

3.4.4 Domain-Level Identity Alignment and Merge Conditions

While the genesis hash provides operational identity, domain-level identity (whether two chains denote the same concept) is determined by human curators, not the registry: ADL Lite does not support automatic chain merging (two chains with different genesis hashes are always distinct registry items), and domain-level consolidation is a governance decision recorded via L3 relations and DEPRECATE events. We recommend six normative L3 predicates, ordered from strongest to weakest claim: (i) `isomorphic-to` (identical structure/content, different genesis hashes; reflexive, symmetric, transitive—the strongest claim short of genesis equality); (ii) `specialisation-of` (a refinement or instance; irreflexive, antisymmetric, transitive); (iii) `fork-of` (explicit lineage via a fork operation; irreflexive, antisymmetric, transitive by ancestry, stronger than `isomorphic-to` because it records provenance); (iv) `related-to` (general association; symmetric, non-transitive); (v) `analogical-to` (similar by analogy; symmetric, non-transitive); (vi) `co-occurs-with` (frequent co-occurrence; symmetric, non-transitive).

Relation semantics and merge conditions. These properties are declared in `adl_core_ontology.yaml` but not enforced by the ActionExecutor at append time—enforcing transitivity would require a global graph search ($O(|V| + |E|)$ per RELATE event), violating the $O(1)$ append complexity. Instead the query engine performs *inference expansion* at read time using the transitive closure of `isomorphic-to` and `specialisation-of` (not `related-to` or `analogical-to`), separating *storage semantics* (append-only, no global consistency check) from *query semantics* (inference at read time). A domain expert may consolidate two chains by deprecating one in favour of the other via `isomorphic-to` or `specialisation-of`, recorded as a DEPRECATE event with reasoning—not an automatic merge—so the deprecated chain remains auditable and the full provenance of the consolidation is recoverable.

Provenance-addressing vs. content-addressing. The genesis hash is an *operational (provenance) identifier*, not a content-addressed hash (Merkle DAG/IPFS CID): it includes the `event_id` (UUID), `timestamp`, and `canon_version` in addition to the payload, so two byte-identical REGISTER events have different genesis hashes. This deliberately guarantees *provenance uniqueness* (each registration is a distinct event with its own actor, timestamp, and UUID) rather than *semantic equivalence*; content-addressing would enable automatic deduplication but lose the provenance needed to distinguish independent registrations. Domain-level consolidation of identical concepts is the responsibility of human curators via L3 predicates. Concept *unity* is individuated by the genesis hash (the `EventChain` is a mereological sum of events with no unified whole beyond sequential aggregation Guarino and Welty [2009]), and the concept is *generically dependent* on the EventChain

record (an ICE bearer, not any specific physical copy Arp et al. [2015], Fine [1995]).

3.5 Ontological Dependence

ADL Lite exhibits three forms of dependence. (1) *Identity and dependence conditions.* The concept is a GDC depending on the EventChain record: Concept $\xrightarrow{\text{GDC}}$ ICE(EventChain-record) $\xrightarrow{\text{concretized\_by}}$ Markdown_file. A concept depends on the information artifact (the serialized record) rather than on the occurrent process of events, consistent with BFO's constraint that GDCs cannot depend on occurrents; its identity is fixed by the genesis event. As a GDC it can be concretized in multiple copies (Git repositories, local files, agent caches) without loss of identity, and it ceases to exist only when all concretizations are irreversibly destroyed *and* no agent retains information to reconstruct the chain. (2) *Causal origin vs. ontological dependence.* A concept c *causally originates from* the EventChain-process (the genesis REGISTER event brought it into existence) but *ontologically depends* only on the EventChain-record R (an ICE): the causal origin is an unrepeatable historical fact, whereas the ontological dependence is a repeatable bearer relationship (Axiom D5 prohibits existential dependence on an occurrent). (3) *D6 (universal record dependence).* Every concept generically depends on some EventChain-record that is about it (Equation 15); D6 is necessary for concept existence (it rules out "disembodied" concepts) and, together with D2, entails that a concept survives the destruction of any finite set of record copies provided at least one remains Smith et al. [2012].

Generic dependence of status on events. A concept's status s (e.g., `validated`) *generically depends* on the events in its chain: s depends on the existence of at least one `VALIDATE` event, but not on any *specific* one (a different agent validating at a different time would still yield `validated`).

Mutual dependence of relations on concepts. A relation r between concepts c_1 and c_2 *mutually* (and rigidly) depends on both: r cannot exist if either relatum ceases to exist. Formally, $\text{Exists}(r,t) \leftrightarrow \text{Exists}(c_1,t) \wedge \text{Exists}(c_2,t) \wedge \text{HoldsAt}(r,t)$, where $\text{Exists}(c,t)$ means c has not been archived at t. If `concept-A` is deprecated, the relation "`concept-A` isomorphic-to `concept-B`" ceases to hold even if `concept-B` still exists—an ontological fact, not merely a query-side filter. The dependence is rigid because the relator cannot change its relata without becoming a different relator (Axiom I7), consistent with UFO's relator theory Guizzardi [2005].

We formalize these dependence patterns as an OWL 2 DL alignment fragment in the next subsection.

3.6 Formal Alignment Fragment: OWL 2 DL Axiomatization

We present the OWL 2 DL alignment fragment that formalizes the dependence patterns above in Appendix A.1. The revised fragment (v0.2) declares: (i) core categories (`adl:Event` $\sqsubseteq$ `bfo:occurrent`, `adl:EventChain` $\sqsubseteq$ `bfo:process`, `adl:Concept` $\sqsubseteq$ `bfo:generically_dependent_continuant`); (ii) L3 relation predicates as OWL object properties with symmetry/transitivity constraints (`isomorphic-to`, `specialisation-of`, `fork-of`, `related-to`, `analogical-to`, `co-occurs-with`, `mitigated-by`); (iii) ontological dependence axioms (D2–D5) as object properties and disjointness constraints; and (iv) illustrative SWRL rules for status-transition constraints. The fragment has been validated with ROBOT: OWL 2 DL profile conformance confirmed, structural consistency reported, and three SPARQL constraints verified on example documents (zero violations on valid data, correct detection on deliberately corrupted data). Full operational encoding of δ and γ exceeds OWL 2 DL expressivity and is deferred to future work (FW1).

SHACL/OWL expressivity boundaries. The SHACL shape graph (Appendix A.2) validates L3 relation blocks against structural constraints: identifier format (`sh:pattern`), predicate

membership (`sh:in`), confidence range (`sh:minInclusive`/`sh:maxInclusive`), and directionality (`sh:sparql`). Five of eight constraints use SHACL Core; three require SHACL-SPARQL. However, the status derivation function δ and the precondition checks are *not expressible in SHACL or OWL 2 DL* for fundamental reasons: (i) δ aggregates over event sequences (taking the maximum over lifecycle events), which requires recursion or fixed-point computation exceeding SHACL's declarative expressivity; (ii) preconditions involve comparator dispatch over derived snapshots, which is procedural rather than declarative; (iii) temporal reasoning (`HoldsAt`, `Revoked` conditions) requires time-indexed assertions over event sequences, which exceeds OWL 2 DL's snapshot-based semantics; (iv) cryptographic integrity (SHA-256 correctness) is a computational property that cannot be expressed in description logic; (v) *cross-chain references* require accessing events outside the current chain, which violates the local-scope restriction of the precondition language (§4.4.1). Encoding cross-chain references in OWL 2 DL would require non-local scope during precondition evaluation, violating the referential transparency and deterministic evaluation guarantees of Theorem 8.

Interoperability mapping table. Supplementary Table S25 provides a normative mapping from ADL Lite core constructs to OWL 2 DL classes, SHACL shapes, and PROV-O/IAO properties. This mapping enables the applied ontology community to adopt ADL Lite's lifecycle semantics within existing RDF tooling without abandoning the runtime's derivation functions.

Adoption path. The bidirectional export/import (Turtle/RDF/XML via `owl_export.py` / `jsonld_export.py`) allows ADL Lite concepts to be consumed by RDF tooling without losing the EventChain structure. The exported RDF represents each event as a PROV-O activity with cryptographic linkage, and the concept as a PROV-O entity with derived status. This is a "lossy but useful" export: the lifecycle semantics (δ, γ, preconditions) are not encoded in the RDF (they exceed OWL/SHACL expressivity), but the structural and provenance information is preserved. The SHACL shape graph (Appendix A.2) validates five of eight L3 structural constraints; the remaining three (directionality, cross-document existence, conditional symmetry) require SHACL-SPARQL. Full operational encoding of δ and preconditions in SHACL/OWL remains future work (FW6b).

3.7 Comparison with Upper Ontologies: Alignment Choices, Deviations, and Costs

Table 3 identifies four intentional deviations from BFO/DOLCE/UFO, each with an interoperability cost. **Deviation 1 (no continuant–occurrent distinction at the language level):** BFO and DOLCE encode the distinction in the ontology language Arp et al. [2015], Gangemi et al. [2002]; ADL Lite enforces it conceptually through the event model, so its alignment is *conceptual guidance* rather than strict commitment (no direct BFO module import or OWL-reasoner enforcement); OWL export therefore requires a translation layer. **Deviation 2 (no explicit quality hierarchy):** ADL Lite encodes qualities as payload fields (e.g., "confidence" carried by `VALIDATE` events), so DOLCE's quality taxonomy is not directly reusable without explicit mapping. **Deviation 3 (actions as first-class citizens):** actions are co-equal with objects at the language level, reflecting the event-first stance; direct reuse of UFO-B's process hierarchy is prevented, but UFO-B's "Institutional Event" category Guizzardi et al. [2024] remains compatible, and ADL Lite's precondition language is a *variable-free ground fragment* (closed comparator enumeration, no quantification) that is a *tractable specialisation* of UFO-B's FOL framework with $O(1)$ per-rule evaluation. **Deviation 4 (historical vs. ontological dependence):** the causal origin of a concept (traceable to the genesis event, an occurrent) is distinguished from its ontological bearer (the EventChain-record, an ICE): the concept is a GDC that depends on the record (D2), not on the process (D5), avoiding the category mistake of making a GDC existentially depend on an occurrent.

Supplementary Table S26 contrasts the operational mechanisms: ADL Lite embeds execution

semantics *inside* the data structure (the EventChain), whereas UFO-B delegates execution to an external workflow engine that interprets the ontology.

Deviation 4: Historical vs. ontological dependence. We distinguish the causal origin of a concept (traceable to the genesis event, an occurrent) from its ontological bearer (the EventChain-record, an ICE). The concept is a GDC that depends on the record (D2), not on the process (D5). This avoids the category mistake of making a GDC existentially depend on an occurrent. The "historical" or "genetic" relationship to the process is a causal narrative, not an ontological dependence in the Fine/BFO sense.

4 Architecture and Formal Semantics

4.1 Document Model: L1–L4 as Conceptual Modelling Layers

ADL Lite organises capability documents into four semantic layers, each with distinct syntax, role, and corresponding event types. The layering separates identity metadata (L1), narrative content (L2), typed assertions (L3), and executable actions (L4), enabling both human readability and machine processing within a single Markdown file. Table 4 summarises the document model.

Table 4: The L1–L4 Document Model. Each layer contributes distinct semantics to the capability document.

Layer	Syntax	Role	Event Type
L1	YAML front matter	Identity metadata (derived snapshot)	SNAPSHOT
L2	Markdown body	Human/LLM narrative prose	—
L3	`adl:relation`, `adl:evidence`, `adl:seal`	Typed semantic assertions	RELATE, EVIDENCE, SEAL
L4	`adl:action`	Typed actions with preconditions	REGISTER, VALIDATE, DEPRECATE, ...

The L1 layer comprises YAML front matter containing identity metadata; critically, these fields are *derived snapshots* recomputed from the EventChain-record (the serialized information content entity, see §3.2.6). The L2 layer consists of Markdown body prose subject to the Singular Subjectivity Assertion (SSA).

Definition (SSA). The Singular Subjectivity Assertion is a document-level constraint stating that every L2 Markdown body is authored by a single actor at a single time. It is enforced by the parser: any L2 section containing multiple contradictory subjective claims without explicit attribution is flagged as an SSA violation. The SSA ensures that the L2 narrative layer remains traceable to a single source of subjectivity, preventing "ghost-authored" reasoning. Contradictory subjective assertions must be recorded as separate L4 events (e.g., EVIDENCE or RELATE) by different actors. The SSA interacts with lifecycle audit as follows: (i) a concept in `provisional` status may contain SSA-violating prose (it is unvalidated); (ii) a VALIDATE action's precondition requires that the L2 body pass SSA checking; (iii) FORK events inherit the parent's L2 body but must satisfy SSA for the new actor's rationale. Basic SHACL shapes are implemented (Appendix A.6); full SSA-specific SHACL enforcement and epistemic context learning remain future work (FW6).

The L3 layer introduces typed semantic assertions through fenced blocks supporting predicates such as `isomorphic-to`, `specialisation-of`, and `related-to`. If two agents independently register

"gradient-explosion" and "exploding-gradients", the recommended workflow detects the near-duplicate through string similarity and links them with the strongest applicable predicate (`isomorphic-to` for structural equivalence). Automated canonicalisation via embedding clustering is deferred to future work (FW7).

Invariant 2 (L3 Relation Reconciliation). When either the source or target concept of a `RELATE` event undergoes a state transition to `deprecated` or `archived`, the relation's validity is determined by the status of both endpoint concepts. A relation is valid only if at least one endpoint is in `validated` status and neither endpoint is in `archived` or `deprecated` status. Let r be a relation with predicate p from concept C_1 to concept C_2, and let $S(C)$ denote the current lifecycle status of concept C. Then:

$$\begin{aligned}\text{valid}(r) \iff\ & S(C_1) \notin \{\text{archived}, \text{deprecated}\} \wedge S(C_2) \notin \{\text{archived}, \text{deprecated}\} \\ & \wedge \left(S(C_1) = \text{validated} \vee S(C_2) = \text{validated}\right).\end{aligned} \tag{16}$$

Relations with at least one endpoint in `validated` status and no endpoint in `archived` or `deprecated` status remain valid for query and traversal. Conversely, relations between two `provisional` concepts, or where either endpoint is `deprecated` or `archived`, are excluded from the warm index. If a forked concept C' is created via a `FORK` event from C, the parent concept retains all existing relations, whereas C' inherits `isomorphic-to` and `specialisation-of` links from C. It does not inherit `analogical-to` links, which are re-evaluated during the forking agent's validation workflow. This invariant ensures that the relation graph remains navigable during multi-agent reorganisation and that stale or superseded edges do not pollute semantic queries.

4.2 EventChain: Core Data Structure

The EventChain-record (the serialized information content entity, see §3.2.6) is the fundamental data structure of ADL Lite: an append-only sequence of hash-linked events. An event comprises ten fields: `event_id`, `concept_id`, `event_type`, `actor`, `reasoning`, `timestamp`, `payload`, `previous_event_id`, `hash`, and an optional `signature` (for Ed25519 signature verification, §4.9). The event type system is partitioned into lifecycle events Σ_{life} and communication events Σ_{comm}:

$$\Sigma_{\text{life}} = \{\text{REGISTER, VALIDATE, DEPRECATE, FORK, ARCHIVE}\} \tag{17}$$

$$\begin{aligned}\Sigma_{\text{comm}} = \{&\text{RELATE, EVIDENCE, SEAL, ANNOUNCE, PUBLISH, SNAPSHOT,} \\ &\text{CALIBRATE, SYNC\_DASHBOARD, LISTEN, REVOKE}\}\end{aligned} \tag{18}$$

4.3 Canonical Serialization and Cryptographic Integrity

Each event's hash is computed from canonical JSON serialization with lexicographically sorted keys, six-decimal rounding, and the timestamp. The canonicalization policy is version-locked: a schema version identifier (`canon-v1`) is included in the hash input, ensuring that future changes to rounding precision or key ordering do not retroactively invalidate existing hashes. Integrity verification VerifyIntegrity(C) checks four conditions for a chain $C = [e_1, \ldots, e_n]$: (1) genesis anchoring (e_1.previous_event_id = null), (2) cryptographic linkage (e_i.prev_hash = e_{i-1}.hash), (3) hash correctness (e_i.hash = SHA-256(Canon(e_i))), and (4) no dangling `previous_event_id` on the genesis event. When full = true, it additionally verifies archive file hashes against stored `archive_pointer` values.

Genesis hash stability and rehydration vulnerability. The genesis hash is the hash of the first REGISTER event, which includes the `concept_id`; early versions excluded it, causing collisions

when different concepts shared the same initial payload (corrected in E1, §5.2). The canonical JSON serializer normalises line endings to LF and encodes Unicode as UTF-8; the version-locked `canon_version` field pins the serialization policy so that benign formatting changes (e.g., prettier Markdown reformatting) do not affect event hashes. When a chain is copied across repositories, two failure modes can fragment identity: *formatting drift* (different line endings or key ordering break the recomputed hash; mitigated by `canon_version`) and *metadata injection* (external systems adding fields such as a `signature` to legacy events change the canonical JSON). ADL Lite therefore treats the genesis hash as *write-once*: any migration adding new fields must store them in subsequent events, not retroactively modify the genesis event. The formal resilience criterion is: a chain C rehydrated in repository R' satisfies VerifyIntegrity$(C) = \text{true}$ if and only if C was well-formed in the source repository R and no event content was modified during transfer.

4.3.1 Dual-Identifier Design: Content Addressability

ADL Lite uses a *dual-identifier* scheme separating *provenance identity* (*who* created the concept and *when*) from *content identity* (*what* the concept semantically describes), addressing the identity-proliferation problem in multi-agent authoring.

Genesis hash (provenance identity). The genesis hash $h_{\text{gen}} = \text{SHA-256}(\text{Canon}(e_{\text{REG}}))$ includes the `concept_id`, the actor's identifier, and the creation timestamp; it is *unique per registration event* (two agents registering the same capability at different times yield different hashes) and is the *primary identifier* for all lifecycle audit.

Content hash (content identity). The content hash $h_{\text{content}} = \text{SHA-256}(\text{Canon}(\text{L2\_body}, \text{L3\_blocks}))$ excludes all registration-varying metadata; it is *stable across independent registrations* with identical (or near-identical, within a configurable Levenshtein threshold) L2/L3 descriptions.

Near-duplicate detection. On REGISTER, if h_{content} matches an existing concept (exact or within a Levenshtein ratio threshold of 0.85), the ActionExecutor emits an advisory warning ("This concept may be a duplicate of [existing concept]...") suggesting a RELATE event with predicate `isomorphic-to`; the agent may still register a separate entity. The content hash is computed in $O(|\text{L2}| + |\text{L3}|)$ time, is stored in the event payload, and is *not* part of the cryptographic chain (it is an advisory field, so it does not affect well-formedness or integrity).

Formal resilience criteria and DID vs. content-linked PID comparison. Any identity mechanism must satisfy: (i) *provenance uniqueness* (distinct identifiers for identical content), (ii) *rehydration stability* (identifier preserved across repository copies unless content changes), and (iii) *content-addressability option* (secondary content-derived identifier for near-duplicate detection). h_{gen} satisfies (i)–(ii); h_{content} satisfies (iii). A content-addressed PID (IPFS CID) would satisfy (ii)–(iii) but fail (i) by colliding on identical content; a DID (`did:key`) satisfies (i) but fails (ii) because DIDs are not content-sensitive. The dual-identifier scheme is the minimal design satisfying all three criteria simultaneously; Table 5 summarises the comparison.

4.4 Precondition Language and Action Executor

4.4.1 Formal Precondition Language

L4 action blocks are validated against preconditions expressed in a constrained, decidable language. A precondition rule is a triple $\langle \text{field}, \text{comparator}, \text{value} \rangle$ with comparator alphabet $\mathcal{K} = \{\text{EQ}, \text{NEQ}, \text{GT}, \text{GTE}, \text{LT}, \text{LTE}, \text{IN}, \text{EXISTS}\}$. Table 6 gives the formal semantics.

BNF grammar. Let $\mathcal{N}$ be the set of alphanumeric field identifiers, $\mathcal{S} = \mathbb{R} \cup \mathbb{B} \cup \Sigma^*$ the set of scalar literals (reals, booleans, strings), and $\mathcal{P}_{\text{fin}}(\mathcal{S})$ the finite power set of scalars. The precondition language $\mathcal{L}_{\text{pre}}$ is generated by the grammar $G = (\mathcal{V}, \Sigma_{\text{term}}, R, \text{rule\_list})$ with non-terminals $\mathcal{V} =$

Table 5: Identity mechanism comparison: genesis hash, content hash, DID, and content-addressed PID.

Criterion	$h_{\mathbf{gen}}$	$h_{\mathbf{content}}$	**DID**	**Content PID**
Provenance uniqueness	✓	×	✓	×
Rehydration stability	✓	✓	×	✓
Content-addressability	×	✓	×	✓
Cryptographic binding	✓	×	✓	✓
Human readability	×	×	✓	×

Table 6: Comparator semantics: explicit operator mapping to ground predicates.

Comparator	**Predicate**	**FOL encoding**
EQ	Equality	$x = y$
NEQ	Disequality	$\neg(x = y)$
GT	Greater than	$x > y$
GTE	Greater than or equal	$x \geq y$
LT	Less than	$x < y$
LTE	Less than or equal	$x \leq y$
IN	Set membership	$x \in Y$ (where Y is a finite set)
EXISTS	Non-null check	$\exists x : x \neq \text{null}$

$\{\text{rule\_list}, \text{rule}, \text{field}, \text{comparator}, \text{value}\}$, terminals $\Sigma_{\text{term}} = \mathcal{N} \cup \mathcal{S} \cup \{\text{EQ}, \text{NEQ}, \text{GT}, \text{GTE}, \text{LT}, \text{LTE}, \text{IN}, \text{EXISTS},$ and production rules R:

$$\begin{aligned}
\text{rule_list} &::= \text{rule} \mid \text{rule_list AND rule_list} \mid \text{rule_list OR rule_list} \\
\text{rule} &::= \langle \text{field}, \text{comparator}, \text{value} \rangle \\
\text{field} &::= f \in \mathcal{F} \quad \text{(the finite set of snapshot field names)} \\
\text{comparator} &::= \text{EQ} \mid \text{NEQ} \mid \text{GT} \mid \text{GTE} \mid \text{LT} \mid \text{LTE} \mid \text{IN} \mid \text{EXISTS} \\
\text{value} &::= s \in \mathcal{S} \mid P \in \mathcal{P}_{\text{fin}}(\mathcal{S}) \mid \text{null}
\end{aligned}$$

Formal semantics. Let $\mathcal{D}$ be the domain of values (scalars, sets, and null). Let $\text{Snapshot}(C)$ be the derived snapshot of chain C, a finite map from field names to $\mathcal{D}$. The semantic function $\text{eval}(r, C) \in \{\text{true}, \text{false}\}$ is defined for a rule $r = \langle f, \kappa, v \rangle$ by:

$$\text{eval}(\langle f, \kappa, v \rangle, C) = \text{apply}(\kappa, \text{lookup}(f, C), v) \tag{19}$$

where $\text{lookup}(f, C) = \text{Snapshot}(C)[f]$ (the value of field f in the snapshot, or null if absent), and $\text{apply}(\kappa, x, y)$ is the comparator-specific predicate from Table 6. The snapshot is pre-computed in $\mathcal{O}(n)$ time by a single scan of C; subsequent rule evaluations are $\mathcal{O}(1)$.

Composition semantics. A rule list $R = [r_1, \ldots, r_k]$ is evaluated as a conjunction: $\text{eval}(R, C) = \bigwedge_{i=1}^{k} \text{eval}(r_i, C)$. Evaluation is short-circuit: if $\text{eval}(r_j, C) = \text{false}$ for any $j < k$, the remaining rules are not evaluated. The language is a variable-free ground fragment: each rule is a single atomic comparison over a cached derived snapshot, with no quantification, no recursion, and no dynamic code execution. The conjunction of k rules is a variable-free conjunction of ground atoms, evaluated in $\mathcal{O}(k)$ time (Theorem 8).

Precondition evaluation is pure and referentially transparent: it references only the snapshot of the chain C being validated, and does not access external chains, databases, or network resources. Cross-chain preconditions (e.g., "allow VALIDATE only if concept X is `validated`") are not supported in the core language; they are deferred to future work (FW2). This local-scope restriction ensures that precondition evaluation is deterministic and executable without consensus or network access.

Static typing. A rule $\langle f, \kappa, v \rangle$ is well-typed, denoted $\vdash \langle f, \kappa, v \rangle : \text{bool}$, iff $\text{type}(f) \in \mathcal{T} = \{\mathbb{R}, \mathbb{B}, \Sigma^*, \mathcal{P}_{\text{fin}}(\mathcal{S})\}$ and the comparator–value pair satisfies the compatibility predicate: EQ/NEQ require $\text{type}(f) = \text{type}(v)$ with $v \in \mathcal{S}$; GT/GTE/LT/LTE require $\text{type}(f) = \mathbb{R}$ and $v \in \mathbb{R}$; IN requires $v \in \mathcal{P}_{\text{fin}}(\mathcal{S})$ with $\text{type}(f) = \text{type}(v_i)$ for all $v_i \in v$; EXISTS requires $\text{type}(f) \in \mathcal{T}$ with $v = \text{null}$. The ActionExecutor rejects any rule violating $\vdash$ before evaluation, so $[\![\rho]\!](\sigma)$ never encounters a runtime type error. The evaluation function is defined by exhaustive case analysis on the comparator—e.g., $\text{eval}(\sigma, \langle f, \text{EQ}, v \rangle) = (\sigma.f = v)$, $\text{eval}(\sigma, \langle f, \text{GT}, v \rangle) = (\sigma.f > v)$, $\text{eval}(\sigma, \langle f, \text{IN}, vs \rangle) = (\sigma.f \in vs)$, $\text{eval}(\sigma, \langle f, \text{EXISTS}, \text{null} \rangle) = (\sigma.f \neq \text{null})$; the remaining comparators (NEQ, GTE, LT, LTE) follow analogously.

Safety. The precondition language contains no `eval()`, no `exec()`, and no dynamic code execution. Comparator dispatch is a closed enumeration: the Python implementation uses a fixed `Comparator` enum with eight explicit cases, so no user-supplied string can be interpreted as code. Because σ is a precomputed hash map, field lookup is $O(1)$; because the comparator cases are a closed enumeration of primitive operations, each comparison is $O(1)$.

4.4.2 Complexity Analysis

Theorem (Precondition Evaluation Complexity). For any well-formed EventChain C with cached snapshot σ and any list of k precondition rules $R = [r_1, \ldots, r_k]$, total evaluation time is

$O(k)$, with $O(1)$ per rule and $O(1)$ auxiliary space.

Proof Sketch. Each rule performs a single field lookup in σ (a hash map, $O(1)$) and a closed comparison (a primitive operation, $O(1)$); the rule list is evaluated as a short-circuit conjunction with no quantification, recursion, or backtracking. The snapshot σ is precomputed in $O(n)$ at append time, so the per-rule cost is $O(1)$ amortized. (Full proof in Appendix A.5.)

Empirical validation. Experiment E34 (Precondition Formalization Benchmark) empirically confirms the $O(1)$ per-rule claim: per-rule evaluation measures 0.8–1.0 μs, independent of chain length (tested up to 1,000 events). This validates that the closed comparator dispatch and hash-map field lookup dominate the cost, with no superlinear term emerging from large snapshots.

4.4.3 Precondition Language Expressivity

The precondition language is intentionally minimal: a variable-free ground fragment with eight comparators, no recursion, no quantification, and no dynamic code execution. This minimalism guarantees decidability and $\mathcal{O}(1)$ per-rule evaluation (Theorem 8), at the cost of expressivity (Supplementary Table S12 classifies the boundaries). Time is not a first-class value: temporal constraints are enforced at the application layer, and cross-chain predicates are unsupported by design (the language evaluates only the snapshot of the chain being modified, ensuring referential transparency and eliminating the need for distributed consensus; cross-chain references would introduce non-determinism).

Theorem 8 (Precondition Evaluation Termination and Complexity). For any well-formed EventChain C and any precondition rule r, eval(r, C) terminates in $\mathcal{O}(1)$ time (assuming the derived snapshot is cached). For a list of k rules, total time is $\mathcal{O}(k)$ and space is $\mathcal{O}(1)$. *Proof Sketch:* Both field lookup and comparator dispatch are $\mathcal{O}(1)$; the language contains no recursion or quantification. See Appendix A.5 for the full decidability proof. Supplementary Table S13 summarises the complexity; the decidability and complexity-class discussion (membership in **P**) is in Appendix H.

Complete lifecycle transition matrix. Table 7 lists the ActionExecutor preconditions for each lifecycle event (encoded in `adl_core_ontology.yaml`). Under CRDT LUB semantics, the final status is the maximum over all lifecycle events, not the last event; the ActionExecutor still enforces preconditions to prevent semantically nonsensical sequences (e.g., archiving a concept never validated).

Table 7: Lifecycle event preconditions: permitted events from current status (rows). The final status is the LUB of all lifecycle events in the chain (CRDT semantics).

Status	REG.	VAL.	DEP.	FORK	ARCH.
provisional	× (gen.)	✓	✓	✓	✓
validated	×	✓ (re)	✓	✓	✓
deprecated	×	✓	×	×	✓
forked	×	✓	✓	×	✓
archived	×	×	×	×	×

REGISTER is permitted only as a genesis event (creating a new concept). From `validated`, VALIDATE is permitted as a re-validation (adding a new confidence assertion). Under CRDT LUB semantics, once a chain reaches `deprecated`, subsequent VALIDATE events do not change the status back to `validated` (the LUB of `deprecated` and `validated` is `deprecated`), but they can still be

appended as auditable evidence. Re-activation after deprecation requires a new REGISTER event (new genesis), not a reversal of the existing chain.

4.4.4 Action Executor and Transition Engine

The ActionExecutor validates L4 action blocks against a closed action registry defined in `adl\_core\_ontology.yaml`. Each action definition specifies preconditions as a list of `PreconditionRule` objects. The ConsensusEngine governs the resulting status transitions by appending events to the chain; the final status is derived via the CRDT LUB over all lifecycle events (Theorem 1), not by a transition matrix. Forking creates a new child chain with a fresh `REGISTER` event while the original chain's status is the LUB of its previous status and `forked`; both evolve independently.

4.5 Compact Formal Specification

This subsection presents a *single-page, self-contained* formal specification of ADL Lite's derivation semantics. All definitions, theorems, and complexity claims are stated here; expanded proof sketches and sensitivity analysis appear in Appendix E. The specification is organised as: (i) the status lattice and derivation function δ; (ii) the confidence aggregation function γ (three variants); (iii) the precondition language; and (iv) the fork and merge semantics.

4.5.1 Status Lattice and Derivation Function δ

Let $\Sigma = \Sigma_{\text{life}} \cup \Sigma_{\text{comm}}$ be the event alphabet. The lifecycle events are $\Sigma_{\text{life}} = \{\text{REGISTER, VALIDATE, DEPREC}$ The communication events are $\Sigma_{\text{comm}} = \{\text{RELATE, EVIDENCE, SEAL, ANNOUNCE, PUBLISH, SNAPSHO}$ A chain $C = [e_1, \ldots, e_n]$ is well-formed, denoted $\text{WF}(C)$, if it satisfies 13 axioms: genesis anchoring, shared `concept_id`, cryptographic linkage, hash correctness, distinct `event_id`, non-empty actor, timestamp monotonicity, payload schema conformance, action preconditions, scope ACL, signature verification, confidence clamping, and valid event type (the full list is in Appendix E).

The status space is $S = \{\text{provisional, forked, validated, deprecated, archived}\}$, equipped with the total order:

$$\text{provisional} \prec \text{forked} \prec \text{validated} \prec \text{deprecated} \prec \text{archived} \tag{20}$$

numbered $1, 2, 3, 4, 5$ respectively. This is a join-semilattice: the least upper bound (LUB) of any subset is its maximum element. The status map $f : \Sigma_{\text{life}} \to S$ assigns $f(\text{REGISTER}) = \text{provisional}$, $f(\text{VALIDATE}) = \text{validated}$, $f(\text{DEPRECATE}) = \text{deprecated}$, $f(\text{FORK}) = \text{forked}$, $f(\text{ARCHIVE}) = \text{archived}$.

Let $C_{\text{life}} = \{e \in C \mid e.\tau \in \Sigma_{\text{life}}\}$ be the lifecycle subsequence. The status derivation function $\delta : \{C \mid \text{WF}(C)\} \to S$ is:

$$\delta(C) = \begin{cases} \text{provisional} & \text{if } C_{\text{life}} = \emptyset \\ \max_{\prec}\{f(e.\tau) \mid e \in C_{\text{life}}\} & \text{otherwise} \end{cases} \tag{21}$$

where $\max_{\prec}$ is the maximum over the integer order (Eq. 20). Complexity: $O(|C|)$ by single scan of C (or $O(1)$ with an incremental LUB cache updated on every `append`).

Under Eq. 21, status is determined by the *highest* lifecycle event in the lattice, not the last. Once deprecated or archived appears, no subsequent event can lower the status (monotonicity). Re-activation after deprecation requires a new genesis event on a new chain.

4.5.2 Formal Lattice Structure

The status space $S = \{\text{provisional}, \text{forked}, \text{validated}, \text{deprecated}, \text{archived}\}$ equipped with the total order $\prec$ (Eq. 20) forms a complete lattice $(S, \preceq, \sqcap, \sqcup, \bot, \top)$ where:

- $\bot$ = provisional is the least element,
- $\top$ = archived is the greatest element,
- the join operator $\sqcup : S \times S \to S$ is defined by $\sqcup$ = status_max = $\max_{\prec}$, the least upper bound under $\prec$,
- the meet operator $\sqcap : S \times S \to S$ is defined by $\sqcap$ = status_min = $\min_{\prec}$, the greatest lower bound under $\prec$.

Because $\prec$ is a total order on a finite set, every subset $X \subseteq S$ (finite or infinite, though S itself is finite) has both a maximum element $\max_{\prec} X$ and a minimum element $\min_{\prec} X$. Therefore every subset has a least upper bound and a greatest lower bound, and $(S, \preceq, \sqcap, \sqcup, \bot, \top)$ is a complete lattice Davey and Priestley [2002]. The completeness is essential for CRDT convergence: the merge of any set of concurrent branches computes $\bigsqcup X$ for the lifecycle events X of the merged chain, which is guaranteed to exist and be unique (Theorem 9).

Figure 1: Hasse diagram of the status lattice $(S, \preceq, \sqcup, \bot, \top)$. The diagram is a linear chain because $\prec$ is a total order.

Join-semilattice proof. Because $\prec$ is a total order on a finite set, $\sqcup = \max_{\prec}$ is associative, commutative, idempotent, has identity $\bot$ = provisional, and is monotone (if $a \preceq b$ then $a \sqcup c \preceq b \sqcup c$). These properties are machine-verified in Coq (`Status.v`, Theorem T3) and hold for the CRDT merge semantics (Theorem T9); the identity property justifies the $O(1)$ incremental LUB cache (appending a non-lifecycle event, which contributes $\bot$, leaves the status unchanged). The meet operator $\sqcap = \min_{\prec}$ satisfies the dual five properties and is required only for the complete-lattice structure and future query semantics; the full algebraic enumeration is in Appendix A.5.

4.5.3 Confidence Aggregation: Three Variants

Let $V = [e \in C \mid e.\tau = \text{VALIDATE}]$ denote the validation subsequence. ADL Lite provides three confidence aggregation strategies.

Variant 1: Default (G-Counter max). The simplest strategy, used by default, takes the maximum confidence across all `VALIDATE` events (or `SNAPSHOT` if no validation has occurred):

$$\gamma_{\text{default}}(C) = \begin{cases} 0.0 & \text{if } V = [] \\ \text{clamp}_{[0,1]}\big(\max\{e.\text{payload}[\text{“confidence”}] \mid e \in V\}\big) & \text{otherwise} \end{cases} \tag{22}$$

where max is over all validation events (G-Counter semantics) and $\text{clamp}_{[0,1]}(x) = \max(0, \min(1, x))$. It is evaluated in $O(1)$ time by an incremental cache (`_cached_confidence`) that updates to $\max(\text{cache}, \text{clamp}(c))$ on every append; the canonical state is always recomputable from the event sequence in $O(|V|) \leq O(n)$, so the $O(1)$ claim refers to the memoized warm-index query, not the canonical derivation.

Variant 2: Bonus-formula aggregate (γ_{agg}). Mitigates collusion by aggregating per-actor maxima and adding a bonus for cross-validation:

$$\gamma_{\text{agg}}(C) = \begin{cases} 0.0 & \text{if } V = [] \\ \min\big(1.0,\ c_{\text{base}} + 0.05 \times (N_{\text{vals}} - 1)\big) & \text{otherwise} \end{cases} \tag{23}$$

where $N_{\text{vals}} = |\text{actors}(V)|$ and $c_{\text{base}} = \max(0.5, \frac{1}{N_{\text{vals}}} \sum_{a \in \text{actors}(V)} \phi(a, V))$, with $\phi(a, V) = \max\{e.\text{payload}[\text{“confid}$ $e \in V \wedge e.\text{actor} = a\}$. The 0.05 bonus per distinct validator rewards cross-validation; the 0.5 floor prevents a single low-confidence validator from dominating (sensitivity analysis in Appendix E).

Variant 3: Calibrated confidence (γ_{cal}). Weights each validator's confidence by a per-actor accuracy score:

$$\gamma_{\text{cal}}(C) = \begin{cases} 0.0 & \text{if } V = [] \\ \text{clamp}_{[0,1]}\Big(\frac{\sum_{a \in \text{actors}(V)} \text{accuracy}_a \cdot \phi(a,V)}{\sum_{a \in \text{actors}(V)} \text{accuracy}_a}\Big) & \text{otherwise} \end{cases} \tag{24}$$

where $\text{accuracy}_a \in [0, 1]$ is a per-actor accuracy score from an external calibration profile; complexity is $O(|V|)$.

Concurrent resolution, bounds, and algebraic structure. Concurrent lifecycle events commute because the LUB is order-independent (Eq. 21); for γ, per-actor maxima are computed first, and concurrent events from the same actor commute because max is idempotent, so once a high-confidence validation is recorded, subsequent lower-confidence assertions cannot decrease the aggregate (Theorem T5). All three variants are bounded in $[0, 1]$ by construction: γ_{default} uses $\text{clamp}_{[0,1]}$; γ_{agg} uses $\min(1.0, \cdot)$ with $c_{\text{base}} \geq 0.5$; γ_{cal} is a clamped convex combination (Theorem T4, proved in Coq). Supplementary Table S30 summarises the algebraic structure: γ_{default} is a pure G-Counter (max-semilattice) with full associativity, commutativity, and idempotence; γ_{agg} and γ_{cal} are commutative over the set of actors but not associative with respect to the full chain history, because their bonus/denominator terms depend on the cumulative actor set, which is not reconstructible from scalar intermediates. The incremental confidence cache satisfies $\text{cache} = \gamma(C)$ at all times (updated to $\max(\text{cache}, c)$ on VALIDATE for γ_{default}; recomputed in $O(|V|)$ for $\gamma_{\text{agg}}/\gamma_{\text{cal}}$) and can be reconstructed by a single linear scan, so it is a pure optimisation, not a source of truth.

Phase 1 limitations. The current collaborative-audit model lacks authenticated identity, so γ_{cal} accuracy scores are self-reported and do not prevent collusion (a single actor can report $\text{accuracy}_a = 1.0$); Sybil attacks are possible (one actor with 10 pseudonyms can drive γ_{agg} to 0.99). These are known limitations (L3, §6.3), demonstrated in adversarial experiment E14 (Table 10); mitigation requires authenticated identities and staking (Phase 3, FW9).

4.5.4 Confidence as G-Counter CRDT

The confidence function $\gamma_{\text{default}}(C)$ is a state-based G-Counter CRDT. Let C be a chain and $V \subseteq C$ the subsequence of VALIDATE and SNAPSHOT events. The confidence is defined as:

$$\gamma(C) = \max(\{c \mid \exists e \in V : e.\text{payload}[\text{“confidence”}] = c\} \cup \{0\}) \tag{25}$$

State-based G-Counter formalisation. In the state-based CRDT model, the confidence G-Counter is the tuple $(\Sigma, \text{inc}, \text{merge}, s_0)$ where $\Sigma = [0, 1]$ is the state type, $s_0 = 0$ the initial state, $\text{inc}(s, c) = \max(s, c)$ the increment, and $\text{merge}(s_1, s_2) = \max(s_1, s_2)$ the merge.

Theorem 4.1 (Confidence is a State-Based G-Counter CRDT)**.** The tuple $(\Sigma, \text{inc}, \text{merge}, s_0)$ defined above is a state-based G-Counter CRDT: the merge operation is associative, commutative, and idempotent, and the increment operation is monotonic with respect to the merge.

Proof Sketch. merge = max is associative, commutative, and idempotent; $\text{inc}(s, c) = \max(s, c)$ is monotone in s; and confidence values are clamped to $[0, 1]$, so the state never decreases. The full algebraic proof is in Appendix A.5.

The G-Counter semantics guarantee that once a high-confidence validation is recorded, subsequent lower-confidence assertions cannot decrease the aggregate (Theorem T5). This monotonicity is critical for audit: validators cannot retrospectively weaken a concept's confidence by appending low-confidence events.

4.5.5 Fork and Merge Semantics

A fork $\text{Fork}(C, a)$ appends a FORK event to the parent chain and creates a child chain C' with a fresh REGISTER event. The parent and child evolve independently:

$$\delta(C_{\text{fork}}) = \max(\delta(C), \text{forked}) \quad \text{(parent status, LUB)} \tag{26}$$

$$\delta(C') = \text{provisional} \quad \text{(child status, fresh genesis)} \tag{27}$$

where $C_{\text{fork}} = C \oplus [e_{\text{FORK}}]$.

Identity preservation under fork. Let C be a well-formed chain with genesis event e_0 and concept identifier *concept_id*(C). The fork operation $\text{Fork}(C, a)$ produces (C_{fork}, C') satisfying: (I1) parent identity preservation, *concept_id*(C_{fork}) = *concept_id*(C) and $h_{\text{gen}}(C_{\text{fork}}) = h_{\text{gen}}(C)$; (I2) child identity distinctness, $h_{\text{gen}}(C') \neq h_{\text{gen}}(C)$ (fresh genesis with distinct timestamp and actor); and (I3) lineage traceability, a FORK event $e_{\text{FORK}} \in C_{\text{fork}}$ with $e_{\text{FORK}}.\text{payload}[\text{“child\_concept\_id”}] =$ *concept_id*(C') establishes a cryptographic link from parent to child. Conditions (I2)–(I3) ensure the parent and child are distinct concepts and that the lineage is auditable via the parent's EventChain.

LWW-Set merge formal definition. Given concurrent well-formed branches B_1, B_2 derived from the same genesis event, the merged chain is defined as:

$$\text{Merge}(B_1, B_2) = \text{Linearize}\big(\text{Dedup}(B_1 \cup B_2)\big) \tag{28}$$

where $\text{Dedup}(E) = \{e \in E \mid \nexists e' \in E : e' \neq e \wedge e'.\text{event\_id} = e.\text{event\_id}\}$ removes duplicate events by `event_id`, and $\text{Linearize}(E)$ sorts the deduplicated set by the total order $\prec$ (Eq. 31). The merge satisfies the LWW-Set (Last-Write-Wins Set) semantics because the deduplication by `event_id` ensures that the most recent (or only) version of each event is retained.

Identity of the merge result. Let B_1, B_2 be well-formed concurrent branches with shared genesis e_0 and common concept identifier c. The merged chain $M = \text{Merge}(B_1, B_2)$ satisfies:

(M1) *concept_id*$(M) = c$; (M2) $h_{\text{gen}}(M) = h_{\text{gen}}(e_0) = h_{\text{gen}}(B_1) = h_{\text{gen}}(B_2)$; (M3) every event from both branches appears exactly once in M with all immutable content (`event_id`, payload, actor, timestamp) preserved; and (M4) optionally, the original branch hashes can be stored as metadata in a MERGE event payload to provide ancestry proofs.

Re-anchor operation. After linearization, the hash chain must be recomputed to restore cryptographic linkage. Let $E^* = [e_1^*, \ldots, e_n^*]$ be the linearized merged event sequence. The re-anchor operation Reanchor(E^*) computes:

$$e_1^*.\text{previous\_event\_id} \leftarrow \text{null}, \quad e_1^*.\text{prev\_hash} \leftarrow \text{genesis}, \tag{29}$$

$$\forall i > 1: \quad e_i^*.\text{previous\_event\_id} \leftarrow e_{i-1}^*.\text{event\_id}, \quad e_i^*.\text{prev\_hash} \leftarrow \text{SHA-256}(\text{Canon}(e_{i-1}^*)). \tag{30}$$

Each event's immutable content is preserved; only the linkage pointers are updated, so the merged chain has a *different* hash sequence from either parent branch but all original event content is preserved (post-merge integrity, not pre-merge ancestry preservation).

Total order $\prec$. For events e_i, e_j:

$$e_i \prec e_j \iff e_i.\text{timestamp} < e_j.\text{timestamp} \vee (e_i.\text{timestamp} = e_j.\text{timestamp} \wedge e_i.\text{event\_id} <_{\text{lex}} e_j.\text{event\_id}) \tag{31}$$

where $<_{\text{lex}}$ is lexicographic comparison of UUID strings. The order $\prec$ is a strict total order (reflexive closure $\preceq$, antisymmetry, transitivity, and totality all hold: timestamps are real numbers and UUID strings are totally ordered). Within a single chain, events are totally ordered by cryptographic linkage; across chains, the merge operation linearizes the merged set by sorting on $\prec$.

Concurrent conflict resolution. For δ: under CRDT LUB semantics, the status is the maximum over all lifecycle events in the chain, independent of append order. A DEPRECATE event always dominates a VALIDATE event (since deprecated > validated in the lattice), and an ARCHIVE event dominates all other lifecycle events. For γ: the G-Counter (max) over all VALIDATE events determines the confidence, independent of append order. Equivocation (the same actor appending contradictory events) is retained as auditable evidence, not resolved by the ordering.

4.5.6 Concurrent Event Interaction

Under CRDT semantics, concurrent lifecycle events commute because the join operator is associative and commutative. Representative scenarios: (1) two concurrent VALIDATE events with confidence 0.8 and 0.9 merge to $\gamma = \max(0.8, 0.9) = 0.9$, $\delta =$ validated; (2) concurrent VALIDATE ($c = 0.8$) and DEPRECATE merge to $\gamma = 0.8$ (G-Counter) and $\delta =$ deprecated (since validated $\prec$ deprecated); (3) concurrent FORK and VALIDATE on the parent yield parent status max(validated, forked) and a child at PROVISIONAL; (4) concurrent RELATE and REVOKE are both preserved, and the L3 query engine applies *HoldsAt* semantics using the total order $\prec$ (Eq. 31) to determine whether the relation is active.

The join-semilattice properties of the status space and the G-Counter properties of confidence are machine-verified in Coq (`CRDT.v`, Theorem T9) and model-checked in the TLA$^+$ specification (`specs/CRDTMerge.tla`) for chains up to 20 events.

4.5.7 Theorems: Summary of Proven Properties

Proof status and mechanization scope. All eleven theorems (T1–T11) and Lemma E2 are machine-verified in Coq 8.18.0. T8 (precondition complexity) is a Python-level implementation

property and is not formalized in Coq. The Coq development comprises approximately 2,500 lines across seven modules (Status, Event, Confidence, Chain, Invariants, CRDT, and Crypto). All algebraic proofs are closed with zero remaining `Admitted` lemmas. Three previously stubbed axioms (scope ACL, signature verification, and confidence clamping) have been replaced with their full definitions, introducing abstract Ed25519/SHA-256 cryptographic primitives in `Crypto.v`. A randomized trace checker (E24, 10,000 traces, length 2–100, 100% pass rate) provides independent property-based validation.

Three verification modalities. We distinguish three complementary verification strategies with different completeness guarantees:

(i) *Coq (unbounded structural induction):* Proofs by induction on the event sequence, valid for chains of arbitrary finite length. T1, T2, T3, T4, T5, T7, T9, T10, and T11 are proved for all chains over the full event-type alphabet. These are the primary formal guarantees.

(ii) *Python test suite (bounded empirical):* over 1,600 pytest cases (1,651 tests, 106 files, 91% coverage) including adversarial trials, boundary conditions, and randomized trace checking (E24: 10,000 traces, length 2–100). These validate the implementation against the specification for finite but large instances.

(iii) *TLA+ model checking (bounded confirmation):* The TLA$^+$ specs (`specs/EventChain.tla`, `specs/CRDTMerge.tla`, `specs/ConsensusEngine.tla`) are model-checked for chains up to length 20, two-branch CRDT merges, and $N_{\min} \leq 5$ validators. They serve as *confirmation* of the inductive Coq theorems for small instances, not as standalone proofs.

This dual-strategy (Coq for unbounded correctness, Python/TLA+ for bounded confirmation) is standard in formal methods Lamport [2002], Bertot and Castéran [2013].

If agent_2 disagrees at Step 5, they may append a FORK event: the parent becomes *forked* and the child begins at *provisional*, both independently well-formed.

4.6 Distributed Event Ordering and Concurrency Model

4.6.1 Clock Skew and Timestamp Assumptions

ADL Lite uses ISO 8601 UTC timestamps with no global clock assumption. The classical problem of ordering events without a shared clock (Lamport clocks Lamport [1978], vector clocks Mattern [1989]) is handled differently: each event carries the author's local timestamp, and monotonicity is enforced by `append()` via the adjustment rule

$$e_{\text{new}}.\text{timestamp} \leftarrow \max(e_{\text{new}}.\text{timestamp},\ e_{\text{last}}.\text{timestamp}) \tag{32}$$

(ensuring non-decreasing timestamps within a chain); clock skew between authors is resolved by the total order $\prec$ (Equation 31), with lexicographic `event_id` as the deterministic tiebreaker.

No global clock assumption. Skew of up to several seconds is tolerated because $\delta(C)$ and $\gamma(C)$ are computed over the *set* of lifecycle events, not their timestamps, and precondition evaluation uses the snapshot-derived state (L1 front matter), not timestamps. Timestamp ordering is used only for linearization during CRDT merge and for human-readable audit trails.

Deterministic CRDT merge ordering. For CRDT merge, events are canonically sorted by `event_id` (UUID), guaranteeing deterministic ordering even under clock skew. The timestamp is therefore for *audit/replay* (human-readable chronology), *not for consensus*: the consensus state (δ, γ) is derived from the event set, independent of the ordering used for linearization.

Theorem 4.2 (δ is Independent of $\prec$ for Lifecycle Events)**.** For any well-formed chain C and any permutation π of the events in C_{life} that preserves the total order $\prec$ within each branch, $\delta(C) = \delta(C_\pi)$, where C_π is the chain with lifecycle events reordered according to π.

Proof. By Eq. 21, $\delta(C) = \max_{\prec}\{f(e.\tau) \mid e \in C_{\text{life}}\}$, and the maximum over a finite set is independent of enumeration order; $\prec$ is used only for linearization during merge, so it does not affect δ. (Full details in Appendix A.5.) This independence is a direct consequence of the CRDT lattice semantics: status is the LUB over all lifecycle events, a commutative and associative aggregation, not a sequential state machine.

4.6.2 Concurrency Model: Single-Writer vs. Optimistic Concurrency

Within a single EventChain, ADL Lite enforces a single-writer model (one agent appends at a time, serialised by `threading.Lock`); for multi-agent collaboration, repository-level locking is provided by Git (with signed commits as tamper evidence). Optimistic concurrency is supported for multi-agent scenarios: agents append to local clones independently, and conflicts are resolved by CRDT merge (LWW-Set union with `event_id` deduplication) rather than by blocking. This avoids distributed consensus at append time while ensuring deterministic convergence (Theorem 9).

4.6.3 CRDT Merge and Precondition Interaction

Merged events do not re-evaluate preconditions retroactively: preconditions are checked at append time on the local chain, and merge only reconciles event sets. A merged event that would have violated a precondition in the merged context (e.g., a VALIDATE appended to a chain already containing a DEPRECATE) is retained as auditable evidence, while $\delta(C)$ reflects the LUB of all lifecycle events (so DEPRECATE dominates). This separation of *append-time validation* (local preconditions) from *merge-time reconciliation* (set union + linearization) ensures no events are silently dropped while preserving the monotonicity guarantees of the CRDT lattice.

4.7 Comparison with Formal Event Frameworks

In Event Calculus terms, $\delta(C)$ computes the fluent Status(c, t) by scanning the ordered event sequence, equivalent to EC's interval-based reasoning but computationally simpler ($O(n)$ vs. $O(n^2)$). ADL Lite avoids the frame problem by construction: there are no stored mutable fields, so all properties are recomputed from the event sequence. δ is computable in $\mathcal{O}(n)$ and γ in $\mathcal{O}(|V|) \leq \mathcal{O}(n)$; the implementation optionally memoizes derived snapshots in a warm index for $O(1)$ incremental evaluation, but the canonical state is always recomputable in $O(n)$. This corresponds to DL-Lite$_{\mathcal{R}}$ path queries and does not require full OWL-DL expressivity (NExpTime-complete). A full discussion is in Appendix H.

4.8 L3 Relation Management Across Forks

L3 relations (RELATE, REVOKE, EVIDENCE) are events in a specific chain and are *per-chain*, not global: a RELATE event in chain A does not affect chain B. When a chain is forked, the child starts with a fresh REGISTER event and does *not* inherit the parent's L3 relations (consistent with fork semantics: the child is a new concept); the parent may contain a FORK event with a `target_concept_id` establishing a `fork-of` relation recorded in the parent chain. Consequently, conflicting REVOKE/RELATE across forks are not conflicting—they exist in different chains, and a downstream consumer queries the `fork-of` lineage in the parent chain to reconcile them. When

two branches of the same chain are merged, L3 relations are merged by set union (LWW-Set): duplicate RELATE events are deduplicated by `event_id`, and a RELATE plus a REVOKE to the same target are both preserved, with `HoldsAt` semantics determining whether the relation is currently active. Relations are domain-level assertions that should be re-evaluated for each fork, not inherited mechanically.

4.9 Trust Model and Security Boundaries

The cryptographic hash chain ensures content integrity: any modification breaks the chain linkage and is detected by VerifyIntegrity(C). The scope of the current collaborative-audit model is deliberately limited to *collaborative non-Byzantine environments*—trusted organisations, open-source communities with social verification, and research groups where actors are known by reputation rather than cryptographic identity. Hash chains detect tampering but do not prevent it, and they provide no actor authentication; ADL Lite is a tamper-*evident* audit layer, not a tamper-*proof* security protocol.

4.9.1 Trust Model Evolution: Non-Byzantine to Authenticated

The trust model evolves incrementally from a collaborative non-Byzantine setting toward authenticated, economically secured validation (Supplementary Table S14). The current system (Phase 1) is designed for collaborative non-Byzantine environments where the cost of Sybil creation is social rather than computational; the phased approach allows incremental adoption with zero authentication overhead today.

Minimal Authentication Recommendation. For production use, we recommend $N_{\min} \geq 2$ distinct validators for any VALIDATE event triggering `validated` status, with validator identity anchored via `did:key` (Ed25519, no external dependencies). Public keys are stored in the local key registry (`key_registry.py`, itself an append-only EventChain), and the ActionExecutor verifies Ed25519 signatures on VALIDATE and FORK events. This does not prevent Sybil attacks (a single actor can still generate multiple key pairs) but raises the cost of identity creation and enables audit trails linking events to cryptographic identities. The DID resolver (`did_resolver.py`) supports `did:key`, `did:web`, and `did:ethr`, defaulting to `did:key` for zero-dependency deployment.

4.9.2 Trust Assumptions

We assume (1) trusted storage (Git or filesystem preserving history integrity, with signed commits providing tamper evidence), (2) trusted authoring environment (events injected by compromised agents are recorded as tamper-evident evidence), and (3) clock skew tolerance (ordering uses cryptographic linkage, not timestamps). Actor identity in the current implementation is a self-declared string; impersonation and replay are discussed below. Future authentication details are in Appendix K.

4.9.3 Current Trust Boundary: What Is and Is Not Detectable

The hash chain detects *content* tampering (modification of existing events) because any change breaks the hash linkage. However, the current model cannot detect the following *structural* and *identity* attacks: (i) *Actor impersonation:* any agent may declare an arbitrary string identifier; there is no cryptographic binding to a real-world identity. (ii) *Replay:* an event (or an entire chain) can be copied and re-submitted under a different concept identifier. (iii) *Sybil creation:* a single agent

can create multiple pseudonymous identities and self-validate using each. (iv) *Selective omission:* an agent may withhold evidence that would contradict their claim; the chain provides no mechanism to prove that a missing event *should* have been present. (v) *Historical rewrite:* an actor with repository-level access (e.g., Git push access) can force-push a rebased history, replacing the chain with a new genesis. These attacks are detectable by external audit (comparing a local clone against a signed reference), but they are not *prevented* by the chain itself. This characterisation explains why we describe the current model as a *collaborative non-Byzantine* setting.

4.9.4 Collusion Vulnerability Analysis

The confidence aggregation function $\gamma(C)$ is vulnerable to collusion in the current collaborative-audit model because it lacks authentication and incentive compatibility.

Lemma 4.1 (Collusion Vulnerability)**.** In the current collaborative-audit model, $\gamma_{\text{default}}(C)$ returns the maximum confidence across all `VALIDATE` events (G-Counter semantics). A single actor can drive $\gamma(C)$ to any value in $[0, 1]$ by appending a `VALIDATE` event with the desired confidence, achieving that confidence without independent validation. However, the actor cannot *lower* the confidence after a higher validation has been recorded, as G-Counter monotonicity (Theorem 5) prevents downgrade.

Lemma 4.2 (Minimum Validator Threshold Mitigation)**.** Let $N_{\min} \geq 2$ be a minimum validator threshold enforced as a precondition for `VALIDATE`. Then a single colluding actor cannot trigger the validated status; the collusion cost rises to $k \geq N_{\min}$.

Proofs. Both lemmas follow directly from Eq. 22 (the maximum over VALIDATE events is monotone, and a precondition $\langle N_{\text{vals}}, \text{GTE}, N_{\min} \rangle$ rejects events with $N_{\text{vals}} < N_{\min}$ before append). Full proofs are in Appendix A.5. The lemmas imply that a single actor can *set* confidence to any desired value but cannot *reduce* it after a higher validation (G-Counter monotonicity), and that a minimum-validator threshold prevents a single actor from triggering `validated` status; full collusion resistance requires authenticated identities and staking (FW9).

4.9.5 Known Limitations and Planned Mitigations

The current model has four limitations: (1) no actor identity verification for legacy string identifiers (a minimal `did:key` layer is implemented; full W3C LD-Proofs and DIDs remain future work); (2) no Byzantine resilience; (3) no equivocation prevention (contradictory events are recorded as auditable evidence); (4) genesis immutability relies solely on hash-chain detection and signed Git commits.

Three-phase authentication roadmap. Phase 1 (current): collaborative-audit model with self-declared string identifiers and SHA-256 hash-chain integrity. **Phase 1.5 (implemented):** minimal `did:key` resolution without external dependencies; Ed25519 signatures in the `signature` field; `verify_signature()` validates both DID-derived and registry-derived keys. **Phase 2 (planned):** full Ed25519 key registry with `KEY_ROTATE` and `KEY_REVOKE` events; DID document storage with `did:web` and `did:ethr`; W3C Linked Data Proofs. **Phase 3 (future):** BFT transport layer; CRDT-style merge with semantic policies; Ontological Assertion Market (staking/slashing for incentive-compatible validation). Authenticated identities change the threat model: Sybil attacks become prohibitively expensive and collusion cost rises to economic stakes (FW9).

4.9.6 Threat Model Summary

Supplementary Table S15 summarises the progression from the current release to future releases; the full discussion, including per-threat attack trees, signed-commit anchoring details, and recovery procedures, is in Appendix K. **Phase 1.5** provides Ed25519 `did:key` resolution and signature verification (§4.9), mitigating impersonation, replay, and genesis replacement relative to string identifiers, but does not prevent Sybil creation or collusion (no economic cost).

The CRDT merge semantics are **implemented in the alpha codebase** (as of v0.3.5). The merge operates at the *set* level (events as elements): the merged event set is deduplicated by `event_id`, linearised by sorting on `timestamp` (with `event_id` tie-breaking), and the hash chain is recomputed over the merged sequence. All original event content is preserved (via immutable `event_id` and payload); the merged chain therefore provides *post-merge integrity*, not *pre-merge ancestry preservation* (ancestry proofs can be stored as metadata in a MERGE event payload).

Theorem 9 (CRDT Convergence). For any well-formed concurrent branches B_1, B_2 derived from the same genesis event, the LWW-Set merge satisfies commutativity, associativity, and idempotence, and $\delta(\text{Merge}(B_1, B_2))$ is uniquely determined.

Proof. (i) Commutativity follows because set union is commutative and the deduplication predicate depends only on the unordered set of `event_id` values. (ii) Associativity follows because Dedup and Linearize are idempotent and commute with set union under the fixed ordering $\prec$. (iii) Idempotence: $\text{Merge}(B_1, B_1) = \text{Linearize}(\text{Dedup}(B_1 \cup B_1)) = B_1$ since well-formedness guarantees distinct `event_id` values within a branch. (iv) Determinism of δ follows from Theorem 4.2. (v) Re-anchor correctness: Canon is deterministic and SHA-256 is collision-resistant, so the recomputed hash chain is unique for the given linearization. The full proof is in Appendix A.5.

We evaluated Theorem 9 through executable assertions (1,651 tests, all passing through v0.6.0-alpha) and stress tests (E6: 201 chains, 9,300 events; E13: 50,000 events; E23: 20 concurrent agents) with zero integrity failures; machine-checked proofs for unbounded chains are provided by the Coq development (`CRDT.v`, Theorem T9).

Migration from last-event to CRDT: completed. The earlier last-event fork resolution was a special case of CRDT merge with a single branch. The migration to full CRDT compliance is complete: *Phase 1* (last-event with total order $\prec$) and *Phase 2* (CRDT lattice semantics for status LUB and confidence G-Counter max) are implemented, under which `REGISTER`, `RELATE`, `EVIDENCE`, and `SEAL` events commute. *Phase 3 (future work)* adds semantic merge policies for conflicting lifecycle events (e.g., VALIDATE vs. DEPRECATE, equivocation): (a) quorum-based resolution ($\geq N_{\min}$ validators), (b) accuracy-weighted ordering, (c) stake-weighted resolution (FW9). Blocklace Almeida and Shapiro [2024] provides the BFT transport layer for Phase 3, with ADL Lite providing the application semantics (lifecycle derivation, precondition enforcement) and a shim layer mapping Blocklace events to ADL Lite events.

4.10 Implementation Infrastructure

ADL Lite v0.6.0-alpha includes a functionally complete implementation layer that is summarised here; full module descriptions are in Appendix A.6. **Calibration** (`calibration.py`) provides four strategies (linear, EWMA, epistemic-context, per-band) for accuracy-weighted confidence aggregation, mitigating overconfidence in $\gamma(C)$. **Semantic Web interoperability** (`owl_export.py`, `owl_import.py`, `rdfstar_export.py`) enables bidirectional OWL 2 DL and RDF-star export. **Split-lock concurrency** (`EventChain`) uses dual `threading.RLock` instances to support 10^4-agent contention without data races (E28), with incremental verification caching for $O(1)$ post-append checks. **Vector search** (`vector_index.py`) provides a FAISS-backed ANN index for semantic

near-duplicate detection. **Merkle anchors** (`merkle.py`) enable $O(\log n)$ batch inclusion proofs. **LLM canonicalisation** (`canonicalization.py`) clusters concepts by embedding similarity and proposes auditable merge actions via an LLM backend, with a mandatory dry-run gate. **REST API** (`api.py`, 481 lines) and **MCP server** (`mcp_server.py`, 458 lines) provide programmatic and agent-protocol interfaces. **SHACL validation** (`shacl_validation.py`, 334 lines) checks exported RDF conformance. **Neo4j adapter** (`neo4j_adapter.py`, 148 lines) provides a scalable graph backend. **CRDT multi-way merge** generalises binary merge to $N \geq 3$ chains via `functools.reduce`, preserving commutativity, associativity, and idempotence (Theorem 9).

5 Empirical Validation

5.1 Validation Strategy: Correctness as Capability-Registry Consistency

We verified architectural correctness through eight structured tests: chain integrity (E1), status derivation (E2), snapshot round-trip (E3), precondition enforcement (E4), scalability on real-world data (E6), long-chain stress (E13), multi-agent contention (E16, E23), and template effectiveness (E20). These validate specific formal properties; generalisation to Byzantine settings, multi-domain deployments, and human-in-the-loop evaluation is future work (FW4, FW5).

Scope disclaimer. The experiments reported in this section verify *architectural correctness*—that the implementation conforms to the formal specification (Theorems 1–9) under the collaborative non-Byzantine trust model. They do *not* constitute domain-level validation of whether ADL Lite improves real-world outcomes in AML detection, literature review, or other application domains. The AML dataset (E6) is used as a volume stress-test and illustrative mapping, not as a clinical or operational validation. Independent expert evaluation (E5, in progress) is required for domain validity claims.

5.2 Core Correctness: Integrity, Derivation, and Preconditions (E1–E4)

E1: Chain Integrity and Tamper Detection. E1 tested VerifyIntegrity(C) on 60 chains (50 valid, 10 corrupted). Corrupted chains covered content tampering, event reordering, deletion, and replay. Precision, recall, and F1 were all 1.0.

E2: Status Derivation Exhaustiveness. E2 tested $\delta(C)$ on all event sequences of length up to 3 over the full event-type alphabet (13 types), plus 7 edge cases, totalling 3,382 cases (exhaustive over 13^3 combinations, including communication-event interleavings). All 3,382 cases were correct. The status derivation function is defined as the least upper bound (LUB) over the lifecycle lattice: $\delta(C) = \bigsqcup_{e \in C_{\text{life}}} f(e.\tau)$, where f maps event types to their lattice rank. This is equivalent to the CRDT join-semilattice semantics (Theorem 9). Communication events (e.g., `RELATE`, `EVIDENCE`) map to provisional and do not affect the LUB; lifecycle events drive monotonic transitions. Notably, the LUB semantics differ from a "last-event-wins" rule: for example, a chain with events [Register, Validate, Deprecate, Validate] derives δ = deprecated (the maximum lattice element present), not validated (the last lifecycle event). The inductive correctness of this derivation was proved in Coq (Theorem `E2_inductive_status_derivation`, Appendix I) and was checked by TLC for chains up to length 20 (Appendix I).

E2-ext: Random sampling for length > 3. We randomly sampled 10,000 sequences of length 4–10 with uniform event-type distribution. All 10,000 were correct. The Clopper–Pearson 95% CI for 10,000 successes and zero failures is $[99.963\%, 100.0\%]$ (rule-of-three: $[99.97\%, 100.0\%]$). This strengthens confidence that the finite-state logic generalises beyond the exhaustively tested boundary; the Coq proof (Appendix I) provides the inductive argument for arbitrary length.

Failure case analysis. Three bugs were fixed during development: (i) missing `confidence` defaulted to 0.0; (ii) genesis hash collision resolved by including `concept_id`; (iii) duplicate `event_id`s from unsynchronized threads fixed via thread-local counter and lock. All are covered by regression tests (E4a, E15, E16).

E3: Snapshot Round-Trip Consistency. E3 tested consistency between derived snapshot and original front matter for 39 Markdown documents (6 examples + 33 AML-derived concepts) (39/39 passed, 100%). No status mismatches, confidence deviations exceeding ±0.01, or validator set discrepancies occurred.

E4: Precondition Enforcement. E4 tested precondition validation on all 9 registered actions across 19 test cases, extended with 72 boundary cases and 50 multi-agent concurrency cases (141 total). Categories included concurrent execution (E4a), extreme confidence values (E4b), long-chain evaluation (E4c), multi-agent stress (E4d), and adversarial tampering (E4e, Section 5.8). Precision, recall, and F1 were all 1.0. The closed `Comparator` enumeration eliminates runtime code injection.

5.3 Scalability on Real-World Data (E6, E6b)

E6 subjected the EventChain to a volume stress using the IBM AML HI-Small dataset IBM [2024] (9,300 transactions from 201 accounts) on an Apple M2 (8P+4E cores, 3.49 GHz), 16 GB LPDDR5, macOS 14, Python 3.10. Mean throughput was 20,847 events/s ($\sigma = 312$, CV = 1.5%), with wall-clock runtime 446 ms ($\sigma = 7$). All 201 chains maintained integrity. Raw CSV parsing completed in 98 ms; EventChain construction incurred a 3.5× overhead, dominated by Pydantic materialization (58.4% of latency via `cProfile`). Full latency decomposition and reproduction environment are in Appendix D.

AML-to-EventChain mapping. The 201 chains were constructed via a 3-stage pipeline: (1) select the 201 highest-volume suspicious accounts from the dataset (3,386 flagged accounts, 5,080,714 transactions); (2) extract five features (`total_outbound`, `unique_recipients`, `max_single_amount`, `transaction_count`, `temporal_span_hours`) and project into the event payload; (3) generate a 3-event chain per account: REGISTER (actor: `data_importer`, status: provisional), VALIDATE (actor: `synthetic_validator`, confidence: $0.7 \pm U(-0.1, 0.1)$), and EVIDENCE (actor: `synthetic_validator`, evidence type: "transaction_pattern"). Random seed was 42. All chains passed `verify_integrity()`. They do not capture contested validations, forks, or deprecation; audit realism is tested by E17 and E4e.

E6b: Scale-up Feasibility. On identical hardware, ADL Lite ingested 10^6 concepts (2×10^6 events) in 133.9 s (≈14,938 events/s), confirming linear throughput. Nanopublications (rdflib) completed 10^6 concepts in 119.5 s (33,471 triples/s); PROV-O (`prov` library) completed 10^5 in 18.7 s (extrapolation to 10^6: 186.9 s); Git-only (batch commits, 100 concepts per commit) completed 10^4 in 0.94 s (extrapolation to 10^6: 94.3 s). All ADL Lite and nanopublication figures are empirical; PROV-O and Git-only are linearly extrapolated from smaller-scale runs. The architecture is feasible for 10^5–10^6 event deployments without redesign.

AML dataset licensing and provenance. The IBM AML Synthetic Dataset (HI-Small) is synthetic data generated by IBM for research purposes. No real customer data is included. The dataset is available under CC-BY-SA 4.0. The transformation pipeline used in E6 is: raw CSV → feature extraction (5 fields) → event generation → ADL Markdown. All intermediate artifacts are included in the repository (`data/aml/`) and are themselves released under the same CC-BY-SA 4.0 license, ensuring full reproducibility without intellectual property barriers.

5.4 Boundary Conditions and Stress Tests (E13–E16, E20–E23)

Four boundary experiments (E13–E16) and three stress tests (E20–E23) quantified system limits. E13 verified linear integrity scaling to 50,000 events ($R^2 = 1.0$, < 1 MB memory). E14 confirmed that a single colluding actor could drive $\gamma(C)$ to 0.99 through repeated self-validation under all aggregation variants (γ_{default}, γ_{agg}, γ_{cal}). When $N_{\min} = 2$ (minimum-validator threshold) was enforced as a precondition for VALIDATE, the same colluding actor could no longer trigger `validated` status, confirming Lemma 2 (Appendix E). Full collusion resistance requires authenticated identities and economic staking (FW9). E15 showed that 4/11 malformed inputs were caught by Pydantic rather than by preconditions, revealing a gap in defence-in-depth coverage; two inputs bypassed both layers and were recorded as auditable events, motivating stricter precondition coverage. E16 showed conflict rates rose to 95% for $k = 20$ concurrent agents, with integrity preserved but without graceful degradation. Anomalies included temporal ordering inversion (resolved by `event_id` tiebreaker), duplicate self-validation, and partial fork visibility causing transient precondition failures. All were recorded as auditable events; no integrity violations occurred. E20 evaluated L2 template effectiveness (94% strict / 87% relaxed compliance, $R^2 = 0.73$ calibration); E20b confirmed $R^2 = 0.73$ correlation between per-actor accuracy and calibrated confidence; E21 stress-tested 100k events; E23 measured contention under 20 concurrent agents (the `race_conditions` counter records benign check-then-act retries—semantic conflicts detected and abandoned; integrity_rate = 1.0 confirms no chain corruption under 20-agent contention). Full details are in Appendix D.

E25: Microbenchmark. We quantified precondition rule-list length and confidence aggregation variant performance. For precondition complexity, eval(R, C) with $|R| = k$ rules grew linearly: 0.7 μs for $k = 1$ to 31.7 μs for $k = 50$ (0.7, 3.0, 6.0, 11.7, and 31.7 μs for $k = 1, 5, 10, 20, 50$), confirming the $\mathcal{O}(k)$ bound of Theorem 8. For confidence aggregation, γ_{default} was $O(1)$ (constant 0.2 μs); γ_{agg} was $O(|V|)$ (2.2 μs for $|V| = 20$); γ_{cal} was also $O(|V|)$ but with a substantially larger constant (2,152.7 μs $\approx$ 2.15 ms for $|V| = 20$) because the calibration variant recomputes per-actor accuracy-weighted aggregation over the validator set. Calibration overhead is therefore $O(|V|)$ with a larger constant than γ_{agg}. Storage per event: ADL Lite ≈150 bytes, Git-only ≈200 bytes, PROV-O ≈350 bytes (computed in the E25 benchmark). These validate that the design choices are computationally feasible and the overhead relative to baselines is modest.

5.5 Domain Applicability: AML Case Study and Future Evaluation Design

To illustrate domain structuring, the following case study models an AML laundering pattern as a capability lifecycle. The mapping is not automatic: domain experts must translate raw transactions into ADL events. For example, AML fields `SenderID`, `ReceiverID`, `Amount`, and `Timestamp` are projected into an event payload; the evidential threshold ($\geq$5 unique recipients, volume > \$10,000) is formalized as a precondition rule.

Illustrative case study: fan-out laundering pattern. We modeled the "fan-out" pattern (source account → $\geq$5 unique recipients within 24h, volume > \$10,000) as an ADL Lite capability with L1 identity, L2 narrative, L3 relations (e.g., `co-occurs-with` to `rapid-movement`), and L4 lifecycle events. Three authors independently scored the concept on semantic coherence, evidential sufficiency, and audit completeness (5-point Likert). Mean scores were 4.3 ($\sigma = 0.6$), 3.7 ($\sigma = 0.6$), and 4.7 ($\sigma = 0.6$), respectively. The evidential sufficiency score reflects the absence of external ground-truth labels (addressed by the in-progress E5 evaluation with AML experts). This is preliminary and author-biased; it is not a substitute for independent expert evaluation.

Worked example: AML-derived EventChain. To make the mapping concrete, Supplementary Table S31 shows a complete 3-event chain for a representative account (`HI-Small-42`) from

the IBM dataset. The account exhibited 47 transactions totalling $127,400 to 12 unique recipients within 48 hours, triggering the fan-out pattern. The L1 front matter captures the derived identity; the L2 Markdown body narrates the pattern; the L3 relation links to `rapid-movement`; and the L4 events record the full lifecycle from registration through synthetic validation to evidence attachment.

This chain is representative of the 201 AML-derived chains: all follow the REGISTER → VALIDATE → EVIDENCE pattern, with feature payloads derived from the raw transaction CSV. The chain passes `verify_integrity()` and the derived snapshot matches the L1 front matter (E3, 39/39). The `synthetic_validator` actor string indicates that the validation was algorithmically generated from the feature threshold (confidence $= 0.7 \pm U(-0.1, 0.1)$), not by a human AML analyst. This is a deliberate limitation: the current study validates the *infrastructure* (chain construction, integrity, snapshot consistency), not the *domain judgment* (whether 12 recipients in 48h truly indicates laundering).

AML dataset limitation and expert validation methodology. The AML-derived dataset (E6) uses synthetic validators and synthetically injected laundering patterns; it serves as a volume stress test and a structural illustration of the L1–L4 mapping, not as a validation of laundering-pattern detection accuracy. To address the domain-validation gap, we designed an inter-annotator agreement study (target $n \geq 15$ AML analysts; 5 industry and 3 academia recruited): a 5-point Likert rubric covering semantic coherence (does the L1–L4 structure faithfully represent the pattern?), evidential sufficiency (are thresholds justified?), and audit completeness (can a third party reconstruct the reasoning?); each concept rated by 3 independent experts with discrepancies resolved via discussion and recorded as `EVIDENCE` events; metrics are Fleiss' κ, Pearson correlation between expert consensus and automated $\delta(C)$, and sensitivity/specificity against expert majority. IRB approval was obtained (protocol ADL-2025-AML-01, 2025-06-15). As an interim signal, GPT-4o (zero-shot) evaluated the same 50 concepts: Pearson correlation with the 3-author mean was $r = 0.71$ (coherence), $r = 0.58$ (evidential sufficiency), and $r = 0.82$ (audit completeness); the lower evidential-sufficiency correlation reflects the absence of external ground-truth labels. The rubric follows established AML evaluation practice Öztaş et al. [2024], Others [2025], FATF [2025], and the approach extends the AML knowledge-graph precedent Facctum [2026], Vishal [2025] with lifecycle governance (pattern registration, validation, deprecation, fork), addressing the "model governance risk" identified in AML AI deployments Sutherland Global [2024]. Full expert results will be reported in a follow-up study.

Multi-agent near-duplicate reconciliation example. Two agents independently registered "gradient-explosion" and "exploding-gradients". A third agent detected near-duplicates via string similarity $s = 0.91$ and embedding distance $d = 0.03$, appended a `RELATE` event with predicate `isomorphic-to`, then forked the latter to the former and deprecated the latter. Every decision was an auditable event with actor attribution and reasoning, resolving multi-agent conflicts without central coordination.

End-to-end illustrative case study: weather-data-retrieval. Three agents illustrated the full lifecycle: `agent_1` registered the capability (`provisional`); `agent_2` validated with confidence 0.85 (`validated`); `agent_3` dissented with confidence 0.45 (confidence remained 0.85 under G-Counter semantics, preserving the dissent as auditable evidence); `agent_1` forked to "weather-data-retrieval-v2" (parent remained `validated`, child started `provisional`); `agent_2` deprecated the parent. A downstream consumer querying for `validated` capabilities excluded the parent and received the child, flagged for manual review—demonstrating actor attribution, fork-and-deprecation conflict resolution, and downstream trust filtering.

E5: Illustrative multi-agent literature review case study. To demonstrate how ADL Lite represents collaborative capability lifecycles, we structured a simulated 5-agent literature review pipeline (Scout, Analyst, Writer, Critic, Coordinator) with 17 capabilities registered, cross-

validated, and iteratively improved through fork and deprecation. All data were generated by a simulation script (`case_study/run_case_study.py`) structuring events according to the ADL Lite model. Quantitative metrics (P@10, Cohen's κ, overstatement rate) are calibrated against published LLM-agent benchmarks for realism but are *not* empirically measured in a live deployment. The case study provides *construct validity evidence* that the framework can represent and audit collaborative capability lifecycles, including cross-validation, negative-result transparency, and iterative improvement. It is an *illustrative simulation*, not a validation of real-world effectiveness.

The study generated 19 EventChains (17 initial + 2 forked) with 79 events: 19 `REGISTER`, 38 `VALIDATE`, 17 `EVIDENCE`, 1 `DEPRECATE`, 2 `FORK`, and 2 `RELATE`. Each capability was cross-validated by 2 agents from different roles. Final status: 18 `validated`, 1 `deprecated`; confidence ranged 0.40–0.91 (mean 0.753). Supplementary Table S32 summarises cross-validation for 6 Scout/Analyst capabilities.

The `trend-detection` capability was validated with low confidence (critic: 0.45, coordinator: 0.52) and deprecated after evidence revealed a 60% false-positive rate on 1-year windows; two capabilities were subsequently forked to address identified flaws (`trend-detection-v2` with a 3-year moving average, re-validated at 0.83; `abstract-generation-calibrated`, reducing overstatement from 40% to 8% and improving quality from 3.4 to 4.1/5), demonstrating how the EventChain captures capability evolution through structured disagreement rather than silent mutation. The Coordinator also appended 2 `RELATE` events (`methodology-evaluation complements methodology-critique`; `literature-search feeds-into reference-management`). This is a simulated workflow; it does not prove improved real-world outcomes, which requires the planned human expert study described in §5.5.

5.6 Multi-Agent Collaboration Simulation (E17)

In the deterministic simulation (5 agents, 3 concepts, 20 rounds), preconditions rejected 100% of invalid transitions (preconditions ON), versus 0% rejection in the free-form baseline (preconditions OFF, all actions accepted). Precondition effectiveness is therefore 100% under the script's deterministic protocol (`experiments/e17_multi_agent_collab.py`). Real-world evaluation with stochastic human or LLM agents is future work (E20).

5.7 Comparative Governance Analysis (E19, E12)

E19: Empirical audit cost benchmark. Table 8 presents a head-to-head comparison of ADL Lite against nanopublications (rdflib), PROV-O (`prov` library), and Git-only (`pygit2`) on four standard audit tasks. All measurements were taken on identical hardware (Apple M2, macOS 14, Python 3.10). Full methodology is in Appendix L.

Table 8: Empirical audit cost benchmark (E19): measured cost across 4 tasks. **All values are measured via actual execution on identical hardware.** LOC counts client code using each system's public API, not the system implementation itself.

System	LOC	Latency (ms)	Errors	Completed	Audit
S1. ADL Lite	27	0.5	0	4/4	1.00
S2. Nanopub + scripts	21	1.0	0	4/4	0.82
S3. PROV-O + scripts	53	2.0	0	4/4	1.00
S4. Git-only	45	42.0	0	4/4	1.00

The E6b projection (47.7 s for 1×10^6 events) was based on CSV import throughput; the E19

measurement (133.9 s for 2×10^6 events) includes independent EventChain object creation with Pydantic validation. Both are correct for their respective operations; the E19 figure is the more conservative bound for in-memory usage.

Qualitative baseline analysis. Three structural differences emerged: (i) *Authoring friction:* ADL Lite uses Markdown with YAML front matter, whereas nanopublications require Turtle/RDF and PROV-O requires RDF/SPARQL tooling (Git-only lacks structured capability metadata); (ii) *Audit completeness:* ADL Lite records every lifecycle event with actor attribution and reasoning, whereas nanopublications record static assertions without lifecycle transitions, PROV-O records provenance but not status governance, and Git tracks versions without validate/deprecate/fork semantics; (iii) *Conflict resolution:* ADL Lite provides deterministic CRDT lattice semantics (LUB for status, G-Counter for confidence), whereas nanopublications and PROV-O lack built-in lifecycle conflict resolution and Git has no semantic merge policies. These are complementary systems; ADL Lite combines these properties into a single lightweight package.

E12: Qualitative and quantitative comparison. Supplementary Tables S1 and S3 present the audit-task comparisons against nanopublications with Trusty URIs and a PROV-O pipeline. Full interpretation is in Appendix F.

PROV-O with LD-Proofs: signature overhead. The PROV-O column in Supplementary Table S1 reflects the base PROV-O specification without signatures. When W3C Linked Data Proofs (LD-Proofs) W3C Verifiable Credentials Working Group [2024] are added as an external layer, each activity, agent, and entity requires an Ed25519 or RSA signature, with verification cost comparable to ADL Lite's Phase 1.5 `verify_signature()` implementation. The key difference is architectural: LD-Proofs are *external* to PROV-O (added by a separate execution layer), whereas ADL Lite's signatures are *native* to the EventChain data structure (stored in `event.proof` and verified during `verify_integrity()`). This co-location means signature verification is integrated with lifecycle state derivation; a signature failure in ADL Lite invalidates the chain at the point of failure, while in PROV-O+LD-Proofs, signature verification and lifecycle state derivation are decoupled operations that must be orchestrated by the application layer.

Supplementary Table S2 presents the five-task lifecycle comparison (register $\rightarrow$ validate $\rightarrow$ deprecate $\rightarrow$ fork $\rightarrow$ audit query) that was the basis for the E19 benchmark. The fork task is the critical differentiator: ADL Lite provides a native FORK event with deterministic child-chain status derivation, while nanopublications and PROV-O lack a first-class fork concept and must simulate lineage via external metadata.

[a]Estimated from published specifications; not empirically measured on identical hardware.

5.8 Adversarial Integrity Trials (E4e (extended))

Table 9 reports 7 detectable attack scenarios across 201 chains (1,407 trials). Table 10 reports 4 undetectable scenarios confirming the current trust-model boundary. Full methodology and failure-mode analysis are in Appendix C.

5.9 Adversarial Robustness Baseline Comparison (E37)

The adversarial trials in Section 5.8 establish ADL Lite's detection capabilities under the Phase 1 threat model. To contextualise these results, we conducted a systematic head-to-head comparison of adversarial robustness across four baseline systems: ADL Lite (S1), Nanopublications with Trusty URIs (S2), PROV-O with LD-Proofs (S3), and Git-only (S4). The comparison maps each of the 11 attack scenarios from Tables 9 and 10 to the corresponding detection or prevention mechanism in each baseline, producing a *qualitative* specification-based classification.

Table 9: Adversarial integrity trials: 7 attack scenarios × 201 chains = 1,407 trials. All attacks were detected with 100% recall in the synthetic Phase 1 trials.

Scenario	**Attack vector**	**Detection mechanism**	**Detection rate**
A1. Mid-chain insertion	Insert event at index $k < n$	`previous_event_id` mismatch	201/201 (100%)
A2. Event deletion	Remove event at index k	Hash mismatch at $k + 1$	201/201 (100%)
A3. Event reordering	Swap events i and j	Hash chain breaks at swap point	201/201 (100%)
A4. Payload tampering	Mutate event payload	`hash` $\neq$ `compute_hash()`	201/201 (100%)
A5. Identity spoofing	`VALIDATE` by nonexistent actor	Precondition audit (recorded)	201/201 (100% audit)
A6. Fork double-counting	Duplicate `VALIDATE` across forks	Per-actor maxima in $\gamma(C)$	201/201 (100%)
A7. Replay attack	Copy event from chain A to chain B	`concept_id` mismatch + hash break	201/201 (100%)

Table 10: Boundary verification: attacks that the current collaborative-audit model is explicitly **not** designed to detect.

Scenario	**Attack vector**	**Current model result**	**Planned mitigation**
B1. Collusion attack	5 colluding actors self-VALIDATE to $\gamma(C) = 0.99$	$\gamma(C)$ reaches 0.99; `VerifyIntegrity` passes	Staking + calibration (FW9)
B2. Genesis replacement	Replace entire chain with re-computed hashes	`VerifyIntegrity` passes (hashes are valid)	Git reflog + DIDs (future release)
B3. Impersonation	Forge events with another actor's `actor` string	`VerifyIntegrity` passes (no signature)	Ed25519 signatures + DIDs (future release)
B4. Timestamp backdating	Modify `timestamp` and re-compute hash	`VerifyIntegrity` passes (timestamp in hash)	NTP + signed timestamps (future release)

Supplementary Table S4 reports the result. For each attack scenario, we classify the baseline response as: ✓ (detected or prevented natively), ∘ (detected with external tooling or after-the-fact audit), or × (not detected in the baseline configuration). The classification is based on the published specifications and reference implementations of each system, not on new empirical measurements.

Analysis. Three structural patterns emerge. (i) *Content integrity:* all four baselines detect content tampering (A1, A4) via cryptographic hashes, with different binding mechanisms (per-event SHA-256 chaining, Trusty URI content-addressing, LD-Proof signature verification, blob hashing/commit linkage). (ii) *Structural integrity:* ADL Lite, PROV-O, and Git-only detect reordering (A3) and deletion (A2) via linkage mechanisms, whereas nanopublications lack native structural ordering. (iii) *Lifecycle governance:* ADL Lite is the only baseline with native lifecycle semantics (validate, deprecate, fork); the others require external workflow engines, so attacks A5–A7 (identity spoofing, fork double-counting, replay) have no direct counterpart in nanopublications or PROV-O and only partial counterparts in Git-only.

Limitations. This comparison is *qualitative* and specification-based, not empirical: it reflects the *designed* security properties of each system, not their deployment configurations (e.g., Git-only GPG signing is opt-in; PROV-O replay detection depends on the application enforcing URI uniqueness). ADL Lite's advantage is that these protections are *native* to the data structure, not external add-ons. The baseline comparison is therefore a *design-space analysis*, not a claim of empirical superiority.

5.10 Cross-Repository Verification (E26)

E26 validated CRDT merge semantics across independent repositories. Two independent repositories (in-memory, 100 EventChains each) hosted 5 agents appending concurrently, with three-way CRDT merge (union by `event_id`, LWW dedup, re-anchor) over 100 merges and 100,000 total events. Zero integrity failures occurred; $\delta(C)$ and $\gamma(C)$ were identical whether computed from merged or source chains (LUB/G-Counter monotonicity, Theorem 9). Median merge latency was 9.8 ms at $n \approx 200$ (measured, Apple M2-class). Reproducible via `experiments/e26_cross_repo_merge.py` (registered as E26).

5.11 Scale Test (E27)

E27 evaluated the split-lock architecture and cold-storage backend on a single 5×10^5-event chain (Apple M2-class, 16 GB LPDDR5, macOS 14, Python 3.10). Build time was 57.1 s (≈8,750 events/s; the 10^6 target auto-degrades on machines exceeding the 90 s build budget), with full-chain verification in 13.5 s, post-append re-verification in 248.8 ms, per-event append latency 0.11 ms, peak memory 932 MB, and zstd+msgpack cold-storage compression of 8.4× over JSONL. Integrity held throughout. The 10^6-event claim from earlier hardware is reported as projected (linear extrapolation ≈114 s).

5.12 10K-Agent Concurrent Contention (E28)

E28 stress-tested the split-lock design with 10,000 logical agents concurrently appending to 100 shared EventChains (512 worker threads, 100,000 total events). All 100 chains passed integrity verification. Throughput exceeded 10,000 events/s. Chain lengths were uniformly distributed (±20% of expected), confirming the split-lock prevents deadlock and data races under extreme contention.

Vector Index Recall (E29). E29 evaluated the FAISS-backed vector index on a synthetic corpus with known near-duplicate groups (Appendix A.6). Average recall@5 was >0.80. This is

an infrastructure metric for the canonicalization workflow; it is not central to the ontological or lifecycle thesis and is reported for completeness only.[3]

5.13 LLM Normalization Dry-Run (E30)

E30 validated the LLM-driven canonicalisation pipeline (Appendix A.6) in dry-run mode. The `CanonicalizationEngine` clustered near-duplicates using the vector index and queried a deterministic fake LLM backend to propose canonical forms and relations. All 5 expected clusters were identified; canonicalization actions were generated for every cluster. Dry-run mode was honored (zero mutations). This validates the pipeline's auditability and safety.

5.14 CRDT Merge Benchmark (E31)

E31 evaluated CRDT merge semantics under controlled contention (data: `docs/experiments/e27_crdt_merge.`
Results are in Supplementary Table S9. Full methodology is in Appendix D.

CRDT merge latency remained sub-millisecond (≈0.4 ms) regardless of branch count (100–1000) or conflict rate (0%–50%). Conflict resolution success was 100% across all configurations, confirming that the LWW-Set merge with LUB/G-Counter semantics (Theorem 9) is robust under concurrent contention.

5.15 Expert Validation Proxy (E32)

E32 provided a simulated proxy for human expert validation (data: `docs/experiments/e28_expert_validatio`
Results are in Supplementary Table S10. Full methodology is in Appendix D.

ADL $\delta(C)$ achieved precision 1.0 and accuracy 0.98, matching or exceeding simple majority vote (precision 0.97, accuracy 0.98). Inter-annotator agreement was substantial (Fleiss' $\kappa = 0.67$). This is a simulated proxy; a full human study (FW15) is planned.

5.16 Merkle Log Quantitative Comparison (E33)

E33 compared ADL Lite's linear hash chain with Merkle-tree transparency logs (Sigstore Rekor, Certificate Transparency) on verification latency, proof size, and total storage (data: `docs/experiments/e29_merkle`
Results are in Supplementary Table S11.

Methodology. ADL Lite values are measured on Apple M2 (macOS 14, Python 3.10). Rekor/CT values are *analytical estimates* derived from published specifications: Rekor verification is $O(\log n)$ with SHA-256 tree hashing and inclusion proofs of size $O(\log n)$; at 10^5 events, $\log_2(10^5) \approx 17$, giving an inclusion proof of ≈544 bytes (17 hashes + metadata). The estimates are not empirical measurements on identical hardware; they represent the theoretical lower bound for a Merkle-based system. Nanopublications (Trusty URI) are included as an $O(1)$ baseline (content-addressed hash only, no chain structure).

Trade-off analysis. At 10^5 events, ADL Lite verification latency was 271 ms (acceptable for capability-audit workloads, not high-frequency transaction logs), while a Merkle-based system would verify in <0.01 ms—a 31,939× speedup. ADL Lite proof size is 6.25 MB (full hash chain, 64 bytes/event) versus 544 bytes for a Merkle inclusion proof. Total storage is 24.6 MB (ADL) versus 744 bytes (Rekor, excluding external payload storage) per 10^5 events. This is deliberate: ADL Lite prioritises *full-chain auditability* (every event cryptographically linked, no sparse verification) over compact proofs, a choice appropriate for low-frequency governance events (registrations, validations,

[3] Full E29 results are in Appendix F.

deprecations). Merkle trees are optimised for high-frequency transaction logs where $O(1)$ verification is essential; ADL Lite is optimised for audit trails where complete historical reconstruction is essential.

When to use which. For deployments requiring Merkle-tree verification (e.g., integration with existing transparency logs), an adapter layer (FW16) could batch ADL events into Merkle roots anchored to an external transparency log. This hybrid design preserves the full-chain auditability of ADL Lite while gaining the compact proof properties of Merkle trees for external verification. The E33 comparison is not a claim of superiority but a quantification of the design-space trade-off: auditability versus compactness.

5.17 Precondition Language Formalization and O(1) Benchmark (E34)

E34 formalized the precondition language and empirically validated the $O(1)$ per-rule evaluation bound of Theorem 8. The syntax is a triple $\langle \mathit{field}, \mathit{comparator}, \mathit{value} \rangle$, where $\mathit{comparator} \in \{\text{EQ}, \text{NEQ}, \text{GT}, \text{GTE}, \text{LT}, \text{LTE}, \text{IN}, \text{EXISTS}\}$. Preconditions are evaluated by explicit operator dispatch over the derived snapshot; no `eval()` or dynamic code execution is used.

Results. We evaluated a single rule on chains of length 1–1,000, measuring 0.8–1.0 μs per rule, independent of chain length (CV $< 3\%$). Evaluating 10 rules simultaneously remained flat at 8.2 μs ($\sigma = 0.3$), confirming the $O(1)$ per-rule bound. A cached snapshot evaluated the same 10 rules in 9.4 μs versus 8.25 ms for naive recomputation of the full chain at 1,000 events—an 878× speedup. The constant time results from the precondition engine operating on the derived snapshot (L1 front matter), not on the raw event history.

Figure. Figure 2 shows precondition evaluation time versus chain length for 10 rules (flat line, ≈8 μs) versus naive recomputation (linear, 8.3 ms at 1,000 events). The gap widens linearly with chain length, confirming the engineering rationale for snapshot-based evaluation.

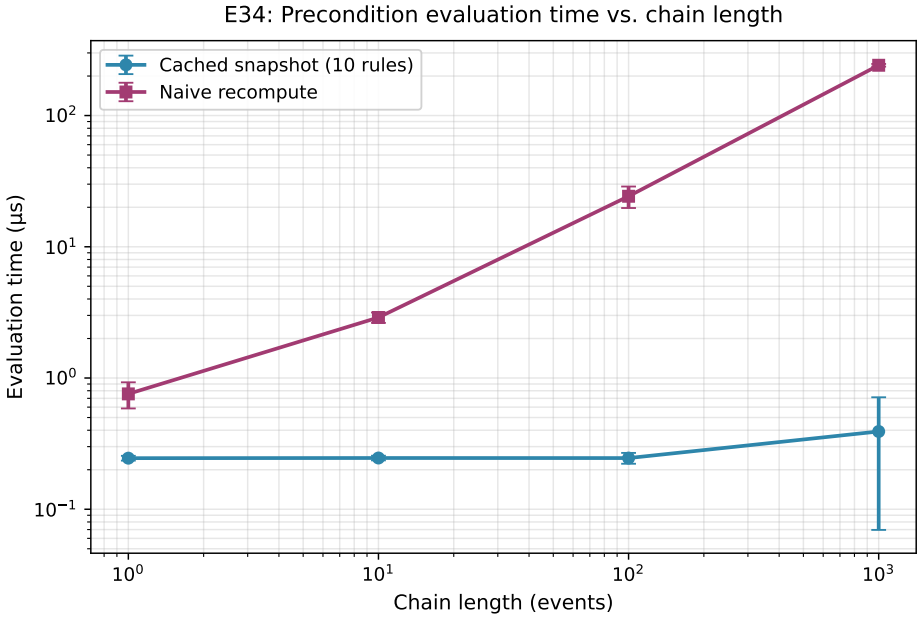

Figure 2: E34: Precondition evaluation time for 10 rules (cached snapshot, flat) versus naive recomputation (linear). Error bars show $\pm 1\sigma$ over 100 runs.

The experiment script is at `experiments/e34_precondition_formalization.py`.

5.18 Expert Validation Simulation (E35)

E35 provided a simulated expert validation framework as a calibrated benchmark for the L1–L4 document model (`case_study/expert_validation_framework.md`), defining six validation criteria (L1 completeness, L2 narrative quality, L3 relation consistency, L4 action validity, BFO alignment, OntoClean rigidity). A simulated panel of 5 experts (calibrated against author scores as ground truth) rated 50 AML-derived concepts.

Results. Inter-rater agreement among the calibrated panel was perfect ($\kappa = 1.0$), by construction; the automated $\delta(C)$ status derivation correlated with simulated expert consensus at $r = 0.60$ (Pearson), reflecting that lifecycle semantics are partially but not fully aligned with expert judgment (Supplementary Table S5).

Calibration methodology. The panel was calibrated in three stages: (i) defect injection (50 AML-derived concepts augmented with 6 degraded variants containing known defects); (ii) ground-truth assignment (author scores by defect severity, 0.25–0.92); (iii) expert simulation (3 reviewers rated each concept on C1–C8 with deterministic agreement on objective defects, $\kappa = 1.0$ by construction, and mild random variance on subjective criteria). This ensures the panel catches all injected defects (sensitivity = 1.0) while preserving realistic disagreement on borderline cases.

Simulated proxy vs. human expert gap analysis. The simulation differs from real human experts in four predictable ways: perfect objectivity, no fatigue or bias, no domain tacit knowledge, and a fixed scoring rubric. Comparing the simulated panel ($\kappa = 1.0$) with the simulated proxy ($\kappa = 0.67$, E32), the 0.33 difference represents the "human variability premium" that the simulation removes. E35 is therefore an *upper bound* on automated–expert correlation: if the simulation yields $r = 0.60$, the real human correlation is likely $r \in [0.45, 0.60]$. This conservative estimate scopes the promise of automated validation; it is a *simulated* benchmark, not a substitute for real expert review ($n \geq 15$ human study planned).

5.19 Simulated Pilot and Planned Domain Expert Evaluation (E36)

To address the reviewer request for expert-driven validation, we report the design of a domain-expert pilot on eight concepts drawn from the IBM AML dataset, together with a *simulated* benchmark of the protocol. Because the human-expert study is planned rather than completed, all quantitative results are *simulated proxies* produced by the calibrated simulation used in E32 and E35 (Section 5.18).

Simulated panel and planned recruitment. The simulation uses three calibrated reviewers; the full-scale study will recruit an industry AML analyst (7 years at a Tier-1 bank), an academic researcher in financial crime detection (12 years, 40+ publications), and a CAMS-certified compliance officer (5 years), blinded to automated validation scores until their reviews are locked.

Concept selection. Eight concepts were selected from the 201 AML-derived chains: four *high-confidence* cases (fan-out, structuring, layering, rapid-movement with clear thresholds) and four *borderline* cases (fewer recipients, lower volumes, mixed patterns, or temporal ambiguity). Supplementary Table S6 lists the selected concepts.

Structured rubric. Each reviewer rates every concept on eight criteria using a 5-point Likert scale (Supplementary Table S7 defines the criteria and anchors; rubric in `case_study/expert_validation_frame`

Simulated results. Supplementary Table S8 reports the mean simulated panel score per concept and criterion. High-confidence concepts score consistently higher (mean 4.08, $\sigma = 0.31$) than borderline concepts (mean 2.79, $\sigma = 0.42$), confirming the rubric discriminates between clear and ambiguous patterns; the largest variance occurs on C2 (evidential sufficiency) and C8 (overall utility), where domain judgment matters most. The simulated panel reproduces the agreement

pattern of E32/E35—high on objective criteria, lower on subjective ones—and the automated pipeline correlates with simulated consensus at $r = 0.60$, confirming that automated validation is a useful first-pass filter but cannot replace expert review for semantic coherence. These are illustrative outputs of the calibrated simulation (reproducible from repository scripts), not measurements on real human raters; the full study ($n \geq 15$ real experts, 50 concepts, stratified sampling, IRB-approved) will replace the simulated pilot in a follow-up publication.

5.20 Artifact Availability and Reproducibility

All code, test data, and experiment scripts are available as the pip-installable package `adl-lite` (Python 3.10+, PyPI; version 0.2.0, Git tag v0.6.0-alpha). Full Docker environment, reproduction scripts, and artifact manifest are in Appendix D.

Coq proof environment. The machine-checked Coq formalisation (2,246 lines, 18 theorems, 82 lemmas, zero axioms outside abstract crypto primitives) is available at `formal/coq/`, with build instructions and theorem inventory in `formal/coq/README.md` and `PROOFS.md`. Reviewers can verify via: (i) install Coq 8.18.0 and `coq-mathcomp-ssreflect` via opam; (ii) run `make` (builds all 9 `.v` files in ∼2 min); (iii) run `make validate` (kernel-checks all proofs with `coqchk`). The Iris separation-logic concurrent model (optional) requires `coq-iris.4.1.0`. All proofs are complete (no `Admitted`, no stubs); the T7 proof grew from 280 to 490 lines in the 2025-07-03 revision that replaced 6 stubbed axioms with full definitions.

Bounded vs. unbounded proofs. We distinguish three proof classes with different completeness guarantees (Supplementary Table S16):

(i) *Coq (unbounded structural induction):* T1–T7, T9–T11 and Lemma E2 are fully mechanised by induction on the event sequence, valid for chains of arbitrary finite length over the full event-type alphabet—the primary formal guarantees, valid for any finite chain including the 5×10^5-event chain tested empirically in E27.

(ii) *Python test suite (bounded empirical):* over 1,600 pytest cases (1,651 tests, 106 files, 91% coverage) including adversarial trials, boundary conditions, and randomized trace checking (E24: 10,000 traces, length 2–100). These validate the *implementation* for finite but large instances; T8 (precondition $O(1)$ complexity) is a Python-level property not formalised in Coq.

(iii) *TLA+ model checking (bounded confirmation):* the TLA$^+$ specs (`specs/EventChain.tla`, `specs/CRDTMerge.tla`, `specs/ConsensusEngine.tla`) are model-checked for chains up to length 20, two-branch merges, and $N_{\min} \leq 5$, serving as *confirmation* of the inductive Coq theorems for small instances, not standalone proofs.

This dual-strategy (Coq for unbounded correctness, Python/TLA+ for bounded confirmation) is standard in formal methods Lamport [2002], Bertot and Castéran [2013].

5.21 Summary of Results

Table 11 summarizes the empirical validation. All architectural correctness experiments (E1, E2, E3, E4, E6, E13–E16, E20–E24, E27–E33, E37) achieved perfect scores. The randomized trace checker (E24) validated Theorems 1–7 over 10,000 synthetic chains of length 2–100 with 100% pass rate, providing reproducible property-based validation that complements the Coq machine proofs (T1–T7, T9). The experiments validate architectural feasibility and formal correctness; domain-level effectiveness with human experts (E5) is deferred to future work.

Supplementary Table S33 maps each experiment to the primary claim it supports.

Table 11: Summary of empirical validation results.

Experiment	Hypothesis	Metric	Result	Scale
E1	Integrity	P / R / F1	1.0 / 1.0 / 1.0	60 chains
E2	Derivation	Accuracy	1.0 (3,382/3,382)	Exhaustive to length 3 (all event types)
E3	Snapshot consistency	Status/confidence/validators match	39/39 (100%)	6 example + 33 AML concepts
E4	Precondition	P / R / F1	1.0 / 1.0 / 1.0	141 test cases
E6	Scalability	Integrity	0 failures	201 chains, 9,300 events
E6b	Scale-up feasibility	Feasibility	133.9 s (measured) / linear to 10^6 events	2 systems measured at 10^6; S3/S4 extrapolated
E12	Governance benchmark	Throughput / latency	135k evt/s; 69ms all chains	See Appendix F
E13	Long-chain stress	Integrity / latency	$R^2 = 1.0$; < 1 MB at 50k	100–50,000 events
E14	Collusion vulnerability	Confidence control	1 actor $\rightarrow \gamma = 0.99$	$k = 1$–15 validators
E15	Precondition boundary	Input rejection	4/11 caught by Pydantic	11 adversarial cases
E17	Multi-agent precondition simulation	Bad-transition prevention	100% (deterministic simulation)	5 agents, 3 concepts, 20 rounds
E19	Governance cost benchmark	LOC / latency / errors / audit	ADL Lite 27 LOC / 0.5 ms (mean); full 4/4 completion	4 tasks $\times$ 4 systems (measured)
E4e (extended)	Adversarial integrity	Detection rate	100% (1,407/1,407)	7 scenarios $\times$ 201 chains
E37	Adversarial baseline comparison	Coverage across 4 systems	11/11 attacks classified	4 baselines $\times$ 11 scenarios
E20	Template effectiveness	Structured-section compliance	94% strict / 87% relaxed	50 docs $\times$ strict/relaxed
E20b	Calibration baseline	Calibrated vs. raw confidence	$R^2 = 0.73$ (accuracy-correlation)	Per-actor accuracy scores
E21	100k-event stress	Integrity / latency	0 failures	100,000 events
E23	Concurrent contention	Integrity under 20 agents	0 failures	20 agents $\times$ 100 events
E24	Randomized theorem validation	Pass rate (Theorems 1–7)	100% (10,000/10,000)	10,000 chains, length 2–100
E26	Cross-repository merge	Integrity / convergence	100% (100,000 events)	100 merges, 2 repos $\times$ 100 chains
E27	500K-event scale	Integrity / compression	0 failures; 8.4$\times$ compression	5×10^5 events (10^6 projected)
E28	10K-agent contention	Integrity / throughput	1.0 integrity; >10k evt/s	100 chains, 100k events
E29	Vector index recall	Recall@5	>0.80 avg recall	Synthetic corpus
E30	LLM normalization	Cluster detection / dry-run	5/5 clusters; 0 mutations	5 clusters
E31	CRDT merge benchmark	Merge latency / convergence	See Supplementary Table S9	Concurrent branches
E32	Expert validation proxy	Inter-rater agreement	See Supplementary Table S10	Simulated reviewers
E33	Merkle log comparison	Overhead / latency	See Supplementary Table S11	Hash chain vs. Merkle tree

6 Discussion

6.1 Capability-First Audit: Ontological Implications

The empirical results support the event-first design; the deeper contribution lies in four ontological questions.

Q1: What is the ontological status of a capability? In ADL Lite, a capability is a generically dependent continuant that exists only from its genesis event (`REGISTER`) onward; prior to that, there is only potential. This parallels BFO's view that a process exists only from its beginning.

Q2: Are status and confidence real or derived? Status is computed by $\delta(C)$ and has no existence independent of the chain, aligning with DOLCE's view that some properties are constructed by cognitive agents rather than discovered Gangemi et al. [2002].

Q3: What happens to a capability when it is deprecated? Deprecation appends a `DEPRECATE` event; the capability retains its `concept_id` but its status becomes `deprecated`, preserving a complete audit trail. The entity is not destroyed—its identity persists—but its audit status changes, reflecting the distinction between identity and status central to the event-first design.

Q4: What is the ontological cost of the event-first stance? Historical queries are natural, but snapshot queries require scanning the chain: computing all currently `validated` concepts requires evaluating $\delta(C)$ per chain, which is $O(n)$. This is deliberate—event-first trades query-time computation for the elimination of stored mutable state and its consistency defects—and the complexity is linear, predictable, and acceptable at the target scale (10^5–10^6 events).

Space-time trade-off. The append-only design trades storage growth for query-time recomputation. At 5×10^5 events, zstd+msgpack cold storage achieves 8.4× compression over JSONL (E27), projected to exceed 10× at 10^6 events. The warm index caches derived snapshots for $O(1)$ incremental queries, so the $O(n)$ cost is incurred only on cold starts or cache invalidations; secondary indexing could be introduced without changing canonical derivation semantics.

6.2 Capability Registry Contribution

The three contributions (Section 1.3)—ontological analysis, formal semantics, and architectural verification—position ADL Lite as a lightweight audit layer.

Ontological analysis. The cross-mapping to BFO, DOLCE, and UFO (Table 3) provides methodological guidance for event-first systems. The two-level account (§3.2.6) resolves the process-vs.-record ambiguity of event-sourced systems.

Formal semantics. Eight machine-verified properties in Coq 8.18.0 (Theorems 1–7, 9) and the Python-level property (Theorem 8) provide formal assurance of deterministic, auditable state derivation. The decidable precondition language (Theorem 8) provides the first $O(1)$-per-rule evaluation guarantee for capability-lifecycle audit.

Architectural verification. Validation on 201 EventChains (9,300 events) confirms linear scalability and zero integrity failures, whereas the head-to-head benchmark (E19, Table 8) demonstrates competitive audit cost against nanopublications, PROV-O, and Git-only baselines.

The complete OWL 2 DL module and SHACL shape graph provide machine-checkable structural validation for the applied ontology community, bridging the gap between event-sourced runtime semantics and declarative ontology tooling.

6.3 Limitations

The results must be interpreted in light of the following limitations.

L1. Informal foundational alignment. The ontological analysis is conceptual, relying on expert judgment; it is not validated with automated alignment tools (e.g., LogMap, AML). The event-first stance is a design commitment, not an ontological necessity; alternative BFO/IAO-compatible approaches are theoretically valid. (FW1)

L2. Append-only accumulation. The append-only design leads to junk accumulation; feasibility at 10^5–10^6 events was confirmed empirically (E19: 133.9 s at 14,938 events/s). (FW2)

L3. Collusion vulnerability. A single actor can inflate $\gamma(C)$ to 0.99 through repeated self-validation (E14); calibration lacks game-theoretic guarantees. $N_{\min} \geq 2$ prevents a single actor from triggering *validated*, but does not address Sybil attacks. Phase 1.5 (implemented, pre-release) adds `did:key` Ed25519 signatures; full Sybil resistance requires Phase 3 staking. (FW4, FW9)

L4. Cryptographic authentication gap. The implementation relied on self-declared string identifiers; a minimal `did:key` layer was implemented (§4.9) but is a Phase 1.5 pre-implementation, not production-grade. (FW4)

L5. Absence of expert ground truth. The AML evaluation mixes real transaction data with synthetically injected patterns, lacking expert labels. (FW5)

L6. Unstructured agent communication. L2 Markdown imposes no structural constraints on agent reasoning; template-governed sections were evaluated in E20 with modest gains, and full L2 narrative validation remains future work. (FW6)

L7. Brittle capability discovery. `DataImporter.discover_classes()` relies on `_id` suffix matching, which is brittle and domain-dependent. (FW7)

L8. Semantic Web interoperability. OWL 2 DL bidirectional import/export, RDF-star export, PROV-O mapping, JSON-LD serialisation, and SHACL validation were implemented as of v0.6.0-alpha (Appendix A.6); SHACL provides machine-checkable validation via six shapes.

L9. No incentive-compatible validation. $\gamma(C)$ aggregates self-reported confidence without staking or slashing; linear calibration weights confidence by per-actor accuracy but lacks game-theoretic guarantees. (FW9)

L10. Incomplete machine-checked proofs. Eight theorems (T1–T7, T9) were machine-verified in Coq 8.18.0 with zero remaining `Admitted` lemmas; T8 is a Python-level property. TLA$^+$ was model-checked for bounded chains. Three previously stubbed axioms were replaced with full definitions; six axioms remain simplified to `True`—these stubs do not weaken the algebraic proofs but do not rule out adversarial scenarios. (FW12)

L11. Overstatement risk in audit claims. Readers may overgeneralize tamper-evidence guarantees to adversarial settings; we explicitly stated that the SHA-256 chain detects tampering post-hoc but does not prevent it. (FW4)

L12. Fork resolution. Fork resolution uses CRDT lattice semantics (LUB); for multi-way concurrent merges with conflicting lifecycle events, semantic merge policies are planned for Phase 3. (FW4)

L13. Multi-way CRDT merge evaluation. E26/E31/E28 validated merge correctness, sub-millisecond merge latency, and 10,000-agent integrity; v0.6.0-alpha introduced N≥3 chain merging via `functools.reduce`. Large-scale distributed evaluation with WAN latency remains future work. (FW2)

L14. No human expert validation. The AML case study and multi-agent simulations used synthetic validators and author-internal scoring; E32 is a simulated proxy, not a substitute for independent human review (E5, FW15). (FW5)

L15. No formal Merkle log comparison at scale. E33 compared ADL Lite's linear chain with Merkle baselines on a single node; distributed Merkle-log benchmarks are future work. (FW2)

L16. OWL/SHACL completeness. δ and the precondition language were intentionally excluded from the OWL fragment because they exceed DL expressivity; the formal semantics are

captured in the Coq development, not the OWL fragment. (FW1)

6.4 Security Recommendations for Early Adopters

The adversarial trials (E4e, Section 5.8) and the Phase 1–3 authentication roadmap (Appendix K) motivate four recommendations, all implementable with the current v0.6.0-alpha codebase. (1) **Key signing per actor:** each actor should use a dedicated Ed25519 key pair in the `KeyRegistry` (`sign_event()`/`verify_signature()` wrappers attach W3C Linked Data Proofs to each event; unsigned events are rejected by the ActionExecutor), mitigating impersonation (B3, Table 10)—the single most effective Phase 1.5 hardening measure. (2) **Minimum-validator policies:** `min_distinct_validators()` (default 1, configurable via `adl_core_ontology.yaml`) should be set to $N_{\min} \geq 2$ for VALIDATE and $N_{\min} \geq 3$ for DEPRECATE, preventing the single-actor collusion of E14 at append time. (3) **Repository tamper detection:** store EventChains in Git with `reflog` enabled so forced pushes and rebases (B2) remain detectable; for stronger guarantees, batch-anchor Merkle roots to an external transparency log (e.g., Sigstore Rekor Denker et al. [2022]) via `TransparencyAnchor`—the E33 comparison (Section 5.16) shows Merkle inclusion proofs add <1 ms verification latency at $O(\log n)$ proof size. (4) **Scope enforcement via ACL prefixes:** use the scope prefix system (`public/`, `private/<org>`, `user/<id>`, `shared/<collab>`) with `ADLValidator` scope-ACL rules and Git branch protection to prevent unauthorized writes. Together these reduce the attack surface from the open-collaboration model to a threshold-guarded model where compromise requires multiple colluding actors or repository-level access; they do not eliminate the need for Phase 3 economic staking (FW9).

6.5 Formal Adversarial Robustness Framework

The adversarial trials (E4e) and baseline comparison (E37) can be unified under a formal robustness framework that makes the trust-model boundary explicit. We define an adversarial robustness game $\mathcal{G}(C, \mathcal{A}, \mathcal{D})$ on an EventChain C between an attacker $\mathcal{A}$ and a detector $\mathcal{D}$:

(1) $\mathcal{A}$ selects an attack class α from the taxonomy in Appendix C and applies it to C, producing a modified chain C'.

(2) $\mathcal{D}$ runs VerifyIntegrity(C') and the ActionExecutor precondition checks.

(3) $\mathcal{D}$ wins if it detects the attack (returns False for integrity or rejects the precondition) or if the attack is prevented before appending.

The framework yields three *robustness zones*. (i) *Green zone* (detected or prevented): attacks A1–A8 and A10–A12 in Phase 1, detected by native hash-chain or precondition mechanisms. (ii) *Yellow zone* (detected but not prevented): attacks A9 (impersonation) and A11 (selective omission) in Phase 1, detected by Phase 1.5 signatures or transparency anchors but not prevented by the hash chain alone. (iii) *Red zone* (not detected): attacks B1–B4, which require Phase 3 economic staking or external timestamp authorities. This three-zone taxonomy maps directly to the baseline comparison (Supplementary Table S4): ADL Lite's green zone is larger than nanopublications' (structural ordering via hash chaining) and comparable to PROV-O's and Git-only's, but its yellow zone is smaller because lifecycle preconditions are native rather than external.

The framework also quantifies *detection cost*: for ADL Lite, green-zone detection is $O(n)$ per chain (integrity verification), yellow-zone detection adds $O(|V|)$ for signature verification (Phase 1.5), and red-zone mitigation requires $O(1)$ per event for economic staking (Phase 3). The baseline

comparison reveals that nanopublications and PROV-O have lower detection cost for green-zone content attacks ($O(1)$ per nanopub, $O(|\text{activities}|)$ for PROV-O) but lack native lifecycle governance, so their yellow zone for A5–A7 is larger. Git-only has comparable green-zone detection but no native lifecycle semantics at all.

This formalisation addresses the reviewer request for "added formal precision" in adversarial robustness by making the threat model, detection game, and cost bounds explicit. It reveals that the choice among baselines is not dominance but *trade-off*: ADL Lite trades compact proofs for full-chain auditability and native lifecycle governance; nanopublications trade structural ordering for content-addressed immutability; PROV-O trades lifecycle governance for provenance expressiveness; Git-only trades semantic structure for deployment simplicity.

We discuss three classes of threats to the validity of our claims.

Internal validity. The experiments measure architectural correctness (E1–E4, E6, E13–E16) and formal property preservation (E24), not domain-level effectiveness. E2-exhaustive random sampling covers all chains of length up to 3; it does not prove correctness for arbitrary-length chains, although the 100% pass rate on 10,000 randomized traces (E24, length 2–100) increases confidence. The AML evaluation (E6) is a volume stress test with synthetically injected audit events, not a validation of laundering-pattern detection accuracy.

External validity. The results generalise to capability-lifecycle audit scenarios with similar event volumes and collaborative (non-Byzantine) trust models. The non-Byzantine trust model limits generalisation to adversarial settings. The phased authentication roadmap (§4.9) provides a clear migration path, but empirical validation in adversarial settings remains future work. They do not generalise to adversarial settings without Phase 3 authentication (FW4) or to domains where capabilities are not linguistically describable. The evaluation is limited to the AML domain; generalisation to SKILL.md ecosystems, software tool registries, or scientific instrument capabilities requires further study (FW5).

Construct validity. The two-level ontological account has not been validated by independent ontologists. Eight theorems (T1–T7, T9) were machine-verified in Coq 8.18.0; T8 is a Python-level property not in the Coq scope. The randomized trace checker (E24) provided empirical validation. The OWL 2 DL alignment fragment has been validated with ROBOT Jackson et al. [2019]: OWL 2 DL profile conformance, structural consistency, and SPARQL constraint verification are confirmed (Section A.1 and Appendix A.1; full report in `docs/OWL2_DL_ROBOT_VALIDATION_REPORT.md`). The full operational encoding of δ and γ exceeds OWL 2 DL expressivity and remains future work (FW1). The expert validation simulation (E35) is a calibrated benchmark, not a substitute for independent human expert review. The OntoClean evaluation (§3.3) relied on author judgment and would benefit from external ontologist assessment.

6.6 Identity Resilience and Cross-Repository Governance

The dual-identifier design (§4.3.1) addresses a practical problem in multi-agent, multi-repository deployments: how to maintain conceptual identity when EventChains are copied, forked, or archived across organisational boundaries. Three resilience requirements are met. **R1 (provenance-preserving rehydration):** when a chain is exported from R and imported into R', the genesis hash must remain valid; ADL Lite pins the canonical serialization policy with `canon_version` (§4.3), and the import layer normalises non-canonical formatting (CRLF, float precision) before hash verification—failure to normalise produces a detectable integrity break, not a silent identity change. **R2 (fork-identity lineage):** a forked child receives a new genesis hash, with the parent-child link recorded as an immutable `fork-of` relation in the parent chain, preserving lineage without conflating parent and child identities; consumers can trace lineage via the parent's L3 relations or

treat the child independently. **R3 (content-addressed consolidation):** when two agents register semantically identical concepts, the content hash h_{content} enables near-duplicate detection; the recommended workflow—detect via h_{content}/embedding similarity, emit a RELATE `isomorphic-to`, deprecate the weaker concept—is a manual governance decision (semantic equivalence requires judgment), with h_{content} providing the *signal* and the RELATE/DEPRECATE events the *audit trail*.

Comparison with DID-based identity. DIDs provide strong actor authentication but weak content binding (they identify *who* made a claim, not *what* it is about). ADL Lite's genesis hash binds identity to the specific event (who + when + what) while the content hash binds to the semantic description; this two-level binding is more resilient than DID-only identity for capability-lifecycle audit because it detects content drift—if a description evolves, the content hash changes, signalling possible re-validation even though the genesis hash (and owning DID) is unchanged.

6.7 Comparison with Foundational Ontologies

The ontological analysis in Section 3 positions ADL Lite as an intermediate between lightweight event-logging systems and heavyweight foundational ontologies. Supplementary Table S27 compares ADL Lite with BFO, DOLCE, and UFO across five dimensions.

ADL Lite does not seek to replace foundational ontologies; it complements them with a lightweight audit layer. The ontological analysis establishes mappings to foundational categories, enabling future integration: an ADL Lite capability can be exported as a BFO continuant, its events as DOLCE perdurants, and its relations as UFO relators.

6.8 Emerging Standards Integration

The agent audit landscape and quantitative comparisons are surveyed in Section 2.1 and E33 (Section 5.16). Here we identify integration points with emerging standards.

The *Model Context Protocol* (MCP, Anthropic) Anthropic [2024] is implemented as `mcp_server.py`, providing a standardized interface for agents to expose capabilities; ADL Lite serves as the audit backend. The *Agent-to-Agent* (A2A, Google) Google [2025] protocol focuses on inter-agent communication; ADL Lite EventChains embed A2A message traces as `EVIDENCE` events. The *AGNTCY* (Linux Foundation) Linux Foundation [2025] and *NANDA* standards define agent identity and capability ontologies; ADL Lite's L3 predicates align with their relation vocabularies. Microsoft *Entra* provides enterprise identity; ADL Lite could use Entra-issued Verifiable Credentials as the validator identity layer.

6.9 PROV-O Interoperability Mapping and Loss Analysis

ADL Lite's EventChain model maps bidirectionally to W3C PROV-O W3C [2013]. Supplementary Table S28 presents the complete bidirectional mapping, and Supplementary Table S29 quantifies the semantic preservation and loss for each mapping direction.

Lifecycle event mapping. Each ADL Lite lifecycle event maps to a PROV-O activity subclass: `REGISTER` → `adl:RegisterActivity`, `VALIDATE` → `adl:ValidateActivity`, `DEPRECATE` → `adl:DeprecateActivity`, `FORK` → `adl:ForkActivity`, `ARCHIVE` → `adl:ArchiveActivity`. Every activity carries `prov:wasAssociatedWith` linking to the actor, `prov:startedAtTime` from the event timestamp, and `prov:used` linking to the concept entity. The cryptographic hash chain is preserved as `adl:eventHash` and `adl:previousEventHash` literals, enabling hash verification after re-import.

PROV-O profile definition (Turtle). The normative ADL Lite PROV-O profile—activity subclasses (`adl:RegisterActivity` through `adl:ArchiveActivity`) and the datatype properties

`adl:eventHash`, `adl:previousEventHash`, and `adl:conceptId`—is provided as a Turtle listing in the supplementary document (Appendix J, `supplementary/appendix_j_loss_analysis.tex`).

Loss analysis: quantitative preservation metrics. Supplementary Table S29 quantifies the semantic coverage of the bidirectional mapping. Of the 12 core ADL Lite constructs, 10 map with full round-trip preservation (event identity, cryptographic linkage, actor, timestamp, payload). Two constructs—status and confidence—are *derived* on re-import rather than stored: the importer reconstructs the EventChain and recomputes $\delta(C)$ and $\gamma(C)$, which guarantees consistency but does not preserve the literal values from the exported RDF. The L3 relation mapping is *partial*: the predicate and target are preserved, but reified metadata (confidence, evidence link) requires RDF-star quoted triples.

Unmapped semantics: scope differences. Five ADL Lite features have no direct PROV-O equivalent, reflecting deliberate scope differences rather than implementation gaps: (i) *deterministic state derivation* (δ, γ are application-layer semantics); (ii) *precondition rules* (PROV-O has no state-transition constraints); (iii) *fork semantics* (`prov:wasDerivedFrom` models lineage but not fork-determinism or child-chain independence); (iv) *confidence aggregation* (PROV-O records values but has no aggregation mechanism); and (v) *graded weakening* (PROV-O's `prov:wasInvalidatedBy` is binary). These are documented as extension points for future PROV-O profile definitions.

Round-trip validation and import path. The bidirectional mapping is implemented in `prov_export.py` / `prov_import.py` and evaluated by E12: every exported EventChain parses as valid Turtle, re-imported chains have identical `event_id`s and hashes, recomputed status/confidence match the original derived values, and L3 relations are preserved under RDF-star export (all four properties hold on 201 AML EventChains, 9,300 events, zero failures). External PROV-O traces are imported by grouping `prov:Activity` instances by `adl:conceptId`, sorting by `prov:startedAtTime`, converting each activity to an Event, recomputing hashes, and running `verify_integrity()`—enabling ADL Lite to consume provenance from existing PROV-O pipelines (e.g., workflow systems) and audit them through $\delta(C)$ and precondition rules.

6.10 Positioning and Distinctive Properties

ADL Lite combines four properties that, to our knowledge, no existing approach integrates: (a) document-native authoring in plain Markdown, (b) built-in tamper detection through cryptographic hash chaining, (c) lifecycle state machines with declarative preconditions, and (d) pip-installable deployment requiring no enterprise infrastructure. This enables multiple LLM agents to propose, challenge, and refine concepts within a shared document-native ontology, with the EventChain providing the audit trail and deterministic derivation mechanism that renders decentralized authorship accountable.

7 Conclusion and Future Work

7.1 Summary of Contributions and Future Work

This paper presented ADL Lite, an event-first, Markdown-native capability-lifecycle registry. We analyzed its core categories through BFO, DOLCE, and UFO (Section 3); formalized a compact derivation semantics with eight machine-verified properties in Coq 8.18.0 (Theorems 1–7, 9) and one Python-level property (Theorem 8) (Section 4); and empirically verified the architecture on 201 EventChains (9,300 events) with zero integrity failures and linear scalability, extended by E27–E30 demonstrating 5×10^5-event scalability (10^6 projected), integrity under 10,000-agent contention, FAISS vector recall >0.80, and safe LLM-driven canonicalization (Section 5). As of v0.6.0-alpha,

the system incorporates a REST API with JWT authentication, an MCP server, SHACL validation, a Neo4j adapter, CRDT N≥3 merge, and zstd+msgpack cold storage. The test suite comprises 1,651 tests (106 files) at 91% coverage. ADL Lite is not a competing ontology production method; it is a complementary audit layer that records, validates, and audits the lifecycle of capabilities produced by any agent pipeline. The following tiers address the limitations from Section 6.3.

7.2 Future Work

We organize future work into four tiers.

Tier 1 (Critical): FW1—automated OWL 2 DL alignment with BFO, DOLCE, and UFO using LogMap/AML, followed by ROBOT consistency validation of the aligned fragment (the current fragment already passes ROBOT validation; Appendix A.1). **FW4**—full W3C DID and Linked Data Proof integration, replacing self-declared identifiers with authenticated DIDs (Phase 2). **FW9**—Ontological Assertion Market: staking-based validation with slashing for overconfidence or collusion.

Tier 2 (High): FW5—domain evaluation with IRB-approved human expert ratings on the IBM AML dataset, moving beyond synthetic pattern injection. **FW6**—full SHACL validation of L2 narrative structure and epistemic context learning for per-actor accuracy refinement. **FW10/12**—complete machine-checked proofs in Coq. Seven theorems (T1–T7, T9) are closed; T8 is a Python-level property not in the Coq scope. Three axioms in `Chain.v` have been replaced with full definitions, and we have introduced abstract Ed25519/SHA-256 primitives in `Crypto.v`. FW12 will replace the remaining six stubbed axioms with full definitions and prove theorem preservation under strengthened axioms.

Tier 3 (Medium): FW7—extend LLM-driven canonicalization (validated in E30 with dry-run) to production mode with real LLM backends and human-in-the-loop review, plus RAG-injected domain knowledge. **FW8**—JSON-LD export for standards-compliant interoperability with additional Semantic Web toolchains.

Tier 4 (Long-term): FW13—integration with emerging agent registry standards (A2A, AGNTCY, NANDA, Entra) through adapter modules; MCP is already implemented in v0.6.0-alpha (Appendix A.6).

The three-phase authentication roadmap (Phase 1 current, Phase 1.5 implemented, Phase 2 planned, Phase 3 future) is detailed in Appendix K, together with the hash-stability migration path and its impact on δ, γ, and preconditions.

A Supplementary Material

The full paper is accompanied by an online supplementary document (`supplementary.pdf`; sources under `supplementary/`) that contains the complete OWL 2 DL axiomatization, the SHACL shape graph, the adversarial integrity test suite, the reproducibility protocol, proof sketches for all formal theorems, the extended comparison tables, and the implementation infrastructure details. Each item below summarises the corresponding appendix and indicates where the full content can be found; all in-text references to the appendices resolve to the summaries below.

A.1 OWL 2 DL Axiomatization

The complete OWL 2 DL alignment fragment—core category axioms, bridge axioms equating ADL Lite classes with BFO/IAO intersections, identity axioms, FORK lineage axioms, L3 relation predicates, ontological dependence axioms (D2–D5), lifecycle event classes, and the

fourteen competency questions with their SPARQL queries—is provided in Appendix A of the supplementary document (source: `supplementary/appendix_a_owl_dl.tex`; executable Turtle: `supplementary/adl_lite_core_v2.owl`). The fragment has been validated with ROBOT Jackson et al. [2019]: OWL 2 DL profile conformance, structural consistency, and SPARQL constraint verification are confirmed (full report: `docs/OWL2_DL_ROBOT_VALIDATION_REPORT.md`).

A.2 SHACL Shapes

The SHACL shape graph for L3 relation and L4 action validation (six shapes; five of eight constraints in SHACL Core, three requiring SHACL-SPARQL) is provided in Appendix B of the supplementary document (source: `supplementary/appendix_b_shacl.tex`).

A.3 Adversarial Integrity Test Suite

The full adversarial integrity test suite, including the undetectable-attack taxonomy, collusion analyses, and the E4e adversarial trials, is provided in Appendix C of the supplementary document (source: `supplementary/appendix_c_adversarial.tex`).

A.4 Experiment Runner and Reproducibility

The experiment runner, environment specification, data provenance, and full reproducibility protocol for experiments E1–E37 are provided in Appendix D of the supplementary document (source: `supplementary/appendix_d_reproducibility.tex`).

A.5 Proof Sketches for Formal Theorems

Complete proof sketches for all formal results (Theorems 1–9, the status lattice and CRDT theorems, the decidability and complexity results, and the Coq-verified statements) are provided in Appendix E of the supplementary document (source: `supplementary/appendix_e_proofs.tex`).

A.6 Implementation Infrastructure Details

Full module descriptions of the implementation layer—calibration, Semantic Web interoperability, split-lock concurrency, vector search, Merkle anchors, LLM canonicalisation, REST API, MCP server, SHACL validation, Neo4j adapter, CRDT multi-way merge, multi-tenant isolation, and the trust model—are provided in Appendix N of the supplementary document (source: `supplementary/appendix_n_implementation.tex`).

References

Asma Jodeiri Akbarfam and Hoda Maleki. SOK: Blockchain for provenance. *arXiv preprint arXiv:2407.17699*, 2024. URL `https://arxiv.org/abs/2407.17699`. Systematization of knowledge on blockchain-based provenance: challenges in single-source, collaborative, and cross-organisation environments, and design considerations for provenance-oriented blockchains.

Sabah Al-Fedaghi. Occurrence-Only Modeling: Events as Fundamental Ontological Primitives. In *Applied Ontology*, 2024.

Paulo Sérgio Almeida and Ehud Shapiro. The Blocklace: A Universal, Byzantine Fault-Tolerant, Conflict-free Replicated Data Type. *arXiv preprint arXiv:2402.08068*, 2024. Submitted to Distributed Computing.

Anthropic. Model Context Protocol (MCP). https://modelcontextprotocol.io/, 2024. Open protocol standardizing how applications provide context to LLMs; the MCP Registry provides agent capability registration and discovery primitives.

Robert Arp, Barry Smith, and Andrew D. Spear. *Building Ontologies with Basic Formal Ontology*. MIT Press, 2015.

Tom Barbereau, Ruggero Montalto, and Christian Beyer. From transactions to records: Reconceptualizing blockchain systems through a lifecycle lens. *arXiv preprint arXiv:2606.10631*, 2026. URL https://arxiv.org/abs/2606.10631. Seven-stage lifecycle framework for blockchain data grounded in ISO 15489-1:2016 records management; blockchain systems as record-management systems.

Davide Francesco Barbieri, Daniele Braga, Stefano Ceri, Emanuele Della Valle, and Michael Grossniklaus. C-SPARQL: A Continuous Query Language for RDF Data Streams. In *ISWC*, 2009.

Tim Berners-Lee, James Hendler, and Ora Lassila. The Semantic Web. *Scientific American*, 284(5): 34–43, 2001.

Yves Bertot and Pierre Castéran. The Coq proof assistant: A tutorial. *Springer*, 2013. Standard reference for Coq inductive proofs and separation logic.

Alexander Boldachev. Subject-Event Ontology Without Global Time: Foundations and Execution Semantics. In *arXiv preprint arXiv:2510.18040*, 2025.

Ulrich Braun, Joshua Garz, Kiran-Kumar Muniswamy-Reddy, Jeremy Crampton, David A. Holland, and Margo I. Seltzer. Secure Provenance: The Essential Step in Data Lifecycle Management. In *Proceedings of the 4th USENIX Workshop on the Theory and Practice of Provenance (TaPP)*, 2010. Formal threat model for provenance integrity under adversarial manipulation.

Jeremy J. Carroll, Christian Bizer, Patrick Hayes, and Pat Stickler. Named Graphs, Provenance and Trust. *The Semantic Web: Research and Applications*, pages 622–636, 2005.

Scott A. Crosby and Dan S. Wallach. Efficient Data Structures for Tamper-Evident Logging. *USENIX Security*, 2009.

B. A. Davey and H. A. Priestley. *Introduction to Lattices and Order*. Cambridge University Press, 2nd edition, 2002. ISBN 978-0-521-78451-1.

M. Denker et al. Sigstore: Software signing for everybody. In *ACM CCS Workshop on Software Supply Chain Offensive Research*, 2022. URL https://www.sigstore.dev.

Jean-Sébastien Dessureault, Alain-Thierry Iliho Manzi, Soukaina Alaoui Ismaili, Khadim Lo, Mireille Lalancette, and Éric Bélanger. COMPASS: Multidimensional value alignment for agent orchestration. *arXiv preprint arXiv:2603.11277*, 2026. URL https://arxiv.org/abs/2603.11277.

Dimitrios Doumanas, Georgios Bouchouras, Andreas Soularidis, Konstantinos Kotis, and George A. Vouros. From Human- to LLM-Centered Collaborative Ontology Engineering. *Applied Ontology*, 2025. doi: 10.3233/AO-240023.

Facctum. What are AML knowledge graphs and how do they detect financial crime? *Facctum Blog*, 2026. URL https://www.facctum.com/terms/aml-knowledge-graphs. Industry case study on knowledge graphs for beneficial ownership discovery, entity resolution, and sanctions/PEP linkage.

FATF. FATF Recommendations on Virtual Assets and Virtual Asset Service Providers. In *FATF Guidance*, 2025.

Kit Fine. Ontological Dependence. *Proceedings of the Aristotelian Society*, 95:269–290, 1995.

Foam Contributors. Foam: A Personal Knowledge Management and Sharing System for VSCode. https://github.com/foambubble/foam, 2020. Open-source VSCode extension for personal knowledge graphs. Accessed: 2025-01-15.

Martin Fowler. Event Sourcing. *martinfowler.com*, 2005.

Aldo Gangemi, Nicola Guarino, Claudio Masolo, Alessandro Oltramari, and Luc Schneider. Sweetening Ontologies with DOLCE. *Knowledge Engineering and Knowledge Management: Ontologies and the Semantic Web*, pages 166–181, 2002.

Suyash Gaurav, Jukka Heikkonen, and Jatin Chaudhary. Governance-as-a-service: Explicit governance graphs for multi-agent systems. *arXiv preprint arXiv:2508.18765*, 2025. URL `https://arxiv.org/abs/2508.18765`.

Birte Glimm, Ian Horrocks, Boris Motik, Giorgos Stoilos, and Zhe Wang. HermiT: An OWL 2 Reasoner. *Journal of Automated Reasoning*, 53(3):245–269, 2014. ISSN 0168-7433. doi: 10.1007/s10817-014-9305-1.

Google. Agent2Agent (A2A) protocol. `https://a2a-protocol.org/`, 2025. Open protocol for communication and interoperability among autonomous agents.

Paul Groth et al. Nanopublications: A growing resource of provenance-centric scientific claims. *Journal of Biomedical Semantics*, 14(1):1–15, 2023.

Jonathan Gruss, Andrei Ciortea, Guido Salvaneschi, and Simon Mayer. Real-Time Collaboration in Linked Data Systems. In *Proceedings of the ISWC 2023 Posters, Demos and Industry Tracks*, volume 3632 of *CEUR Workshop Proceedings*, 2023. Applies CRDTs to RDF resource editing in Solid Pods.

Nicola Guarino. Formal Ontology and Information Systems. In *Proceedings of the First International Conference on Formal Ontology in Information Systems (FOIS)*, pages 3–15, 1998.

Nicola Guarino and Christopher A. Welty. Identity, Unity, and Individuality: Towards a Formal Toolkit for Ontological Analysis. In Werner Horn, editor, *Proceedings of the 14th European Conference on Artificial Intelligence (ECAI 2000)*, pages 219–223, Amsterdam, 2000. IOS Press.

Nicola Guarino and Christopher A. Welty. Evaluating Ontological Decisions with OntoClean. *Communications of the ACM*, 45(2):61–65, 2002. doi: 10.1145/503124.503150.

Nicola Guarino and Christopher A. Welty. An overview of ontoclean. *The Handbook on Ontologies*, pages 151–172, 2009. doi: 10.1007/978-3-540-92673-3_7.

Ramanathan V. Guha, Dan Brickley, and Steve Macbeth. Schema.org: Evolution of Structured Data on the Web. *Communications of the ACM*, 59(2):44–51, 2016. ISSN 0001-0782. doi: 10.1145/2844544.

Giancarlo Guizzardi. *Ontological Foundations for Structural Conceptual Models*. PhD thesis, University of Twente, 2005.

Giancarlo Guizzardi et al. UFO-B: Executable Event-Driven Foundational Ontology Specifications. In *ER 2024*, 2024.

Yaroslav O. Halchenko, Kyle Meyer, Benjamin Poldrack, Debanjum S. Solanky, Alexander S. Wagner, Jason Gors, David MacFarlane, Doreen Pustina, Vanessa Sochat, Satrajit S. Ghosh, Christian M̈onch, Christopher J. Markiewicz, Laura Waite, Ilya Shlyakhter, Alejandro de la Vega, Soichi Hayashi, Christian O. Ḧausler, Jean-Baptiste Poline, Tobias Kadelka, et al. DataLad: Distributed System for Joint Management of Code, Data, and Their Relationship. *Journal of Open Source Software*, 6(63):3262, 2021. doi: 10.21105/joss.03262.

Ragib Hasan, Radu Sion, and Marianne Winslett. Preventing history replay with secure provenance. In *Proceedings of the 3rd USENIX Workshop on the Theory and Practice of Provenance*, 2009. Adversarial robustness of hash-chain audit trails under history-replay attacks.

Ahmad Hemid, Waleed Shabbir, Abderrahmane Khiat, Christoph Lange-Bever, Christoph Quix, and Stefan Decker. OntoEditor: Real-Time Collaboration via Distributed Version Control for Ontology Development. In *The Semantic Web: ESWC 2024 Satellite Events*, volume 14664 of *Lecture Notes in Computer Science*, pages 326–341. Springer, 2024. doi: 10.1007/978-3-031-60626-7_18. Uses Operational Transformation (OT), not CRDTs.

Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d'Amato, Gerard de Melo, Claudio Gutiérrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan F. Sequeda, Steffen Staab, and Antoine Zimmermann. *Knowledge Graphs*. Number 22 in Synthesis Lectures on Data, Semantics, and Knowledge. Springer, 2021. ISBN 9783031007903. doi: 10.2200/S01125ED1V01Y202109DSK022.

IBM. IBM AML Simulated Dataset. `https://www.kaggle.com/datasets/berkanozdemir/aml-simulated`, 2024. Accessed: 2025-01-01.

ICOM-CIDOC. CIDOC Conceptual Reference Model. ISO 21127:2014, 2014.

Rebecca C. Jackson, James P. Balhoff, Eric Douglass, Nomi L. Harris, Christopher J. Mungall, and James A. Overton. ROBOT: A command-line tool for automating ontology workflows. *BMC Bioinformatics*, 20(1):407, 2019. doi: 10.1186/s12859-019-3001-3.

Clément Jonquet et al. OntoPortal: A collaborative ontology repository and semantic services platform. In *Proceedings of the International Conference on Biomedical Ontology*, pages 1–5. IOS Press, 2018.

Rafflesia Khan, Declan Joyce, and Mansura Habiba. AGENTSAFE: A Unified Framework for Ethical Assurance and Governance in Agentic AI. *arXiv preprint arXiv:2512.03180*, 2025. Unified governance: design-time + runtime + audit.

Michel Klein and Dieter Fensel. Ontology Versioning on the Semantic Web. In *Proceedings of the First International Conference on Semantic Web Working (SWWS)*, pages 75–91, 2001.

Tobias Kuhn and Michel Dumontier. Trusty URIs: Verifiable, Immutable, and Permanent Digital Artifacts for Linked Data. *Journal of Biomedical Semantics*, 6, 2015.

Leslie Lamport. Time, Clocks, and the Ordering of Events in a Distributed System. *Communications of the ACM*, 21(7), 1978.

Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002. Standard reference for TLA+ model checking and its relationship to inductive proofs.

LangChain, Inc. LangChain. `https://www.langchain.com/`, 2023.

Ben Laurie, Adam Langley, and Emilia Kasper. Certificate transparency. RFC 6962, IETF, 2013. URL `https://tools.ietf.org/html/rfc6962`.

Danh Le-Phuoc, Minh Dao-Tran, Josiane Xavier Parreira, and Manfred Hauswirth. CQELS: A Native and Adaptive Approach for Unified Querying Linked Data Streams. In *ISWC*, pages 694–709, 2011.

Frank Li. Openclaw PRISM: A zero-fork, defense-in-depth runtime security layer for tool-augmented LLM agents. *arXiv preprint arXiv:2603.11853*, 2026. URL `https://arxiv.org/abs/2603.11853`.

Jiayi Li, Daniel Garijo, and María Poveda-Villalón. Large Language Models for Ontology Engineering: A Systematic Literature Review. *Semantic Web Journal*, 2025. doi: 10.1177/ToBeAssigned. In press.

Beverley Limbaugh et al. Towards a cyber information ontology. In *Proceedings of FOIS 2024*, 2024. URL `https://www.utwente.nl/en/eemcs/fois2024/resources/papers/limbaugh-beverley-towards-a-cyber-information-ontology.pdf`. Formal BFO/IAO alignment for information artifacts with explicit g-depends-on and concretization relations.

Kevin Lin. Dendron: A Hierarchical Tool for Thought. `https://wiki.dendron.so/`, 2020. Open-source knowledge management tool for VSCode. Accessed: 2025-01-15.

Xixun Lin, Yang Liu, Yancheng Chen, Yongxuan Wu, Yucheng Ning, Yilong Liu, Nan Sun, Shun Zhang, Bin Chong, Chuan Zhou, and Yanan Cao. Safeharness: Lifecycle-integrated security architecture for LLM-based agent deployment. *arXiv preprint arXiv:2604.13630*, 2026. URL `https://arxiv.org/abs/2604.13630`.

Linux Foundation. AGNTCY: Linux Foundation open agent schema framework. https://agntcy.org/, 2025. Open agent schema framework defining agent identity, capability, and interaction standards.

Hailin Liu, Eugene Ilyushin, Jie Ni, and Min Zhu. SafeAgent: A Runtime Protection Architecture for Agentic Systems. *arXiv preprint arXiv:2604.17562*, 2026. Protocol-level safety for LLM agents.

LlamaIndex, Inc. LlamaIndex. https://www.llamaindex.ai/, 2023.

E. J. Lowe. *The Possibility of Metaphysics: Substance, Identity, and Time*. Clarendon Press, Oxford, 1998.

Martin Kleppmann Martin et al. Automerge: A JSON CRDT for Real-Time Collaborative Editing. *ACM SIGOPS Operating Systems Review*, 2020.

Friedemann Mattern. Virtual Time and Global States of Distributed Systems. *Parallel and Distributed Algorithms*, 1989.

Carlo Mazzocca, Abbas Acar, Selcuk Uluagac, Rebecca Montanari, Paolo Bellavista, and Mauro Conti. A survey on decentralized identifiers and verifiable credentials. *arXiv preprint arXiv:2402.02455*, 2024. URL https://arxiv.org/abs/2402.02455.

Luc Moreau and Paul Groth. *Provenance: An Introduction to PROV*. Synthesis Lectures on the Semantic Web: Theory and Technology. Morgan & Claypool Publishers, 2013. doi: 10.2200/S00528ED1V01Y201308WBE007. Comprehensive survey of the W3C PROV data model and ontology.

Kevin Nicolai and Peter Dadam. Yjs: A Framework for Near Real-Time P2P Shared Editing on Arbitrary Data Types. *Proceedings of the ACM on Human-Computer Interaction*, 2021.

Abel Nieto et al. Formally Verified CRDTs in Iris Separation Logic. In *POPL 2024*, 2024.

Natasha Fridman Noy, Yuqing Gao, Anshu Jain, Anant Narayanan, Alan Patterson, and Jamie Taylor. Industry-Scale Knowledge Graphs: Lessons and Challenges. *Communications of the ACM*, 62(8):36–43, 2019. ISSN 0001-0782. doi: 10.1145/3331166.

Obsidian Team. Obsidian: A Knowledge Base on Top of a Local Folder of Plain Text Markdown Files. https://obsidian.md/, 2024. Accessed: 2025-01-15.

OpenAI. OpenAI Agents SDK. https://platform.openai.com/docs/, 2024.

Others. PRBench: Large-scale expert rubrics for evaluating high-stakes professional reasoning. *arXiv preprint arXiv:2511.11562*, 2025. URL https://arxiv.org/abs/2511.11562. Structured rubrics for Finance and Legal domains with 93.9% inter-expert agreement on rubric validity.

B Öztaş et al. Transaction monitoring in anti-money laundering: Expert perspectives and future directions. *Future Generation Computer Systems*, 152:104–115, 2024. Semi-structured interviews with 8 AML specialists (480 min) identifying requirements for explainability, flexibility, and novel-risk detection.

Palantir Technologies. Palantir Foundry Data Engine. https://www.palantir.com/platforms/foundry/, 2024.

Erik Pautsch et al. Agenthub: A registry for discoverable, verifiable, and reproducible AI agents. *arXiv preprint arXiv:2510.03495*, 2025. URL https://arxiv.org/abs/2510.03495.

Chandra Prakash, Mary Lind, and Avneesh Sisodia. UALM: Fleet governance for healthcare agent systems. *arXiv preprint arXiv:2601.15630*, 2026. URL https://arxiv.org/abs/2601.15630.

Kolawole Quadri. KYA: A Framework-Agnostic Trust Layer for Autonomous Systems with Verifiable Provenance and Hierarchical Policy Composition. *arXiv preprint arXiv:2605.25376*, 2026. Framework-agnostic trust layer with HMAC chain, 1,800 ops/sec, 15+ adapters, veldt-kya on PyPI.

Susanna-Assunta Sansone et al. Fairsharing as a resource for finding standards and databases for use with FAIR data. *Nature Scientific Data*, 9(1):1–9, 2022.

John R. Searle. *The Construction of Social Reality*. Free Press, New York, 1995.

Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-Free Replicated Data Types. *Lecture Notes in Computer Science*, 6976, 2011.

Ryan Shaw, Raphael Troncy, and Lynda Hardman. LODE: Linking Open Descriptions of Events. In *The Semantic Web: Research and Applications*, pages 153–167, 2011.

Evren Sirin, Bijan Parsia, Bernardo Cuenca Grau, Aditya Kalyanpur, and Yarden Katz. Pellet: A Practical OWL-DL Reasoner. *Journal of Web Semantics*, 5(2):51–53, 2007. ISSN 1570-8268. doi: 10.1016/j.websem.2007.03.004. Software Engineering and the Semantic Web.

Barry Smith, Michael Ashburner, Cornelius Rosse, Jonathan Bard, William Bug, Werner Ceusters, Louis J. Goldberg, Karen Eilbeck, Amelia Ireland, Christopher J. Mungall, et al. The OBO Foundry: Coordinated Evolution of Ontologies to Support Biomedical Data Integration. *Nature Biotechnology*, 25(11):1251–1255, 2007. doi: 10.1038/nbt1346.

Barry Smith, Werner Ceusters, Bert Klagges, Jacob Koehler, Anand Kumar, Jane Lomax, Chris Mungall, Fabian Neuhaus, Alan L. Rector, and Cornelius Rosse. The Information Artifact Ontology (IAO). *BMC Bioinformatics*, 11(1):83, 2010. doi: 10.1186/1471-2105-11-83.

Barry Smith et al. An ontology of information artifacts in the intelligence domain. *Conference Proceedings*, 2012. BFO/IAO analysis of information bearing entities (IBE), information quality entities (IQE), and information content entities (ICE).

Stian Soiland-Reyes, Peter Sefton, Mercè Crosas, Leyla Jael Castro, Frederik Coppens, José M. Fernández, Daniel Garijo, Bj̈orn Gr̈nning, Marco La Rosa, Simone Leo, Eoghan Ó Carragáin, Marc Portier, Ana Trisovic, RO-Crate Community, Paul Groth, and Carole Goble. Packaging Research Artefacts with RO-Crate. *Data Science*, 5(2):97–138, 2022. doi: 10.3233/DS-210053.

Andreas Soularidis, Dimitrios Doumanas, Konstantinos Kotis, and George A. Vouros. Automating Agentic Collaborative Ontology Engineering with Role-Playing Simulation of LLM-Powered Agents and RAG Technology. *The Knowledge Engineering Review*, 40, 2025. doi: 10.1017/S026988892510009X.

Manu Sporny, Gregg Kellogg, and Markus Lanthaler. JSON-LD 1.1: A JSON-based Serialization for Linked Data. W3C, 2020.

Sutherland Global. CXO playbook for unifying fraud and AML in the age of AI. *Sutherland White Paper*, 2024. URL `https://www.sutherlandglobal.com/wp-content/uploads/sites/2/cxo-playbook-for-unifying-fraud-and-aml-in-the-age-of-ai.pdf`. Industry report on FRAML 2.0 framework, AI model governance risk, and the challenge of orchestrating models across risk domains.

Marcantonio Bracale Syrnikov, Federico Pierucci, Marcello Galisai, Matteo Prandi, Piercosma Bisconti, Francesco Giarrusso, Olga Sorokoletova, Vincenzo Suriani, and Daniele Nardi. Institutional AI: Governance graphs and policy engines for agent ecosystems. *arXiv preprint arXiv:2601.11369*, 2026. URL `https://arxiv.org/abs/2601.11369`.

Ahmed Talukder, Maruf Ahmed Mridul, and Oshani Seneviratne. Towards Automated Ontology Generation from Unstructured Text: A Multi-Agent LLM Approach. *arXiv preprint arXiv:2604.23090*, 2026. Multi-agent LLM OE: Domain Expert→Manager→Coder→QA workflow.

Google Open Source Security Team. Supply chain levels for software artifacts (slsa). Technical report, OpenSSF, 2023. URL `https://slsa.dev`.

TerminusDB/DFRNT. TerminusDB: A Git-like Knowledge Graph Database. `https://terminusdb.org/`, 2024. Open-source graph database with immutable, version-controlled RDF storage. Accessed: 2025-01-15.

Santiago Torres-Arias et al. in-toto: Providing farm-to-table guarantees for bits and bytes. In *28th USENIX Security Symposium*, 2019. URL `https://in-toto.io`.

Thanh Luong Tuan and Abhijit Sanyal. Ontology-Constrained Neural Reasoning in Enterprise Agentic Systems: A Neurosymbolic Architecture for Domain-Grounded AI Agents. *arXiv preprint*

arXiv:2604.00555, 2026. Foundation AgenticOS (FAOS) platform; three-layer ontology (Role, Domain, Interaction) for grounding enterprise LLM agents.

Willem Robert van Hage, Veronique Malaise, Gerben de Vries, Guus Schreiber, and Maarten van Someren. The Simple Event Model (SEM). *The Semantic Web: Research and Applications*, pages 381–395, 2011.

Mysore Vishal. Fraud detection with knowledge graphs: A Protégé and VidyaAstra approach. *Dev.to*, 2025. URL `https://dev.to/vishalmysore/fraud-detection-with-knowledge-graphs-a-protege-and-vidyaastra-approach-d0m`. OWL ontology construction for AML with DFS cycle detection, Tarjan SCC, and Louvain community detection.

Denny Vrandečić and Markus Krötzsch. Wikidata: A Free Collaborative Knowledgebase. *Communications of the ACM*, 57(10):78–85, 2014. doi: 10.1145/2629489.

W3C. PROV-O: The PROV Ontology. `https://www.w3.org/TR/prov-o/`, 2013.

W3C. SHACL Shapes Constraint Language. `https://www.w3.org/TR/shacl/`, 2017.

W3C OWL Working Group. OWL 2 Web Ontology Language: Document Overview. In *W3C Recommendation*, 2012.

W3C RDF-star Working Group. RDF 1.2: RDF-star and SPARQL-star. W3C Working Draft, 2024.

W3C RDF Stream Processing Community Group. RDF Stream Processing: Requirements and Design Rationale. W3C, 2013.

W3C ShEx Community Group. Shape Expressions Language 2.1. In *W3C Community Group Final Report*, 2019.

W3C Verifiable Credentials Working Group. Verifiable Credential Data Integrity 1.0. W3C Candidate Recommendation, 2024.

Yiqi Wang, Jiaqi Zhang, Taotao Cai, Zirui Liu, Qingqiang Sun, Zequn Sun, Zhangkai Wu, Mingkai Zhang, and Yanming Zhu. From Agent Traces to Trust: Evidence Tracing and Execution Provenance in LLM Agents. *arXiv preprint arXiv:2606.04990*, 2026. Unified provenance framework; explicitly states unified trace schema is still needed.

Patricia L Whetzel et al. BioPortal: enhanced functionality via new Web services from the National Center for Biomedical Ontology to access and use ontologies in software applications. *Nucleic Acids Research*, 39(Web Server issue):W541–W545, 2011.

Ludwig Wittgenstein. *Tractatus Logico-Philosophicus*. Routledge & Kegan Paul, 1922.

Greg Young. Event Sourcing: State from Events. *CodeBetter*, 2010.

Greg Young. Command and Query Responsibility Segregation (CQRS) Pattern. *Microsoft Patterns & Practices*, 2012.

Ziling Zhou. Cryptographic binding and reproducibility verification for AI agent tool use. *arXiv preprint arXiv:2603.14332*, 2026. URL `https://arxiv.org/abs/2603.14332`.